dbt from Zero to Expert

A Hands-On Learning Guide for VS Code + Databricks

Complete tutorial, lab workbook, and interview preparation

Edition 1.0 | October 2026 | dbt Core 1.12 and dbt v2 aware

How to Use This Guide

This is a complete, hands-on path from "I have never used dbt" to "I can design, build, test, deploy, govern, and explain a production dbt project on Databricks." It is written for your environment: **Windows + VS Code + Databricks + dbt-databricks**, with examples that run against Unity Catalog.

The guide is intentionally both a tutorial and a reference. Read Chapters 1-11 in order, do the labs, then use the later chapters as a field manual. Every chapter ends with exercises; Part IX is a full capstone; Part X contains 100+ interview questions with answers, cheat sheets, a glossary, and a study plan.

The environment this guide targets

Item	Target used in this book	Notes
Operating system	Windows 10/11, PowerShell	Commands are PowerShell-friendly
Editor	VS Code	dbt extension, Databricks extension, Python, YAML
Warehouse	Databricks (Unity Catalog, SQL warehouse)	Works with serverless or pro SQL warehouse
dbt engine	dbt Core 1.12.x + dbt-databricks 1.12.x	The version found in your project
Newer engine	dbt v2 (Rust/Fusion)	Covered in Chapter 17; syntax is largely compatible
Language	SQL + Jinja + limited Python models	PySpark for Python models
Catalog placeholder	`analytics_lab`	Replace with your own catalog, for example `clearpay`
Schemas	`bronze`, `silver`, `gold`, plus per-developer schemas	Medallion layout

> **Version note.** dbt Labs now ships two major lines: **dbt v1.x** (the Python-based engine, including the 1.12 release) and **dbt v2** (the Rust-based engine formerly called Fusion, installed with `pip install dbt`). Most project syntax is shared. This guide teaches the concepts and code that work on both, calls out differences, and uses your installed 1.12 line for the labs so that everything runs today.

How each chapter is organized

Each chapter follows the same rhythm:

1. **Why it matters** - the mental model and the business problem.
2. **How dbt works** - the mechanics, configuration, and SQL/Jinja.
3. **Do it** - copy-paste examples for Databricks.
4. **Production notes** - performance, cost, security, and failure modes.
5. **Exercises** - short tasks, with answers or expected results.
6. **Interview check** - the questions an interviewer is likely to ask about that topic.

The 12-week learning path

Weeks	Level	Focus	Deliverable
1-2	Beginner	Chapters 1-4: concepts, setup, models, DAG, materializations, staging/marts	A working project with 6 models and tests
3-4	Beginner+	Chapters 5-8: Jinja, packages, tests, sources, seeds, snapshots, docs	Reusable macros, source freshness, SCD2 snapshot
5-6	Intermediate	Chapters 9-11: incremental models, microbatch, Databricks features, selection/state	Incremental fact table and slim CI selection
7-8	Intermediate+	Chapters 12-14: orchestration, CI/CD, performance, governance	GitHub Actions pipeline and cost review
9-10	Advanced	Chapters 15-17: Semantic Layer, AI features, dbt v2/Fusion	Metrics layer and an AI-assisted workflow
11-12	Expert	Chapters 18-20: capstone, interview drills, teach-back	Capstone shipped, 100+ questions answered

If you only have one hour per day, do one section plus its exercises. If you have a full week, do a chapter per day and reserve the weekend for the capstone.

Conventions used

- `analytics_lab` is a placeholder catalog. Replace it with your catalog (`clearpay` in your current profile) or create a dedicated learning catalog.
- `dbt_dbx` is the project name in examples. Yours may differ; the commands are otherwise identical.
- Code blocks marked `sql`, `yaml`, `bash`, `powershell`, `jinja`, or `python` are complete unless marked "fragment."
- **Exercise** boxes are tasks. **Answer** boxes are deliberately short so you can check yourself without reading a full solution.
- Commands assume the virtual environment is activated. On Windows: `.\.venv\Scripts\Activate.ps1`.
- Where a command is specific to dbt v2, it is labelled **(v2)**.

What you need before Chapter 1

- A Databricks workspace you can sign in to.
- Permission to use a SQL warehouse (serverless is ideal for learning) and to create a schema in a catalog.
- A Databricks personal access token (PAT) or OAuth set up (Chapter 2 walks through both).
- Python 3.10-3.13 recommended. Python 3.14 works with dbt Core 1.12 but some third-party packages may lag; if you hit installation problems, create the venv with 3.12 or 3.13.
- Git and a GitHub account (free) for the CI/CD chapters.
- VS Code installed.

Contents at a glance

Part I - Foundations

1. Analytics Engineering, dbt, and the Databricks Lakehouse
2. Installing and Connecting: Databricks, Python, dbt, VS Code

Part II - Core dbt

3. Models, refs, sources, the DAG, and materializations
4. Building a Layered Project: staging, intermediate, marts

Part III - Reusability

5. Jinja and Macros
6. Packages and the dbt Ecosystem

Part IV - Trust

7. Testing, Unit Tests, and Contracts
8. Sources, Seeds, Snapshots, and Documentation

Part V - Scale

9. Incremental Models and Microbatch

Part VI - Databricks

10. The Databricks Adapter in Depth

Part VII - Advanced Engineering

11. Configuration, Selection, State, and Deployment Patterns
12. Orchestration and CI/CD
13. Performance and Cost Optimization
14. Governance, Security, and dbt Mesh

Part VIII - Intelligence

15. The dbt Semantic Layer and Metrics
16. AI in dbt: Wizard, Copilot, MCP, and Agent Skills
17. dbt v2 / Fusion: What Changes and How to Migrate

Part IX - Capstone

18. Capstone: Build the ClearPay Lakehouse End to End

Part X - Mastery

19. Interview Questions and Answers (100+)
20. Cheat Sheets, Glossary, Resources, and a 30-Day Plan

PART I - FOUNDATIONS

1. Analytics Engineering, dbt, and the Databricks Lakehouse

1.1 The problem dbt solves

Every company that uses data eventually builds the same pipeline: extract data from applications, load it into a warehouse, and transform it into tables that analysts, dashboards, and machine-learning models can trust. The extract and load parts are largely commoditized. The transform part is where organizations drown.

Before dbt, transformation logic usually lived in one of four places:

Where logic lived	Typical failure
GUI ETL tools	Logic is hidden in a proprietary canvas; hard to review, test, or version
Stored procedures	Thousands of lines of imperative SQL; no dependency graph; no tests
Scheduled scripts	Ordering is manual; reruns duplicate data; no documentation
Analyst spreadsheets	No lineage, no governance, no reproducibility

dbt (data build tool) turns transformation into **software engineering**: SQL files in Git, a dependency graph, automated tests, documentation, environments, code review, and CI/CD. It does not extract or load data. It transforms data **inside** the warehouse, using the warehouse's own compute.

The simplest definition:

> dbt compiles and runs `SELECT` statements in dependency order, persists the results as views/tables/incremental models, tests the results, and publishes metadata about everything it built.

1.2 ETL versus ELT

Traditional ETL transforms data before loading it into the warehouse. Modern ELT loads raw data first and transforms it in the warehouse.

	ETL	ELT with dbt
Transform engine	External tool/server	Warehouse (Databricks SQL/Spark)
Raw data	Often discarded	Preserved in a bronze/raw layer
Reprocessing	Re-run the ETL job	Re-run SQL against raw history
Logic ownership	ETL developers	Analytics engineers, in Git
Testing	Tool-specific	Native dbt tests plus warehouse constraints

	ETL	ELT with dbt
Scaling	ETL server	Warehouse elasticity

ELT is possible because cloud warehouses separate storage from compute. Databricks' lakehouse is a good example: Delta tables live in cloud storage, while SQL warehouses and clusters provide compute on demand.

1.3 What dbt is - and is not

dbt is:

- A compiler: Jinja + SQL becomes executable SQL.
- An orchestrator of one project: it understands dependencies and runs nodes in DAG order.
- A testing framework: generic tests, singular tests, unit tests, contracts.
- A documentation generator: a searchable site plus machine-readable artifacts.
- A metadata engine: `manifest.json`, `run_results.json`, `catalog.json`, lineage.
- A collaboration workflow: Git, pull requests, environments, CI/CD.

dbt is not:

- An ingestion tool (use Fivetran, Airbyte, Databricks Auto Loader, Kafka, etc.).
- A storage engine (Delta Lake, Iceberg, Parquet do that).
- A general-purpose scheduler (dbt platform has jobs; many teams use Airflow, Dagster, or Databricks Workflows).
- A BI tool (though the Semantic Layer serves metrics to BI tools).
- Magic: a bad model is still bad; dbt makes it visible and testable.

1.4 The dbt mental model

Learn these five sentences and most of dbt becomes predictable:

1. **A model is a `SELECT` statement saved as a `.sql` file.** dbt wraps it in DDL/DML depending on the materialization.
2. **`ref()` declares a dependency.** It returns the correct relation name for the current target and adds an edge to the DAG.
3. **The materialization decides how the result is persisted** (view, table, incremental, ephemeral, materialized view, streaming table).
4. **A test is a query that returns bad rows.** Zero rows = pass.
5. **Configuration is inherited and overridden** from project to folder to model, with in-file config taking precedence.

Everything else - macros, packages, snapshots, metrics, exposures - is an extension of those five ideas.

1.5 How a dbt run actually works

When you run `dbt run`, roughly this happens:

1. **Parse**: dbt reads `dbt_project.yml`, models, YAML properties, macros, packages, and profiles. It renders Jinja where needed and builds a manifest of nodes and dependencies.

2. **Compile**: each selected model's Jinja is rendered into SQL. `ref()` and `source()` are resolved to fully qualified relation names.
3. **Graph**: dbt topologically sorts selected nodes, respecting dependencies.
4. **Execute**: dbt opens connections (up to `threads`) and runs the materialization for each node. Models at the same DAG level can run in parallel.
5. **Record**: dbt writes artifacts to `target/`, including compiled SQL, run results, and timing.
6. **Test** (if `dbt build` or `dbt test`): tests run after their parent nodes; failures can warn or error.

Two consequences are worth memorizing:

- **Selection happens before execution.** `dbt run -s +fct_orders` selects the fact table and all ancestors, then runs them in order.
- **A failed parent skips its children by default.** In dbt 1.12 you can opt into `on_error: continue` for specific models, but the default is to stop downstream work.

1.6 Databricks essentials for dbt developers

You do not need to be a Spark expert, but you must understand the objects dbt touches.

Workspace, compute, and storage

Object	What it is	How dbt uses it
Workspace	The Databricks UI/API home for notebooks, jobs, repos	Where you inspect tables, run SQL, manage jobs
SQL warehouse	Serverless or provisioned SQL compute	Default `http_path` in `profiles.yml`; runs models and tests
All-purpose cluster	Interactive Spark cluster	Optional per-model compute; required for some Python model modes
Job cluster	Ephemeral cluster created for a job	Python models or Databricks Workflows
Unity Catalog	Governance layer for data and AI assets	Catalogs, schemas, tables, volumes, grants, tags, lineage
Delta Lake	Transactional table format	Default table format for dbt-databricks models

The three-level namespace

Unity Catalog uses `catalog.schema.table`:

```
select *
from clearpay.silver.fct_orders
limit 10;
```

In dbt:

- `catalog` maps to the `catalog` field in your profile (older adapters call it `database`).
- `schema` is the target schema plus optional custom schema logic.
- `identifier` is the model alias (defaults to the file name).

Your current profile uses catalog `clearpay` and schema `bronze`. The examples in this guide use `analytics_lab`; substitute your catalog.

Medallion architecture

A practical layering for Databricks:

Layer	Purpose	Typical materialization	Example
Bronze	Raw ingestion, append-only, minimal cleaning	View or streaming table over raw files	`bronze.orders_raw`
Silver	Cleaned, conformed, deduplicated, joined	View/table/incremental	`silver.stg_orders`, `silver.int_orders_enriched`
Gold	Business-ready facts, dimensions, marts	Table/incremental/materialized view	`gold.fct_orders`, `gold.dim_customers`

dbt does not require the medallion pattern, but it maps cleanly to dbt's staging/intermediate/marts convention. Chapter 4 builds it.

Delta Lake in one minute

Delta Lake adds a transaction log (`_delta_log`) to Parquet files. That enables:

- ACID transactions and time travel (`VERSION AS OF`, `TIMESTAMP AS OF`).
- `MERGE`, `UPDATE`, `DELETE` (important for incremental models and snapshots).
- Schema evolution and enforcement.
- Efficient metadata operations and `OPTIMIZE`/`ZORDER`/liquid clustering.

dbt-databricks defaults to Delta for tables and incremental models. Some configs (`replace_where`, `delete+insert`, `microbatch`) require Delta.

1.7 Where dbt fits in the modern data stack

```
Sources            Ingestion              Storage              Transformation        Consumption
SaaS APIs   ->     Fivetran/Airbyte  ->   Delta/UC       ->    dbt models        ->   BI, notebooks,
Operational DBs    Auto Loader/Kafka      Bronze/Silver/Gold     tests, metrics         Semantic Layer,
Files/Events       Custom pipelines       (Unity Catalog)        docs, lineage          ML, apps
```

dbt sits at the transformation and definition layer. The Semantic Layer (Chapter 15) extends it into a metrics layer. dbt Mesh (Chapter 14) extends it across teams and projects. AI features (Chapter 16) extend it into an agentic development workflow.

1.8 The analytics engineer role

An analytics engineer sits between data engineering and analytics. The role typically owns:

- Modeling raw data into clean, documented, tested marts.
- Defining business logic once (revenue, active customers, churn) and reusing it.
- Maintaining data quality tests and incident response for transformation failures.
- Reviewing SQL like software: style, performance, contracts, lineage.
- Partnering with analysts on metric definitions and with data engineers on ingestion.
- Owning the dbt project's structure, CI/CD, and documentation.

Skills that make an expert:

Skill	Why it matters
SQL fluency (window functions, joins, CTEs, dedup)	The daily work
Dimensional modeling	Stable, understandable marts
Jinja/macros	DRY logic without hiding it
Testing and data quality	Trust
Warehouse performance	Cost and latency
Git/CI/CD	Safe change
Databricks/Unity Catalog	Governance and operations
Communication	Metrics are a social contract

1.9 A first look at a dbt project

```
dbt_dbx/
├── dbt_project.yml          # project identity and config
├── profiles.yml             # connection (often in ~/.dbt/)
├── packages.yml             # external packages
├── models/
│   ├── staging/             # 1:1 with sources
│   ├── intermediate/        # reusable business logic
│   └── marts/               # facts, dimensions, wide tables
├── macros/                  # reusable Jinja
├── tests/                   # singular tests
├── seeds/                   # small CSVs
├── snapshots/               # SCD2 history
├── analyses/                # compiled but not materialized
├── target/                  # artifacts (generated)
├── logs/                    # logs (generated)
└── README.md
```

That is the whole surface area. The sophistication comes from how you use it.

1.10 Exercises

Exercise 1.1 - Concepts. In your own words, explain the difference between dbt and an ingestion tool, and between dbt and a scheduler.

Exercise 1.2 - Mental model. A model `fct_orders` selects from `stg_orders`, which selects from `source('raw', 'orders')`. Draw the DAG. Which node runs first? What happens to `fct_orders` if `stg_orders` fails?

Exercise 1.3 - Databricks. Write the fully qualified name of a table called `fct_orders` in schema `gold` in catalog `clearpay`. Then write the `ref()` call that would produce it if the model file is `fct_orders.sql`.

Exercise 1.4 - Materialization intuition. For each case, choose view, table, incremental, or materialized view: (a) a lightweight rename of a source, (b) a dashboard table rebuilt nightly, (c) a billion-row event table where only new events arrive, (d) an aggregation that Databricks should refresh automatically.

Answers. 1.1: dbt transforms data already in the warehouse; ingestion moves data in; a scheduler triggers work, while dbt understands dependencies and executes a project. 1.2: source -> `stg_orders` -> `fct_orders`; `stg_orders` runs first; by default `fct_orders` is skipped when `stg_orders` errors. 1.3: `clearpay.gold.fct_orders`; `{{ ref('fct_orders') }}`. 1.4: (a) view, (b) table or materialized view, (c) incremental, (d) materialized view (or streaming table for ingestion).

1.11 Interview check

- What does dbt actually execute? How does it know the order?
- Why is ELT more common than ETL today?
- What is the difference between `ref()` and `source()`?
- What is stored in `manifest.json`, and why would a platform team care?
- When would you *not* use dbt?
- Explain the medallion architecture and its trade-offs.

2. Installing and Connecting: Databricks, Python, dbt, VS Code

This chapter gets you from zero to a green `dbt debug` and a first model. Do not skip the permission checks; most "dbt is broken" issues are authentication, catalog, or compute issues.

2.1 Prerequisites and permissions

Databricks side

You need:

1. A workspace URL, for example `https://dbc-xxxx.cloud.databricks.com`.
2. A SQL warehouse (serverless or pro). Note its **HTTP path** from **SQL Warehouses > your warehouse > Connection details**.
3. A catalog and a schema where you can create objects.
4. A personal access token (PAT) or OAuth login.

Minimum Unity Catalog privileges for a development schema:

Privilege	Object	Why
`USE CATALOG`	Catalog	See the catalog
`USE SCHEMA`	Schema	See objects
`CREATE TABLE`, `CREATE VIEW`	Schema	Create models
`SELECT`	Tables/views	Read sources and upstream models
`MODIFY`	Tables	Incremental writes, snapshots
`CREATE SCHEMA`	Catalog	Optional, if dbt should create per-developer schemas

Ask your Databricks admin for a development catalog (or a schema in an existing catalog) and a SQL warehouse. In a personal/free workspace you are usually the metastore admin and already have access.

Local side

- Python 3.10-3.13 (3.12 is a safe default). Check with `python --version`.
- Git.
- VS Code.
- PowerShell (default on Windows).

2.2 Create the Databricks objects (SQL editor)

Run these in the Databricks SQL editor, adjusting names. If your admin already created a catalog, skip the catalog statement.

```
-- A learning catalog. Skip if you must use an existing catalog.
CREATE CATALOG IF NOT EXISTS analytics_lab;

-- Schemas for the medallion layers and a landing area for raw data.
CREATE SCHEMA IF NOT EXISTS analytics_lab.bronze;
CREATE SCHEMA IF NOT EXISTS analytics_lab.silver;
CREATE SCHEMA IF NOT EXISTS analytics_lab.gold;

-- Optional: a schema for dbt seeds and snapshots.
CREATE SCHEMA IF NOT EXISTS analytics_lab.utility;

-- Verify.
SHOW SCHEMAS IN analytics_lab;
```

If you are using an existing catalog (for example `clearpay`), replace `analytics_lab` everywhere. Do not create objects in a production schema while learning.

A source dataset you can use

Databricks workspaces commonly expose the `samples` catalog. Check:

```
SHOW CATALOGS LIKE 'samples';
SHOW SCHEMAS IN samples;
SELECT * FROM samples.tpch.orders LIMIT 5;
```

If `samples.tpch` exists, you have a ready-made retail dataset: `orders`, `lineitem`, `customer`, `part`, `supplier`, `nation`, `region`. If it does not, use any small CSV you own and load it into `analytics_lab.bronze`, or use the seed files in Chapter 8.

If you want raw tables that look like an ingestion landing zone, create them with CTAS:

```
CREATE OR REPLACE TABLE analytics_lab.bronze.orders_raw AS
SELECT * FROM samples.tpch.orders;

CREATE OR REPLACE TABLE analytics_lab.bronze.customer_raw AS
SELECT * FROM samples.tpch.customer;

CREATE OR REPLACE TABLE analytics_lab.bronze.lineitem_raw AS
SELECT * FROM samples.tpch.lineitem;
```

> In a shared workspace, `samples` may be read-only but readable. If not, ask your admin or load your own CSV.

2.3 Create and activate a virtual environment

From your project folder (`C:\Users\THINKBOOK\Documents\dbt_dbx`):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

If PowerShell blocks activation:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Then activate again. You should see `(.venv)` in the prompt.

2.4 Install dbt-databricks

For the dbt Core 1.12 line:

```
pip install "dbt-databricks>=1.12,<1.13"
dbt --version
```

Expected output includes `dbt-core`, `dbt-databricks`, and `dbt-spark`. If you want the newest engine instead, see Chapter 17: `pip install dbt` installs dbt v2, which is a different distribution. Do not install both into the same environment.

`dbt-databricks` brings in `dbt-core`, the Databricks adapter, and the Spark adapter. Pin versions in `requirements.txt` for reproducibility:

```
dbt-databricks==1.12.8
```

Then:

```
pip install -r requirements.txt
```

2.5 Authenticate to Databricks

Option A: Personal access token (simplest for learning)

1. In Databricks: click your avatar > **Settings** > **Developer** > **Access tokens** > **Generate new token**.
2. Copy the token once.
3. Store it in an environment variable, never in Git:

```
# Current session only:
$env:DATABRICKS_TOKEN = "dapi-xxxxxxxxxxxxxxxx"

# Persist for your Windows user (new terminals):
[Environment]::SetEnvironmentVariable("DATABRICKS_TOKEN", "dapi-xxxxxxxxxxxxxxxx", "User")
```

4. Your `profiles.yml` references it with `{{ env_var('DATABRICKS_TOKEN') }}`.

Option B: OAuth (recommended for humans)

```
dev:
  type: databricks
  catalog: analytics_lab
  schema: bronze
  host: dbc-xxxx.cloud.databricks.com
  http_path: /sql/1.0/warehouses/xxxxxxxxxxxxxxxx
  auth_type: oauth
  threads: 4
```

`dbt debug` or the first run opens a browser for consent. OAuth tokens expire, so you may re-authenticate periodically.

Option C: Service principal (CI/CD)

Create a Databricks service principal, grant it access to the catalog/schema/warehouse, and store its client ID/secret in your CI secret store. Use `auth_type: oauth` with client credentials or a service-principal token. Never commit secrets.

Where does `profiles.yml` live?

dbt looks in this order:

1. `--profiles-dir` if passed.
2. `DBT_PROFILES_DIR` environment variable.
3. The current directory.
4. `~/.dbt/`.

For a single project, keeping `profiles.yml` in the project root is convenient but risky: it must be gitignored. Many teams keep it in `~/.dbt/profiles.yml`. This guide shows it in the project for clarity, with `.gitignore` protection.

2.6 Understand `profiles.yml`

A minimal, correct profile for Unity Catalog:

```
dbt_dbx:
  target: dev
  outputs:
    dev:
      type: databricks
      catalog: analytics_lab
      schema: bronze
      host: dbc-xxxx.cloud.databricks.com
      http_path: /sql/1.0/warehouses/xxxxxxxxxxxxxxxx
      token: "{{ env_var('DATABRICKS_TOKEN') }}"
      threads: 4
      connect_timeout: 600
      connect_retries: 3
```

Key fields:

Field	Meaning	Common mistake
`type`	Adapter name	Using `databricks` with the wrong package installed

Field	Meaning	Common mistake
`catalog`	Unity Catalog catalog	Using a Hive Metastore database name
`schema`	Default target schema	Expecting it to be the full three-part name
`host`	Workspace hostname, no `https://`	Including `https://` or a trailing path
`http_path`	SQL warehouse or cluster path	Copying the cluster path for a warehouse
`token`	PAT or OAuth	Hard-coding a token; committing it
`threads`	Parallel connections	Setting it higher than the warehouse can handle

> **Your current project.** The `dbt_project.yml` name is `dbt_dbx` but model configs are nested under `databricks_dbt`. dbt only applies config under the project name, so those bronze/silver/gold settings are currently ignored. Either rename the key to `dbt_dbx` or change `name:` to `databricks_dbt`. This is a classic silent misconfiguration; Chapter 3 explains config resolution.

2.7 Create the project

If starting fresh:

```
dbt init dbt_dbx
```

The wizard asks for:

- project name (`dbt_dbx`)
- database/catalog (`analytics_lab`)
- schema (`bronze`)
- host
- http path
- authentication method
- threads

It writes `dbt_project.yml`, a starter model, and a profile. If you already have the project, skip this.

Project anatomy

```
dbt_dbx/
├── analyses/            # SQL compiled but not materialized
├── macros/              # Jinja macros
├── models/
│   ├── staging/
│   │   └── _sources.yml
│   ├── intermediate/
│   └── marts/
├── seeds/               # CSV files
├── snapshots/           # SCD2 snapshots
├── tests/               # singular tests
├── dbt_project.yml
├── packages.yml
├── profiles.yml         # or ~/.dbt/profiles.yml
├── requirements.txt
├── .gitignore
└── README.md
```

A good `.gitignore`:

```
.venv/
target/
dbt_packages/
logs/
profiles.yml
.env
*.log
__pycache__/
```

2.8 Configure `dbt_project.yml`

A clean starter configuration:

```yaml
name: dbt_dbx
version: "1.0.0"
config-version: 2
profile: dbt_dbx

model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

clean-targets:
  - target
  - dbt_packages

models:
  dbt_dbx:
    staging:
      +materialized: view
      +schema: silver
      +tags: ["staging"]
    intermediate:
      +materialized: view
      +schema: silver
      +tags: ["intermediate"]
    marts:
      +materialized: table
      +schema: gold
      +tags: ["marts"]

seeds:
  dbt_dbx:
    +schema: utility
    +quote_columns: false

snapshots:
  dbt_dbx:
    +target_schema: silver
    +tags: ["snapshot"]
```

Important details:

- `name` must match the profile key and the top-level key under `models:`.
- `config-version: 2` is the modern format.
- `+` prefixes are required in `dbt_project.yml` to distinguish config from folder names.

- `+schema: silver` does **not** replace the target schema by default; it becomes `<target_schema>_silver` unless you override `generate_schema_name`. Many teams prefer one schema per layer; Chapter 3 shows the custom macro.
- Materializing staging as views is a good default: it keeps raw logic close to sources and avoids storing redundant data.

2.9 Connect and verify

```
dbt debug
```

Expected: connection OK, profile OK, project OK. If the profile is in the project root and dbt cannot find it, run:

```
dbt debug --profiles-dir .
```

Then check warehouse permissions and the catalog:

```
dbt run-operation current_catalog
```

If you do not have that macro yet, use a simple model (next section).

2.10 Your first model

Create `models/staging/stg_orders.sql`:

```
select
    o_orderkey      as order_id,
    o_custkey       as customer_id,
    o_orderstatus   as order_status,
    o_totalprice    as order_total,
    o_orderdate     as order_date,
    o_orderpriority as order_priority,
    o_clerk         as clerk,
    o_comment       as comment
from {{ source('bronze', 'orders_raw') }}
```

Create `models/staging/_sources.yml`:

```
version: 2

sources:
  - name: bronze
    catalog: analytics_lab
    schema: bronze
    tables:
      - name: orders_raw
        description: Raw orders landed from the source system.
        loaded_at_field: _ingested_at
        freshness:
          warn_after: {count: 12, period: hour}
          error_after: {count: 24, period: hour}
```

If your raw table has no `_ingested_at`, remove the freshness block for now; Chapter 8 covers source freshness properly.

Create a schema file with tests, `models/staging/_staging.yml`:

```yaml
version: 2

models:
  - name: stg_orders
    description: "One row per order, renamed and cast from the raw orders table."
    columns:
      - name: order_id
        description: "Primary key."
        data_tests:
          - unique
          - not_null
      - name: customer_id
        data_tests:
          - not_null
      - name: order_status
        data_tests:
          - accepted_values:
              values: ["O", "P", "F"]
```

Run:

```
dbt run --select stg_orders
dbt test --select stg_orders
```

Verify in Databricks:

```sql
SELECT * FROM analytics_lab.silver.stg_orders LIMIT 10;
```

If `+schema: silver` is configured as above and `generate_schema_name` is default, the table may actually be `analytics_lab.bronze_silver.stg_orders`. Run `dbt show --select stg_orders --limit 5` or check the run log for the exact relation. Chapter 3 fixes schema naming.

2.11 VS Code setup

Install these extensions:

Extension	Publisher	Purpose
dbt	dbt Labs (`dbtLabsInc.dbt`)	Parse, IntelliSense, lineage, run/test, compiled SQL
Databricks	Databricks	Browse catalogs, run SQL, manage clusters
Python	Microsoft	Virtual env and Python models
YAML	Red Hat	YAML validation (see conflict note)
SQLFluff	SQLFluff	Linting (optional; `dbt lint` exists in v2)
GitLens	GitKraken	Git history (optional)

Recommended settings (`.vscode/settings.json`):

```json
{
  "files.associations": {
    "*.sql": "jinja-sql",
    "*.yml": "yaml"
  },
  "editor.formatOnSave": true,
  "python.defaultInterpreterPath": "${workspaceFolder}/.venv/Scripts/python.exe",
  "dbt.perspective": "sql"
}
```

Notes:

- The official dbt extension is the one that understands `ref()`, sources, macros, and lineage. Avoid installing several dbt extensions at once; they can conflict over language servers.
- If the Red Hat YAML extension flags dbt-only keys, disable it for the workspace or add the dbt schema.
- The dbt extension works with both v1 and v2; features such as column-level lineage and strict static analysis require `static_analysis: strict` on v2.
- The extension can run `dbt run`, `dbt build`, and `dbt test`, preview compiled SQL, and show lineage. Use the command palette (`Ctrl+Shift+P`) and search "dbt".

Databricks in VS Code

The Databricks extension lets you:

- Browse Unity Catalog tables and schemas.
- Run SQL against a warehouse.
- Attach notebooks and sync files (not required for dbt).
- Configure a cluster/warehouse for Python development.

For dbt work, the SQL warehouse connection in `profiles.yml` is what matters; the extension is a convenience for inspection.

2.12 First-run troubleshooting

Symptom	Likely cause	Fix
`Could not find profile named 'dbt_dbx'`	Profile name/key mismatch or wrong directory	Match `profile:` in `dbt_project.yml`; run with `--profiles-dir`
`Catalog 'x' does not exist`	Wrong catalog or no `USE CATALOG`	Check `SHOW CATALOGS`; ask for grants
`Schema 'x' does not exist`	Target schema missing and no create permission	Create schema or grant `CREATE SCHEMA`
`Invalid access token`	Expired PAT, env var not set in this terminal	Re-set `$env:DATABRICKS_TOKEN`; test with `dbt debug`
`http_path` error	Cluster path used instead of warehouse path	Copy the SQL warehouse HTTP path
`ModuleNotFoundError: dbt`	venv not activated	Activate `.venv`; check `dbt --version`
`dbt-databricks requires Python <3.14` (or similar)	Python too new for a dependency	Use Python 3.12/3.13 venv
Tables land in an unexpected schema	Default `generate_schema_name` concatenates	Override the macro (Chapter 3)
Model config in `dbt_project.yml` ignored	Key does not match project `name`	Fix the top-level key
`Database Error: PERMISSION_DENIED`	Missing `SELECT`/`MODIFY`/`CREATE`	Ask admin; check UC grants
`dbt run` hangs at connect	Warehouse starting or network/proxy	Increase `connect_timeout`; check warehouse state
YAML parse error on `data_tests`	Older dbt expects `tests:`	1.12 supports `data_tests`; check version
`dbt docs serve` port busy	Another process uses 8080	`dbt docs serve --port 8081`

2.13 A repeatable environment checklist

Before every new project:

1. `python --version` and `dbt --version` inside the venv.
2. `dbt debug` passes.
3. `dbt deps` succeeds (if packages are used).
4. `dbt parse` succeeds.
5. A trivial `dbt run --select <one_model>` succeeds.
6. `dbt test --select <one_model>` succeeds.
7. `git status` shows no secrets, `target/`, or `dbt_packages/`.
8. CI uses a pinned `requirements.txt` and a service principal.

2.14 Exercises

Exercise 2.1 - Connect. Create a `profiles.yml` for catalog `analytics_lab`, schema `bronze`, using an environment variable for the token. Run `dbt debug` and fix any errors.

Exercise 2.2 - Permissions. In Databricks SQL, run `SHOW GRANTS ON SCHEMA analytics_lab.bronze`. Which privileges do you have? Which are missing for creating a table? For reading a source?

Exercise 2.3 - First model. Create `stg_customers.sql` from `customer_raw` with at least five renamed columns. Add `unique` and `not_null` tests on the customer key. Run `dbt build --select stg_customers`.

Exercise 2.4 - Inspect artifacts. After the run, open `target/compiled/dbt_dbx/models/staging/stg_customers.sql` and `target/run_results.json`. What SQL did dbt execute? How long did it take?

Exercise 2.5 - VS Code. Open the model in VS Code, preview compiled SQL, and open the lineage view. Confirm the source -> staging edge is visible.

Answers. 2.2: You need `USE CATALOG`, `USE SCHEMA`, `SELECT` on the source, and `CREATE TABLE`/`CREATE VIEW` plus `MODIFY` for writes. 2.3: Use `{{ source('bronze', 'customer_raw') }}`; tests go under `columns:`. 2.4: The compiled file shows the rendered SQL with the fully qualified source; `run_results.json` contains status, timing, and adapter response. 2.5: The official dbt extension's lineage view requires the project to parse; check the Problems panel if it does not.

2.15 Interview check

- Walk me through `dbt debug` and what it validates.
- What is the difference between `catalog`, `schema`, and `target`?
- How do you keep Databricks credentials out of Git?
- Why might a dbt model land in an unexpected schema?
- What happens if two developers run dbt into the same schema at the same time?
- How would you set up dbt for a new developer in under 30 minutes?

PART II - CORE dbt

3. Models, refs, sources, the DAG, and Materializations

3.1 The anatomy of a model

Every model file is a SQL `SELECT` with optional Jinja:

```
{{ config(
    materialized = 'table',
    tags = ['marts', 'finance']
) }}

with orders as (
    select * from {{ ref('stg_orders') }}
),

final as (
    select
        order_id,
        customer_id,
        order_date,
        order_total
    from orders
)

select * from final
```

Rules:

- The file must contain one final `SELECT` (or a set of statements ending in a `SELECT` for some adapters).
- Do not end with a semicolon in a model file; dbt wraps the SQL.
- `config()` can appear anywhere, but convention is at the top.
- The file name becomes the relation name unless `alias` is configured.
- Folders do not affect the relation name by default, only configuration inheritance and organization.

3.2 `ref()`: the most important function in dbt

```
select * from {{ ref('stg_orders') }}
```

`ref()` does three jobs:

1. Resolves the correct relation for the current target (`analytics_lab.silver.stg_orders`, a dev schema, etc.).
2. Adds a dependency edge to the DAG.
3. Enables lineage, documentation, and selection.

Never hard-code a model's schema/table in another model. `ref()` is what makes environments portable and DAG ordering correct.

`ref()` also accepts a version (Chapter 11):

```
select * from {{ ref('dim_customers', version=2) }}
```

3.3 `source()`: declaring raw data

Sources are declared in YAML:

```
version: 2

sources:
  - name: bronze
    catalog: analytics_lab
    schema: bronze
    description: "Raw landing tables loaded by ingestion."
    tables:
      - name: orders_raw
        identifier: orders_raw
        description: "One row per order, append-only."
        loaded_at_field: _ingested_at
        freshness:
          warn_after: {count: 12, period: hour}
          error_after: {count: 24, period: hour}
```

Use them:

```
select * from {{ source('bronze', 'orders_raw') }}
```

Benefits:

- Lineage starts at the source, not at a hard-coded table.
- Freshness checks work.
- Source tests and descriptions work.
- You can swap environments (dev/prod catalogs) without editing models.

`source()` returns a relation object; you can call `.identifier` or use it in `adapter.get_relation` if needed.

Source vs staging

A common debate: should staging models select directly from sources, or should ingestion create views first? Best practice:

- Sources point at raw tables.
- Staging models are the only place that references sources.
- Staging models rename, cast, deduplicate, and standardize.
- Marts never reference sources directly.

This gives you one place to change when a source schema changes.

3.4 The DAG

dbt builds a directed acyclic graph from `ref()` and `source()` edges. You can inspect it:

```
dbt list --select stg_orders+
dbt list --select +fct_orders
dbt list --select @fct_orders
dbt ls --resource-type model
```

Graph operators:

Operator	Meaning	Example
`+model`	All ancestors	`+fct_orders`
`model+`	All descendants	`stg_orders+`
`+model+`	Ancestors and descendants	`+fct_orders+`
`@model`	Model, ancestors, and descendants	`@fct_orders`
`n+model`	Ancestors up to n levels	`2+fct_orders`

The DAG is why dbt can run independent branches in parallel. It is also why changing a staging model can rebuild a large part of the project; use selectors and state (Chapter 11) to control scope.

3.5 Materializations

A materialization is the strategy dbt uses to persist a model. Choose based on freshness, query cost, data volume, and who consumes the table.

Materialization	What dbt does	Best for	Avoid when
`view`	`CREATE VIEW` each run	Light transforms, staging, always-fresh data	Heavy joins/aggregations queried often
`table`	`CREATE TABLE AS` each run	Marts, dashboards, moderate data	Huge tables rebuilt often
`incremental`	First run table; later DML (merge/append/replace)	Large event/fact tables	Small tables; logic that must fully recompute
`ephemeral`	No relation; inlined as CTE	Lightweight reuse in one or two models	You need to query it, test it directly, or use contracts
`materialized_view`	Databricks managed MV; auto-refresh	Aggregations serving dashboards	Complex unsupported SQL; need row-level control
`streaming_table`	Databricks streaming table (Lakeflow)	Ingestion/append-only streams	Non-streaming sources, complex updates

Databricks-specific notes:

- `materialized_view` and `streaming_table` require Unity Catalog and a serverless SQL warehouse (per Databricks docs).
- `streaming_table` is provided by dbt-databricks, not core dbt.
- MVs and streaming tables support `partition_by`, `liquid_clustered_by` (1.11+), `tblproperties`, `databricks_tags`, and schedules.
- Python models support only `table` and `incremental`.

Choosing a materialization: a decision tree

1. Is the model consumed directly by a BI tool or many downstream models?
    - Yes -> table, incremental, or materialized view.
    - No -> view or ephemeral.
2. Is the source append-only and huge?
    - Yes -> incremental (merge/append) or streaming table.
3. Does the warehouse support automatic refresh with acceptable latency?
    - Yes -> materialized view.
4. Is the transformation trivial and cheap?
    - Yes -> view.
5. Is the logic reused in only one or two models and never queried directly?
    - Yes -> ephemeral (sparingly).

3.6 Configuring materializations

Three places, in increasing specificity:

Project file (applies to a folder):

```yaml
models:
  dbt_dbx:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

Properties file (applies to a model):

```yaml
models:
  - name: fct_orders
    config:
      materialized: incremental
      unique_key: order_id
```

In the model file (highest precedence):

```sql
{{ config(materialized='incremental', unique_key='order_id') }}
```

Precedence: in-file > properties YAML > project file. CLI flags can override some configs at runtime.

3.7 Configuration inheritance and the + prefix

In `dbt_project.yml`, + means "this is a config, not a subfolder":

```yaml
models:
  dbt_dbx:          # project name
    staging:        # folder
      +materialized: view
      +tags: ["staging"]
```

Without +, dbt treats `materialized` as a folder name and silently ignores it (or errors in v2 strict mode). This is one of the most common beginner mistakes.

The project-name trap

Config only applies under the project's `name`. If `name: dbt_dbx` but config is under `databricks_dbt`, nothing applies. Your current project has exactly this mismatch. Fix it before building further.

Config scopes

Config	Can be set in project	Can be set on model	Notes
`materialized`	Yes	Yes	Model wins
`schema`	Yes	Yes	Often combined with `generate_schema_name`
`tags`	Yes	Yes	Model tags merge with folder tags
`enabled`	Yes	Yes	Disable a model without deleting it
`meta`	Yes	Yes	Arbitrary metadata for tooling
`grants`	Yes	Yes	Databricks/UC privileges
`contract`	No	Yes	Model-level only
`unique_key`	Yes	Yes	Incremental
`incremental_strategy`	Yes	Yes	Incremental

3.8 Schemas, databases, and aliases

By default, dbt creates relations in the target schema from `profiles.yml`. `+schema` customizes it.

Default behavior

`generate_schema_name` in dbt-core does this:

- If a custom schema is configured: `<target_schema>_<custom_schema>`.
- If not: `<target_schema>`.

So with target schema `bronze` and `+schema: silver`, you get `bronze_silver`, not `silver`. Many teams find this surprising.

Override for clean layer schemas

Create `macros/generate_schema_name.sql`:

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}
{%- endmacro %}
```

Now `+schema: silver` produces `analytics_lab.silver`. In development, you may want per-developer isolation instead:

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- set default_schema = target.schema -%}
    {%- if target.name == 'prod' -%}
        {%- if custom_schema_name is none -%}{{ default_schema }}
        {%- else -%}{{ custom_schema_name | trim }}{%- endif -%}
    {%- else -%}
        {{ default_schema }}_{{ custom_schema_name | trim if custom_schema_name else 'default' }}
    {%- endif -%}
{%- endmacro %}
```

Result in dev: `dev_jane_silver`; in prod: `silver`. This prevents developers from overwriting each other.

Aliases

```
{{ config(alias='orders') }}
```

The relation becomes `<schema>.orders` instead of `<schema>.stg_orders`. Use aliases for stable public names or to match an existing table, not to hide model names.

Databases/catalogs

To materialize in a different catalog:

```
{{ config(database='analytics_lab_metrics') }}
```

On Databricks, `database` is an alias for catalog in many contexts; prefer `catalog` in profiles and `database` in configs only if your adapter expects it. Chapter 10 clarifies.

3.9 Ephemeral models: use and abuse

```
{{ config(materialized='ephemeral') }}

select ...
```

dbt inlines the model as a CTE named `__dbt__cte__<model>` in downstream models. Pros: no storage, reusable logic. Cons: cannot be queried directly, cannot be tested easily, can make compiled SQL enormous, and does not support contracts.

Use ephemeral for tiny helpers used once or twice. Prefer views for anything you might need to debug.

3.10 Model properties: descriptions, tests, docs

```yaml
version: 2

models:
  - name: fct_orders
    description: "One row per order with payment and customer attributes."
    config:
      materialized: incremental
      unique_key: order_id
    columns:
      - name: order_id
        description: "Primary key."
        data_tests:
          - unique
          - not_null
      - name: customer_id
        description: "Foreign key to dim_customers."
        data_tests:
          - relationships:
              to: ref('dim_customers')
              field: customer_id
```

This YAML is not decoration: it drives tests, docs, and the catalog.

3.11 `target`, `this`, and other context

Inside a model or macro:

Object	Meaning
`target.name`	Environment name (`dev`, `ci`, `prod`)
`target.schema`	Default schema
`target.catalog` / `target.database`	Catalog
`this`	The current model's relation object
`is_incremental()`	True if the model exists and this is not a full refresh
`var('name', default)`	Project/CLI variable
`env_var('NAME')`	Environment variable
`model.name`	Current model name
`model.path`	File path
`run_started_at`	Timestamp of the run
`invocation_id`	Unique run ID

These are available in models and macros; not all are available in YAML (v2 expands YAML macro support).

3.12 Running and inspecting models

```
dbt run                          # all models
dbt run -s stg_orders            # one model
dbt run -s staging               # everything in a folder
dbt run -s tag:finance           # by tag
dbt run -s +fct_orders           # ancestors + model
dbt run -s fct_orders+           # model + descendants
dbt build -s marts               # models + tests + seeds + snapshots in DAG order
dbt compile -s stg_orders        # render SQL without running
dbt show -s stg_orders --limit 5
dbt list -s stg_orders --output json
```

`dbt build` is usually preferred in CI and production because it interleaves tests with models: a failed test prevents downstream models from building.

3.13 Exercises

Exercise 3.1 - Fix the config. Your `dbt_project.yml` has `name: dbt_dbx` but model config under `databricks_dbt`. What happens? Fix it and verify with `dbt compile` or `dbt list --output json`.

Exercise 3.2 - Schema logic. With target schema `dev_alice`, `+schema: silver`, and the default macro, what schema is used? With the override macro from 3.8, what schema is used in dev and prod?

Exercise 3.3 - Materializations. Build `stg_orders` as a view and `fct_orders` as a table. Inspect Databricks: `SHOW CREATE TABLE`. What DDL did dbt run? Then change `fct_orders` to incremental and run twice; what changes?

Exercise 3.4 - Ephemeral. Create an ephemeral model `int_order_totals` used by `fct_orders`. Compile `fct_orders` and find the inlined CTE. What are two drawbacks?

Exercise 3.5 - DAG. Run `dbt list -s +fct_orders --output json` and count nodes. Which nodes are ancestors? Which would rebuild if `stg_orders` changed?

Answers. 3.1: No folder configs apply; models use defaults (view) and the target schema. 3.2: Default -> `dev_alice_silver`; override -> `dev_alice_silver` in dev (if implemented that way) and `silver` in prod. 3.3: View -> `CREATE VIEW`; table -> `CREATE TABLE AS`; incremental first run -> table, later runs -> merge/insert. 3.4: Look for `__dbt__cte__int_order_totals`; drawbacks: can't query/test directly, bloats downstream SQL. 3.5: Use the JSON output and follow `depends_on`.

3.14 Interview check

- Why does dbt discourage `select *` in marts?
- What is the difference between a source and a staging model?
- How do folder configs and model configs interact?
- When would you use ephemeral? What are the risks?
- How does dbt decide the order of execution?
- What is the difference between `dbt run` and `dbt build`?
- How do you change the schema of a single model?

4. Building a Layered Project: staging, intermediate, marts

4.1 The layered architecture

A scalable dbt project separates concerns:

Layer	Prefix	Responsibility	Materialization
Staging	`stg_`	One source table per model; rename, cast, deduplicate, basic cleaning	View
Intermediate	`int_`	Joins, business logic, reusable transformations; not usually exposed to BI	View or ephemeral
Marts	`fct_`, `dim_`, `rpt_`	Facts, dimensions, wide tables for consumption	Table/incremental/MV
Utilities	`util_`	Date spines, mappings, helper models	View/table

Rules of thumb:

- Staging models reference sources only.
- Intermediate models reference staging/intermediate only.
- Marts reference intermediate/staging and are the only layer BI should query.
- Each layer should be understandable without reading the layer below in detail.

4.2 Naming conventions

Object	Convention	Example
Staging model	`stg_<source>__<entity>` (double underscore optional)	`stg_shopify__orders`
Intermediate	`int_<entity>_<verb>`	`int_orders_pivoted`
Fact	`fct_<process>`	`fct_orders`
Dimension	`dim_<entity>`	`dim_customers`
Report	`rpt_<audience>_<grain>`	`rpt_finance_daily`
Primary key	`<entity>_id`	`order_id`
Foreign key	`<referenced_entity>_id`	`customer_id`
Boolean	`is_` / `has_`	`is_refunded`
Timestamp	`_at`	`created_at`
Date	`_date`	`order_date`
Amount	`_amount` / `_total`	`order_total`
Count	`_count`	`item_count`

Consistency beats cleverness. Document the convention in your README and enforce it in review.

4.3 The example domain

We will model a simplified payments/retail business:

- `orders_raw`: order header (TPCH `orders`).
- `customer_raw`: customer (TPCH `customer`).
- `lineitem_raw`: order lines (TPCH `lineitem`).
- A seed `payment_methods.csv` mapping codes to names.
- A seed `refunds.csv` with refund events.

Business questions:

1. Revenue by day, customer, and payment method.
2. Order count, average order value, refund rate.
3. Customer lifetime value and first/last order dates.
4. Which orders are late or high-risk?

4.4 Staging models

stg_orders

```
{{ config(materialized='view') }}

with source as (
    select * from {{ source('bronze', 'orders_raw') }}
),

renamed as (
    select
        o_orderkey::bigint            as order_id,
        o_custkey::bigint             as customer_id,
        o_orderstatus::string         as order_status,
        o_totalprice::decimal(18,2)   as order_total,
        o_orderdate::date             as order_date,
        o_orderpriority::string       as order_priority,
        o_clerk::string               as clerk,
        o_comment::string             as comment
    from source
)

select * from renamed
```

stg_customers

```
{{ config(materialized='view') }}

with source as (
    select * from {{ source('bronze', 'customer_raw') }}
),

renamed as (
    select
        c_custkey::bigint       as customer_id,
        c_name::string          as customer_name,
        c_address::string       as address,
        c_nationkey::bigint     as nation_id,
        c_phone::string         as phone,
        c_acctbal::decimal(18,2) as account_balance,
        c_mktsegment::string    as market_segment,
        c_comment::string       as comment
    from source
)

select * from renamed
```

stg_lineitems

```
{{ config(materialized='view') }}

with source as (
    select * from {{ source('bronze', 'lineitem_raw') }}
),

renamed as (
    select
        l_orderkey::bigint       as order_id,
        l_partkey::bigint        as part_id,
        l_suppkey::bigint        as supplier_id,
        l_linenumber::int        as line_number,
        l_quantity::decimal(18,2) as quantity,
        l_extendedprice::decimal(18,2) as extended_price,
        l_discount::decimal(18,4) as discount,
        l_tax::decimal(18,4)     as tax,
        l_returnflag::string     as return_flag,
        l_linestatus::string     as line_status,
        l_shipdate::date         as ship_date,
        l_commitdate::date       as commit_date,
        l_receiptdate::date      as receipt_date,
        l_shipinstruct::string   as ship_instruction,
        l_shipmode::string       as ship_mode
    from source
)

select * from renamed
```

Deduplication pattern

If a source can contain duplicate keys, deduplicate explicitly:

```sql
with source as (
    select * from {{ source('bronze', 'orders_raw') }}
),

deduped as (
    select *
    from source
    qualify row_number() over (
        partition by o_orderkey
        order by _ingested_at desc
    ) = 1
)

select * from deduped
```

`QUALIFY` is supported by Databricks SQL. Always document which record wins and why.

4.5 Intermediate models

`int_orders_enriched`

```sql
{{ config(materialized='view') }}

with orders as (
    select * from {{ ref('stg_orders') }}
),

customers as (
    select * from {{ ref('stg_customers') }}
),

payments as (
    select * from {{ ref('seed_payment_methods') }}
)

select
    o.order_id,
    o.customer_id,
    c.customer_name,
    c.market_segment,
    o.order_date,
    o.order_status,
    o.order_priority,
    o.order_total,
    p.payment_method_name,
    p.is_refundable
from orders o
left join customers c on o.customer_id = c.customer_id
left join payments p on o.order_priority = p.payment_method_code
```

This is a learning example; mapping payment method to order priority is not realistic. In a real project you would join a payment table. The point is the pattern: one intermediate model per business concept, with clear joins.

int_order_line_totals

```
{{ config(materialized='view') }}

with lineitems as (
    select * from {{ ref('stg_lineitems') }}
)

select
    order_id,
    count(*)                                    as line_count,
    sum(quantity)                               as total_quantity,
    sum(extended_price * (1 - discount))        as net_line_revenue,
    sum(extended_price * (1 - discount) * (1 + tax)) as gross_line_revenue
from lineitems
group by order_id
```

4.6 Mart models

fct_orders (incremental)

```
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge',
    on_schema_change='sync_all_columns',
    tags=['marts', 'finance']
) }}

with orders as (
    select * from {{ ref('int_orders_enriched') }}
),

line_totals as (
    select * from {{ ref('int_order_line_totals') }}
),

joined as (
    select
        o.order_id,
        o.customer_id,
        o.order_date,
        o.order_status,
        o.order_priority,
        o.order_total,
        o.payment_method_name,
        o.is_refundable,
        coalesce(l.line_count, 0)             as line_count,
        coalesce(l.total_quantity, 0)         as total_quantity,
        coalesce(l.net_line_revenue, 0)       as net_line_revenue,
        coalesce(l.gross_line_revenue, 0)     as gross_line_revenue
    from orders o
    left join line_totals l on o.order_id = l.order_id
)

select * from joined

{% if is_incremental() %}
where order_date >= (
    select coalesce(max(order_date), '1900-01-01') from {{ this }}
)
{% endif %}
```

Notes:

- The `is_incremental()` block only applies after the first run.
- `merge` with `unique_key` upserts changed orders.
- The lookback here is "same or newer date"; Chapter 9 discusses late-arriving data and lookbacks.
- `on_schema_change` handles new columns in development; choose deliberately in production.

dim_customers

```
{{ config(materialized='table', tags=['marts', 'customer']) }}

with customers as (
    select * from {{ ref('stg_customers') }}
),

orders as (
    select
        customer_id,
        min(order_date) as first_order_date,
        max(order_date) as most_recent_order_date,
        count(*)        as number_of_orders,
        sum(order_total) as lifetime_value
    from {{ ref('fct_orders') }}
    group by customer_id
)

select
    c.customer_id,
    c.customer_name,
    c.market_segment,
    c.account_balance,
    coalesce(o.first_order_date, null)       as first_order_date,
    coalesce(o.most_recent_order_date, null) as most_recent_order_date,
    coalesce(o.number_of_orders, 0)          as number_of_orders,
    coalesce(o.lifetime_value, 0)            as lifetime_value
from customers c
left join orders o on c.customer_id = o.customer_id
```

rpt_revenue_daily

```
{{ config(materialized='table', tags=['marts', 'finance']) }}

select
    order_date,
    count(*)                         as order_count,
    count(distinct customer_id)      as customer_count,
    sum(order_total)                 as revenue,
    avg(order_total)                 as average_order_value,
    sum(case when order_status = 'F' then order_total else 0 end) as fulfilled_revenue
from {{ ref('fct_orders') }}
group by order_date
order by order_date
```

4.7 The YAML layer

`models/marts/_marts.yml`:

```yaml
version: 2

models:
  - name: fct_orders
    description: "Order-level fact with customer and line-item metrics."
    config:
      materialized: incremental
      unique_key: order_id
    columns:
      - name: order_id
        description: "Primary key."
        data_tests: [unique, not_null]
      - name: customer_id
        description: "FK to dim_customers."
        data_tests:
          - not_null
          - relationships:
              to: ref('dim_customers')
              field: customer_id
      - name: order_total
        description: "Order total in USD."
        data_tests:
          - not_null
      - name: order_date
        data_tests: [not_null]

  - name: dim_customers
    description: "Customer dimension with lifetime metrics."
    columns:
      - name: customer_id
        data_tests: [unique, not_null]
```

Run:

```
dbt build --select staging intermediate marts
```

Or, in production:

```
dbt build --select +fct_orders +dim_customers +rpt_revenue_daily
```

4.8 Modeling best practices

1. **One purpose per model.** If the description needs "and", consider splitting.
2. **Explicit columns in marts.** Avoid `select *`; it breaks contracts and hides schema changes.
3. **Cast at the boundary.** Staging is where types become explicit.
4. **Deduplicate once.** Do not repeat `qualify` logic in five models.
5. **No business logic in staging.** Staging is mechanical.
6. **Name booleans as questions.** `is_active`, `has_refund`.
7. **Use consistent grains.** State the grain in the model description: "one row per order."
8. **Keep joins narrow.** Join on keys, then select only needed columns.
9. **Test keys and relationships.** At minimum: unique + not_null on PKs, relationships on FKs.
10. **Document assumptions.** A future engineer will thank you.

4.9 Common modeling anti-patterns

Anti-pattern	Why it hurts	Better
`select *` in marts	Schema drift, wasted columns, broken contracts	Explicit column list
Giant 800-line model	Untestable, slow to iterate	Break into intermediate models
Business logic in staging	Mixes cleaning and semantics	Keep staging mechanical
Direct `source()` in marts	Bypasses staging contract	Reference staging only
Using `ref()` for a seed as if it were raw	Seeds are versioned data, not ingestion	Use seeds for small mappings only
Circular references	dbt cannot build a DAG	Extract shared logic
Duplicate logic in multiple models	Inconsistent metrics	Macro or intermediate model
No primary key tests	Silent duplicates	Test uniqueness
One schema for everything	Hard to govern and grant	Layer schemas

4.10 Exercises

Exercise 4.1 - Build the layer. Create `stg_orders`, `stg_customers`, `stg_lineitems`, `int_orders_enriched`, `int_order_line_totals`, `fct_orders`, `dim_customers`, and `rpt_revenue_daily`. Run `dbt build` and confirm all tests pass.

Exercise 4.2 - Grain. Write the grain statement for each mart model in one sentence. Add it to the YAML description.

Exercise 4.3 - Dedup. Add a duplicate row to `orders_raw` (same `o_orderkey`, newer `_ingested_at`) and rerun staging. Prove that the staging model returns one row. What happens to the incremental fact table?

Exercise 4.4 - Refactor. Move the payment-method join out of `int_orders_enriched` into a dedicated `int_orders_payment_methods` model. Why might this be better or worse?

Exercise 4.5 - Test. Add `relationships` tests from `fct_orders.customer_id` to `dim_customers.customer_id` and from `stg_lineitems.order_id` to `stg_orders.order_id`. Run tests.

Exercise 4.6 - Query. Write a query against `rpt_revenue_daily` for the last 30 days: revenue, orders, AOV, and a 7-day moving average.

Answers. 4.3: The fact table merges on `order_id`, so the deduplicated staging row replaces the target row; without a lookback, an older order date may not be reselected. 4.4: Dedicated intermediate models improve testability and reuse, but add nodes and storage if materialized; use views. 4.6: Use a window function: `avg(revenue) over (order by order_date rows between 6 preceding and current row)`.

4.11 Interview check

- What belongs in staging versus marts?
- How do you decide the grain of a fact table?
- Why avoid `select *` in a mart?
- How would you model a many-to-many relationship?
- What is a degenerate dimension?
- How do you handle late-arriving dimensions?

- Where do you put business logic that is reused by three marts?

PART III - REUSABILITY

5. Jinja and Macros

5.1 Why Jinja

Jinja is a templating language that turns SQL into a program. In dbt it enables:

- Dynamic relation names via `ref()` and `source()`.
- Conditional SQL such as `is_incremental()`.
- Loops over columns, dates, or lists.
- Reusable macros for logic that must be identical everywhere.
- Environment-specific behavior via `target.name` and variables.

The goal is not clever code. The goal is **one definition of a business rule**, with SQL that remains readable and reviewable.

5.2 Jinja basics

```
{% set tax_rate = 0.2 %}

select
    order_id,
    order_total,
    order_total * {{ tax_rate }} as order_total_with_tax
from {{ ref('stg_orders') }}
```

Output:

```
select
    order_id,
    order_total,
    order_total * 0.2 as order_total_with_tax
from analytics_lab.silver.stg_orders
```

Delimiters:

Syntax	Purpose
`{{ ... }}`	Output an expression
`{% ... %}`	Statement (if, for, set, macro)
`{# ... #}`	Comment
`{%- ... -%}`	Whitespace control

Whitespace control matters when generating SQL. `{%-` strips whitespace before the tag; `-%}` strips after it. A stray newline is harmless, but a stray comma is not.

5.3 Variables, conditionals, and loops

```
{% set payment_methods = ['card', 'bank_transfer', 'wallet'] %}

select
    order_id,
    {% for method in payment_methods %}
    sum(case when payment_method = '{{ method }}' then order_total else 0 end)
        as {{ method }}_revenue
        {%- if not loop.last %},{% endif %}
    {% endfor %}
from {{ ref('fct_orders') }}
group by 1
```

Loop variables: `loop.index`, `loop.index0`, `loop.first`, `loop.last`, `loop.length`.

Conditionals:

```
{% if target.name == 'prod' %}
    where order_date >= dateadd(day, -365, current_date)
{% else %}
    where order_date >= dateadd(day, -30, current_date)
{% endif %}
```

Use environment conditionals sparingly. Hidden production-only logic makes bugs hard to reproduce. Prefer variables with explicit defaults.

5.4 Filters and tests

Filters transform values:

```
{{ 'orders' | upper }}                -- ORDERS
{{ ['a', 'b'] | join(', ') }}         -- a, b
{{ 3.14159 | round(2) }}              -- 3.14
{{ none | default('unknown') }}       -- unknown
{{ model.name | replace('stg_', '') }} -- orders
```

dbt adds filters such as `as_text`, `as_number`, `as_bool`, `as_native`, `fromjson`, `tojson`, `zip`, and `split`.

Tests use `is`:

```
{% if loop.first %}...{% endif %}
{% if value is none %}...{% endif %}
{% if value is defined %}...{% endif %}
```

5.5 The dbt Jinja context

Available in models and macros (not all in YAML):

Name	What it gives you
`ref`, `source`	Relations and dependencies
`config`	Set configuration
`var`	Project/CLI variables
`env_var`	Environment variables

Name	What it gives you
`target`	Current target (name, schema, catalog, type)
`this`	Current relation
`is_incremental()`	Whether to run incremental logic
`model`	Current model metadata (`name`, `path`, `config`)
`run_started_at`	Run timestamp
`invocation_id`	Run identifier
`execute`	True during execution, false during parsing
`flags`	Global flags
`graph`	Nodes and dependencies
`adapter`	Adapter methods (`get_relation`, `dispatch`, etc.)
`exceptions`	`raise_compiler_error`, `warn`
`log`	`log(msg, info=True)`
`modules`	Python modules (`re`, `itertools`, `datetime`)

A useful pattern is to guard introspection during parsing:

```
{% if execute %}
    {% set relation = adapter.get_relation(
        database=target.catalog,
        schema='bronze',
        identifier='orders_raw'
    ) %}
    {{ log('Relation: ' ~ relation, info=True) }}
{% endif %}
```

If you call `adapter.get_relation` during parsing without the guard, dbt may query the warehouse before it is ready, or return `None` unexpectedly.

5.6 Macros

A macro is a reusable Jinja function. Define it in `macros/`:

```
{% macro cents_to_dollars(column_name, scale=2) -%}
    (({{ column_name }} / 100.0)::numeric(18, {{ scale }}))
{%- endmacro %}
```

Use it:

```
select
    order_id,
    {{ cents_to_dollars('order_total_cents') }} as order_total
from {{ ref('stg_orders') }}
```

Macros can return SQL, values, or both. Use whitespace control to keep generated SQL tidy.

Arguments and defaults

```
{% macro revenue(column_name, status='F', include_tax=false) -%}
    sum(
        case
            when order_status = '{{ status }}'
            then {{ column_name }}
            else 0
        end
    )
{%- endmacro %}
```

Calling macros from YAML

```
models:
  - name: fct_orders
    config:
      meta:
        owner: "{{ var('team_owner', 'analytics') }}"
```

In dbt v1, YAML Jinja is restricted to built-ins; project macros are not available in YAML. In dbt v2, project macros are available. This is a meaningful migration difference.

`return()`

`return()` stops macro execution and returns a value:

```
{% macro safe_divide(numerator, denominator) -%}
    {%- if denominator == 0 -%}
        {{ return('null') }}
    {%- endif -%}
    ({{ numerator }} * 1.0 / {{ denominator }})
{%- endmacro %}
```

5.7 Practical macro library

Surrogate key

```
{% macro surrogate_key(fields) -%}
    md5(concat_ws('|',
        {%- for field in fields -%}
            coalesce(cast({{ field }} as string), '')
            {%- if not loop.last %}, {% endif -%}
        {%- endfor -%}
    ))
{%- endmacro %}
```

Usage:

```
{{ surrogate_key(['order_id', 'line_number']) }} as order_line_key
```

Prefer `dbt_utils.generate_surrogate_key`; it handles nulls and types consistently and is tested by the community.

Grant select after build

```
{% macro grant_select(schema, role) -%}
    {% set sql %}
        grant select on schema {{ schema }} to role {{ role }};
    {% endset %}
    {% do run_query(sql) %}
    {{ log('Granted select on ' ~ schema ~ ' to ' ~ role, info=True) }}
{%- endmacro %}
```

Use it in a post-hook. Chapter 11 covers grants properly.

Date spine

```
{% macro date_spine(start_date, end_date) -%}
    select explode(sequence(
        to_date('{{ start_date }}'),
        to_date('{{ end_date }}'),
        interval 1 day
    )) as date_day
{%- endmacro %}
```

Databricks supports `sequence` and `explode`. `dbt_utils.date_spine` is more portable.

Union relations

```
{% macro union_relations(relations) -%}
    {% for relation in relations %}
        select '{{ relation.name }}' as source_model, *
        from {{ relation }}
        {% if not loop.last %}union all{% endif %}
    {% endfor %}
{%- endmacro %}
```

In practice, use `dbt_utils.union_relations` so schemas are aligned by column name, not position.

5.8 `run_query` and statement blocks

`run_query` executes SQL during compilation/execution and returns a table:

```
{% macro get_max_order_date() -%}
    {% set query %}
        select max(order_date) as max_date from {{ ref('fct_orders') }}
    {% endset %}
    {% set result = run_query(query) %}
    {% if execute %}
        {{ return(result.columns[0].values()[0]) }}
    {% endif %}
    {{ return(none) }}
{%- endmacro %}
```

Rules:

- Only run queries when `execute` is true.
- `run_query` inside a model is usually a smell; prefer a variable, a model, or an incremental filter.
- Use it for metadata, grants, dynamic SQL, and operations.

`{% call statement(...) %}` executes side-effecting SQL:

```
{% call statement('grant', fetch_result=false) %}
    grant select on {{ this }} to role analyst;
{% endcall %}
```

5.9 `adapter.dispatch`: cross-platform macros

If your project runs on multiple adapters, dispatch lets each adapter provide its own implementation:

```
{% macro date_add(datepart, interval, date) -%}
    {{ return(adapter.dispatch('date_add', 'my_project')(datepart, interval, date)) }}
{%- endmacro %}

{% macro default__date_add(datepart, interval, date) -%}
    dateadd({{ datepart }}, {{ interval }}, {{ date }})
{%- endmacro %}

{% macro databricks__date_add(datepart, interval, date) -%}
    date_add({{ date }}, {{ interval }})
{%- endmacro %}
```

dbt's own `dbt.dateadd` macro already dispatches across adapters; use it before writing your own.

5.10 Debugging Jinja

Tool	Use
`dbt compile -s model`	Render SQL; inspect `target/compiled/...`
`{{ log('msg', info=True) }}`	Print during parse/compile
`dbt --debug run`	Verbose logs
`{{ this }}`	Check the resolved relation
`dbt run-operation macro_name --args '{...}'`	Run a macro without a model
`dbt parse`	Catch syntax/parse errors early
VS Code dbt extension	Inline Jinja diagnostics and compiled SQL preview

Common errors:

- Forgetting `{% endif %}` or `{% endfor %}`.
- Calling `ref()` in a macro that runs at parse time when `execute` is false.
- Quoting: `'{{ var("x") }}'` versus `{{ var("x") }}`.
- A trailing comma after a loop.
- Case sensitivity in `target.name` comparisons.
- Using `this` in a macro called outside a model context.

5.11 Exercises

Exercise 5.1 - Cents. Write `cents_to_dollars(column, scale=2)` and use it in `stg_payments` to convert an integer cents column to decimal dollars.

Exercise 5.2 - Pivot. Write a macro that pivots payment methods into columns. Use a list variable and a loop. Compile the model and inspect the SQL.

Exercise 5.3 - Safe divide. Write `safe_divide(numerator, denominator)` that returns `null` when the denominator is zero. Test it with a zero denominator.

Exercise 5.4 - Date spine. Generate one row per day from 2024-01-01 to 2024-12-31 using a macro or `dbt_utils.date_spine`. Use it to fill missing dates in revenue.

Exercise 5.5 - Dispatch. Write a `date_add` macro that dispatches to `databricks__date_add` and `default__date_add`. Call it from a model.

Exercise 5.6 - Debug. A macro produces a trailing comma before `from`. Diagnose using whitespace control and `loop.last`.

Answers. 5.1: `({{ column_name }} / 100.0)::decimal(18, {{ scale }})`. 5.2: Build `case` expressions in a loop and join with commas. 5.3: Use `nullif(denominator, 0)` or an `if`. 5.4: `dbt_utils.date_spine(datepart='day', start_date='2024-01-01', end_date='2025-01-01')`. 5.5: `adapter.dispatch('date_add', 'dbt_dbx')`. 5.6: Strip whitespace before the comma and emit it only when `not loop.last`.

5.12 Interview check

- When should logic be a macro versus a model?
- What is the difference between `run_query` and a normal `select`?
- Why is `execute` important?
- How do you make a macro portable across adapters?
- What are the risks of overusing Jinja?
- How do you test a macro?
- How do you document a macro?

6. Packages and the dbt Ecosystem

6.1 Why packages

Packages let you reuse tested macros, tests, models, and semantic definitions maintained by the community or your organization. They reduce boilerplate and encode best practices. They also add supply-chain risk, so pin versions and review what you install.

6.2 Installing packages

`packages.yml`:

```yaml
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.3.0", "<2.0.0"]

  - package: calogica/dbt_expectations
    version: [">=0.10.0", "<0.11.0"]

  - package: dbt-labs/codegen
    version: [">=0.13.0", "<0.14.0"]

  - package: dbt-labs/audit_helper
    version: [">=0.12.0", "<0.13.0"]

  - package: dbt-labs/dbt_project_evaluator
    version: [">=1.0.0", "<2.0.0"]
```

Then:

```
dbt deps
```

Packages install into `dbt_packages/` (gitignored). `dbt clean` removes them.

Other package sources

```yaml
packages:
  - local: ../shared_dbt_models
  - git: https://github.com/your-org/your-package.git
    revision: v1.2.0
  - package: your-org/your-package
    version: 1.2.0
```

For private Git, use SSH keys or a token in CI. dbt 1.12 supports private packages with native SSH configuration.

Version pinning

Always pin:

- Exact (`1.3.0`) for maximum reproducibility.
- A narrow range (`>=1.3.0, <2.0.0`) for patches but not breaking changes.

Commit `package-lock.yml` (generated by `dbt deps`) so CI installs the same versions.

6.3 `dbt_utils`: the essential toolbox

Macro	What it does	Example
`generate_surrogate_key`	Hash a list of columns into a key	`{{ dbt_utils.generate_surrogate_key(['order_id','line_number']) }}`
`star`	Select all columns except some	`{{ dbt_utils.star(from=ref('stg_orders'), except=['comment']) }}`
`union_relations`	Union relations by column name	`{{ dbt_utils.union_relations(relations=[ref('a'), ref('b')]) }}`
`date_spine`	Generate a date/period spine	`{{ dbt_utils.date_spine('day', '2024-01-01', '2025-01-01') }}`

Macro	What it does	Example
`pivot`	Pivot values into columns	`{{ dbt_utils.pivot('payment_method', ['card','bank']) }}`
`unpivot`	Unpivot columns into rows	`{{ dbt_utils.unpivot(ref('t'), cast_to='string', exclude=['id']) }}`
`deduplicate`	Deduplicate by key	`{{ dbt_utils.deduplicate(relation=ref('x'), partition_by='id', order_by='updated_at desc') }}`

Example: explicit columns plus a surrogate key.

```
select
    {{ dbt_utils.generate_surrogate_key(['order_id', 'line_number']) }} as order_line_key,
    {{ dbt_utils.star(from=ref('stg_lineitems'), except=['comment']) }}
from {{ ref('stg_lineitems') }}
```

Caveat: `star` hides schema changes. Use it in staging, not in marts with contracts.

6.4 `dbt_expectations`: richer tests

```
models:
  - name: fct_orders
    columns:
      - name: order_total
        data_tests:
          - dbt_expectations.expect_column_values_to_be_between:
              min_value: 0
              max_value: 1000000
          - dbt_expectations.expect_column_values_to_not_be_null
      - name: order_status
        data_tests:
          - dbt_expectations.expect_column_distinct_count_to_be_between:
              min_value: 1
              max_value: 10
```

Useful tests: value ranges, row counts, column-pair consistency, regex matches, freshness, quantiles, null proportions, uniqueness within a group.

6.5 `codegen`: generate YAML and models

Codegen macros generate boilerplate:

- `generate_model_yaml` from a table's columns.
- `generate_source` from a schema.
- `generate_staging_model`.

```
dbt run-operation generate_model_yaml --args '{"model_names": ["stg_orders"]}'
```

Paste the output into YAML, then review and improve descriptions. Generated tests are a starting point, not a substitute for thought.

6.6 `audit_helper`: validate a refactor

When migrating logic, compare two relations:

```
{{ audit_helper.compare_relations(
    a_relation=ref('fct_orders_old'),
    b_relation=ref('fct_orders_new'),
    primary_key='order_id'
) }}
```

It returns counts of rows in A only, B only, and differing values. Use it before swapping a model in production.

6.7 `dbt_project_evaluator`: project governance

This package tests your project's structure: model naming, test coverage, source usage, folder layering, duplicate sources, and more. It is a health check for the dbt project itself, not the data.

```
dbt build --select package:dbt_project_evaluator
```

Review failures as a backlog, not a blocker for every PR.

6.8 Elementary: observability

Elementary is an observability package/platform that collects dbt artifacts, test results, run times, and data quality metrics into dashboards. It can alert on failures and anomalies.

Use it when you need:

- Historical test results.
- Anomaly detection.
- Run-time trends.
- Alert routing to Slack/email.

6.9 Other notable packages

Package	Use
`dbt-labs/dbt_utils`	General macros
`calogica/dbt_expectations`	Rich data tests
`dbt-labs/codegen`	Generate YAML/SQL
`dbt-labs/audit_helper`	Compare relations
`dbt-labs/dbt_project_evaluator`	Project structure tests
`elementary-data/elementary`	Observability
`dbt-labs/dbt_artifacts`	Load artifacts into tables for analysis
Fivetran source/transform packages	Prebuilt staging models for SaaS sources

6.10 Package governance

1. Pin versions and commit the lock file.
2. Review the package's license and maintenance activity.
3. Avoid packages that run introspective queries on every parse.
4. Do not let packages dictate your folder structure or naming.
5. Test upgrades in a branch; compare run times and test results.

6. Keep packages in `packages.yml`, never vendored into your repo.
7. For a monorepo, consider an internal package for shared macros and models.

6.11 Exercises

Exercise 6.1 - Install. Create `packages.yml` with `dbt_utils` pinned to a range, run `dbt deps`, and use `generate_surrogate_key` in a model.

Exercise 6.2 - Star. Use `dbt_utils.star` to select all columns except `comment` from `stg_orders`. Compile and inspect the SQL.

Exercise 6.3 - Tests. Add two `dbt_expectations` tests to `fct_orders`: `order_total` between 0 and 1,000,000, and `order_status` distinct count between 1 and 10. Run them.

Exercise 6.4 - Codegen. Generate YAML for `dim_customers`, paste it into a file, and improve the descriptions. Add at least one test that codegen did not generate.

Exercise 6.5 - Audit. Create a copy of `fct_orders` with a deliberate bug, then use `audit_helper.compare_relations` to quantify the difference.

Answers. 6.2: The compiled SQL lists every column except `comment`. 6.3: Use the package-qualified test names. 6.4: Codegen produces generic descriptions; replace them. 6.5: The audit output shows rows only in A, only in B, and differing rows.

6.12 Interview check

- How do you install and pin a dbt package?
- What are the risks of third-party packages?
- When would you write a macro instead of using a package?
- How does `dbt_utils.star` interact with contracts?
- How do you audit a refactor?
- What does `dbt_project_evaluator` test?

PART IV - TRUST

7. Testing, Unit Tests, and Contracts

7.1 The testing mindset

A dbt test is a SQL query that returns **rows that violate an expectation**. If the query returns zero rows, the test passes. This is simple and powerful: you can test anything expressible in SQL.

There are four layers of trust:

Layer	Question	Tool
Schema tests	Are keys unique, non-null, and related?	Generic tests in YAML
Business tests	Does the logic match the business rule?	Singular tests, dbt_expectations
Unit tests	Does this model produce the expected output for known input?	dbt unit tests
Contracts	Does the model's shape match the promised schema?	Model contracts

7.2 Generic tests

Built-in generic tests:

- `unique`
- `not_null`
- `accepted_values`
- `relationships`

```yaml
version: 2

models:
  - name: fct_orders
    columns:
      - name: order_id
        data_tests:
          - unique
          - not_null
      - name: order_status
        data_tests:
          - accepted_values:
              values: ["O", "P", "F"]
              quote: true
      - name: customer_id
        data_tests:
          - relationships:
              to: ref('dim_customers')
              field: customer_id
```

Configuration options:

```yaml
data_tests:
  - unique:
      severity: warn
      config:
        where: "order_date >= current_date - interval 30 days"
        limit: 100
        store_failures: true
```

`store_failures: true` writes failing rows to a table in the target schema (named `<model>_<test>`). This is invaluable for debugging, but it creates objects; clean them up or use a dedicated schema.

7.3 Severity and thresholds

Severity	Effect
`error` (default)	Test failure fails the build; downstream models are skipped
`warn`	Logs a warning; build continues

Use `warn` for new tests that may be noisy, then promote to `error` after cleaning the data. Do not leave warnings forever; they become noise.

7.4 Singular tests

A singular test is a SQL file in `tests/` that returns failing rows:

`tests/assert_order_total_non_negative.sql`:

```sql
select *
from {{ ref('fct_orders') }}
where order_total < 0
```

`tests/assert_no_future_orders.sql`:

```sql
select *
from {{ ref('fct_orders') }}
where order_date > current_date()
```

Run:

```
dbt test --select assert_order_total_non_negative
```

Singular tests are best for cross-column or business rules that generic tests cannot express. Keep them small and named after the assertion.

7.5 Custom generic tests

Create `macros/test_positive_value.sql`:

```
{% test positive_value(model, column_name) %}
    select *
    from {{ model }}
    where {{ column_name }} < 0
{% endtest %}
```

Use it:

```yaml
columns:
  - name: order_total
    data_tests:
      - positive_value
```

With arguments:

```
{% test value_between(model, column_name, min_value, max_value) %}
    select *
    from {{ model }}
    where {{ column_name }} < {{ min_value }}
       or {{ column_name }} > {{ max_value }}
{% endtest %}
```

```yaml
data_tests:
  - value_between:
      min_value: 0
      max_value: 1000000
```

Custom tests are reusable across models and packages. Namespace them with a package if you distribute them.

7.6 Unit tests

Unit tests validate a model's logic with fixed inputs, without depending on warehouse data. They are fast, deterministic, and ideal for complex SQL.

```yaml
version: 2

unit_tests:
  - name: test_fct_orders_revenue
    model: fct_orders
    given:
      - input: ref('stg_orders')
        rows:
          - {order_id: 1, customer_id: 10, order_date: 2024-01-01, order_status: F, order_total: 100}
          - {order_id: 2, customer_id: 11, order_date: 2024-01-02, order_status: O, order_total: 200}
      - input: ref('int_order_line_totals')
        rows:
          - {order_id: 1, line_count: 2, total_quantity: 3, net_line_revenue: 90, gross_line_revenue: 108}
          - {order_id: 2, line_count: 1, total_quantity: 1, net_line_revenue: 200, gross_line_revenue: 240}
    expect:
      rows:
        - {order_id: 1, order_total: 100, line_count: 2, net_line_revenue: 90}
        - {order_id: 2, order_total: 200, line_count: 1, net_line_revenue: 200}
```

Notes:

- Unit tests can override macros, vars, and inputs.
- They run with `dbt test` (or `dbt build`) and are included in `dbt test --select test_type:unit`.
- Unit tests are not a replacement for data tests; they test logic, not production data.
- In dbt v2, `dbt build` runs all unit tests first. Local execution with DuckDB is experimental (`compute: local`).

Run only unit tests:

```
dbt test --select test_type:unit
```

7.7 Model contracts

A contract declares the model's column names and data types and enforces them at build time.

```yaml
models:
  - name: fct_orders
    config:
      contract:
        enforced: true
    columns:
      - name: order_id
        data_type: bigint
        constraints:
          - type: not_null
      - name: customer_id
        data_type: bigint
      - name: order_date
        data_type: date
      - name: order_total
        data_type: decimal(18,2)
```

Requirements:

- Every column in the model must be listed with a `data_type`.
- The model's SQL must produce exactly those columns (order can be enforced).
- Contracts are supported for table, view, incremental, and materialized view; not ephemeral.
- Databricks enforces `not_null` and `check` constraints where supported; primary/foreign key constraints are informational in many cases (verify current Databricks behavior).

Contracts are a commitment to downstream consumers. Use them for public marts, not every staging model.

7.8 Source freshness

Freshness answers: "Is the raw data recent enough?" Configure it on sources:

```yaml
sources:
  - name: bronze
    schema: bronze
    tables:
      - name: orders_raw
        loaded_at_field: _ingested_at
        freshness:
          warn_after: {count: 6, period: hour}
          error_after: {count: 24, period: hour}
        filter: "_ingested_at >= current_date - interval 7 days"
```

Run:

```
dbt source freshness
```

Results go to `target/sources.json`. In dbt v2, `dbt freshness` also checks models with freshness configured.

Freshness is not a data quality test; it checks timestamps. Pair it with row-count and null tests.

7.9 Testing strategy by layer

Layer	Minimum tests	Why
Staging	PK unique + not_null; accepted values for statuses	Catch source anomalies early
Intermediate	Uniqueness at the intended grain; relationships	Catch join fan-out
Facts	PK unique + not_null; FK relationships; amount ranges	Trust the numbers
Dimensions	PK unique + not_null; one row per entity	Prevent duplicate dimensions
Reports	Row count > 0; reconciliation to source	Catch empty/broken dashboards

The testing pyramid for data:

1. Many cheap schema tests on keys.
2. Some business-rule tests on marts.
3. A few unit tests for complex logic.
4. Contracts on public interfaces.
5. Freshness and volume checks for operations.

7.10 Testing anti-patterns

Anti-pattern	Problem	Fix
Testing everything as `error` on day one	Noisy failures block delivery	Start with `warn`, clean data, promote
No `where` on huge tests	Full scans cost money	Filter to recent partitions
Testing only at the end	Bugs are expensive to trace	Test staging and intermediate
Assuming tests replace contracts	Shape changes still break consumers	Use contracts on public marts
Ignoring warnings	Real issues get lost	Triage and fix or remove
Tests that depend on current date	Flaky at month boundaries	Use fixed dates or vars
No ownership	Failures have no owner	Tag models/tests with a team

7.11 Exercises

Exercise 7.1 - Generic tests. Add `unique` and `not_null` to `stg_orders.order_id`, `accepted_values` to `order_status`, and `relationships` from `fct_orders.customer_id` to `dim_customers.customer_id`.

Exercise 7.2 - Singular test. Write a test that fails if any `fct_orders.order_date` is in the future. Run it.

Exercise 7.3 - Custom generic test. Write `not_negative` and apply it to `order_total` and `net_line_revenue`.

Exercise 7.4 - Unit test. Write a unit test for `int_order_line_totals` with two line items for one order. Assert the count and sum.

Exercise 7.5 - Contract. Add a contract to `dim_customers` with explicit data types for five columns. Intentionally add an extra column to the model and observe the failure.

Exercise 7.6 - Freshness. Configure freshness on `orders_raw` with a `loaded_at_field`. Run `dbt source freshness` and interpret the result.

Answers. 7.1: Add tests under each column in YAML. 7.2: `where order_date > current_date()`. 7.3: `{% test not_negative(model, column_name) %} ... where {{ column_name }} < 0 {% endtest %}`. 7.4: Use `given` for `stg_lineitems` and `expect` for the aggregate. 7.5: The contract failure lists missing/extra columns. 7.6: `PASS`, `WARN`, or `ERROR` with max loaded timestamp.

7.12 Interview check

- What does a dbt test return when it passes?
- Difference between a generic test and a singular test?
- When would you use a unit test instead of a data test?
- What does a model contract enforce on Databricks?
- How do you test an incremental model?
- How do you keep tests cheap on a billion-row table?
- What is the difference between source freshness and a data test?

8. Sources, Seeds, Snapshots, and Documentation

8.1 Sources in depth

A source definition is a contract with the ingestion layer. Include:

- `name`: logical source name.
- `catalog`/`database` and `schema`.
- `tables` with `name`, optional `identifier`, `description`.
- `loaded_at_field` and freshness.
- `quoting` when identifiers are case-sensitive.
- `meta` for ownership, PII, SLA.
- `columns` with tests and descriptions.

```yaml
version: 2

sources:
  - name: bronze
    catalog: analytics_lab
    schema: bronze
    description: "Raw landing zone; append-only."
    meta:
      owner: "data-platform"
      sla: "hourly"
    freshness:
      warn_after: {count: 2, period: hour}
      error_after: {count: 6, period: hour}
    tables:
      - name: orders_raw
        description: "Order headers from the commerce platform."
        loaded_at_field: _ingested_at
        columns:
          - name: o_orderkey
            description: "Order key."
            data_tests:
              - unique
              - not_null
      - name: customer_raw
        description: "Customer master data."
        loaded_at_field: _ingested_at
```

Source tests

You can test source columns directly; failures indicate ingestion or schema problems, not model bugs. This is valuable for early warning.

Source quoting

```yaml
sources:
  - name: legacy
    schema: "Sales"
    quoting:
      database: true
      schema: true
      identifier: true
```

Use quoting only when needed; it makes SQL case-sensitive.

Source freshness filters

For large append-only tables, filter to recent partitions:

```yaml
filter: "_ingested_at >= current_date - interval 2 days"
```

This makes freshness cheap.

8.2 Seeds

Seeds are CSV files in `seeds/` that dbt loads as tables. They are version-controlled reference data, not a production ingestion mechanism.

`seeds/payment_methods.csv`:

```
payment_method_code,payment_method_name,is_refundable
1,Credit Card,true
2,Bank Transfer,true
3,Wallet,false
4,Cash,false
```

`dbt_project.yml`:

```yaml
seeds:
  dbt_dbx:
    +schema: utility
    +quote_columns: false
```

Load:

```
dbt seed
dbt seed --full-refresh
```

Use seeds for:

- Small lookup tables (country codes, payment methods).
- Static mappings that analysts should review.
- Test fixtures.

Do not use seeds for:

- Large datasets (they are versioned in Git and loaded on every run).
- Frequently changing data.
- PII or secrets.
- Data that belongs in ingestion.

Seed configuration

Per-seed in `seeds/_seeds.yml`:

```yaml
version: 2

seeds:
  - name: payment_methods
    description: "Payment method lookup."
    config:
      column_types:
        payment_method_code: int
        payment_method_name: string
        is_refundable: boolean
      delimiter: ","
      quote_columns: false
```

Seed tests

Seeds support the same tests:

```yaml
seeds:
  - name: payment_methods
    columns:
      - name: payment_method_code
        data_tests: [unique, not_null]
```

8.3 Snapshots (SCD Type 2)

A snapshot records how a row changes over time. It produces a history table with `dbt_valid_from`, `dbt_valid_to`, and (optionally) `dbt_scd_id`, `dbt_updated_at`, `dbt_is_deleted`.

Why: source systems overwrite data. If a customer changes address, the source keeps only the new address. A snapshot preserves the old address and when it changed.

Timestamp strategy

Use when the source has an `updated_at` column that changes on every update.

`snapshots/customers_snapshot.sql`:

```sql
{% snapshot customers_snapshot %}

{{
    config(
        target_schema='silver',
        unique_key='customer_id',
        strategy='timestamp',
        updated_at='updated_at',
        hard_deletes='invalidate'
    )
}}

select
    customer_id,
    customer_name,
    market_segment,
    updated_at
from {{ ref('stg_customers') }}

{% endsnapshot %}
```

Check strategy

Use when there is no reliable `updated_at`; dbt compares a list of columns.

```
{% snapshot customers_snapshot_check %}

{{
    config(
        target_schema='silver',
        unique_key='customer_id',
        strategy='check',
        check_cols=['customer_name', 'market_segment']
    )
}}

select
    customer_id,
    customer_name,
    market_segment
from {{ ref('stg_customers') }}

{% endsnapshot %}
```

Key configs

Config	Meaning
`unique_key`	Business key of the row
`strategy`	`timestamp` or `check`
`updated_at`	Timestamp column for timestamp strategy
`check_cols`	Columns to compare, or `all`
`hard_deletes`	`ignore`, `invalidate`, or `new_record` (version-dependent)
`invalidate_hard_deletes`	Legacy way to mark deleted rows
`snapshot_meta_column_names`	Rename metadata columns
`dbt_valid_to_current`	Use a sentinel date for current rows instead of null
`target_schema`	Where to write the snapshot

Run:

```
dbt snapshot
```

Query:

```
select *
from analytics_lab.silver.customers_snapshot
where dbt_valid_to is null;  -- current records
```

Snapshot best practices

- Snapshot a **staging** model, not a mart.
- Choose a stable `unique_key`; changes to it create new history rows.
- Timestamp strategy is cheaper and more reliable when a trustworthy `updated_at` exists.
- Snapshot tables grow forever; plan retention and clustering.
- Test uniqueness of `(unique_key, dbt_valid_from)`.
- Do not `--full-refresh` snapshots casually; you can lose history.
- Document what a change means (address change, plan change, etc.).

8.4 Documentation

dbt docs are generated from:

- Model/source/seed/snapshot descriptions in YAML.
- Column descriptions.
- Doc blocks (reusable Markdown).
- Exposures (dashboards, ML models, apps).
- Tests and lineage.

Generate and serve:

```
dbt docs generate
dbt docs serve --port 8080
```

Artifacts:

- `target/manifest.json`: full project graph.
- `target/catalog.json`: warehouse metadata (columns, types, stats).
- `target/run_results.json`: execution results.
- `target/sources.json`: freshness results.

Doc blocks

`models/docs.md`:

```
{% docs order_status %}
One of:
- `O`: order placed
- `P`: order shipped/processing
- `F`: order fulfilled
{% enddocs %}
```

Reference it:

```
columns:
  - name: order_status
    description: "{{ doc('order_status') }}"
```

Exposures

Exposures document downstream consumers:

```yaml
version: 2

exposures:
  - name: revenue_dashboard
    type: dashboard
    maturity: high
    url: https://bi.example.com/dashboards/123
    description: "Daily revenue dashboard for finance."
    owner:
      name: Finance Analytics
      email: finance-analytics@example.com
    depends_on:
      - ref('rpt_revenue_daily')
      - ref('fct_orders')
    tags: ["finance"]
```

Exposures appear in lineage and help assess impact: "If I change `fct_orders`, which dashboards break?"

Documentation quality

- Describe the grain ("one row per order").
- Describe caveats ("excludes test orders", "revenue is net of refunds").
- Define ambiguous terms ("active customer").
- Link to source systems and owners.
- Keep docs close to code; review them in PRs.

8.5 Exercises

Exercise 8.1 - Source. Declare `lineitem_raw` as a source with a description and a `loaded_at_field`. Add a `unique` test on `(order_id, line_number)` using a custom test or a singular test.

Exercise 8.2 - Seed. Create `payment_methods.csv`, load it, and join it to `stg_orders` in an intermediate model. Add a `not_null` test.

Exercise 8.3 - Snapshot. Create a timestamp snapshot of `stg_customers` using an `updated_at` column. If your source lacks one, add a synthetic `updated_at` in staging or use the check strategy. Update a customer's name in the raw table, rerun the snapshot, and query the history.

Exercise 8.4 - Docs. Add a doc block for `order_status` and reference it from `fct_orders`. Add an exposure for `rpt_revenue_daily`.

Exercise 8.5 - Impact. Use `dbt list -s +exposure:revenue_dashboard` to see everything the dashboard depends on. What would break if `stg_orders` changed?

Answers. 8.1: Use a singular test with `group by order_id, line_number having count(*) > 1`. 8.3: The old row gets `dbt_valid_to` set and a new row is inserted with `dbt_valid_to = null`. 8.5: The list includes the exposure's parents; direct changes to staging propagate to the fact and report.

8.6 Interview check

- What is the difference between a seed and a source?
- When would you snapshot a source versus a staging model?
- Timestamp vs check strategy: trade-offs?
- How do you document a dashboard dependency?

- What happens to a snapshot when a row is deleted?
- How do you test a snapshot?
- What is in `catalog.json` that is not in `manifest.json`?

PART V - SCALE

9. Incremental Models and Microbatch

9.1 Why incremental

A full table rebuild is simple but expensive at scale. Incremental models process only new or changed data and apply it to an existing table. The trade-off is complexity: you must define what "new" means, handle late data, and ensure idempotency.

When to use incremental:

- The table is large (millions+ rows) and grows append-only.
- A full rebuild exceeds the job window or costs too much.
- The transformation is expensive relative to the new data volume.

When not to:

- The table is small.
- The logic requires a full recomputation (for example, global ranking).
- The source data is not time-ordered and changes unpredictably (use a snapshot or full rebuild).

9.2 How dbt incremental works

On the first run (or `--full-refresh`):

1. dbt creates a table from the model's `SELECT` (often via a temporary relation).
2. Subsequent runs: dbt compiles the model with `is_incremental()` returning true, builds a temporary relation, and applies the configured strategy (merge, append, delete+insert, replace_where, insert_overwrite, microbatch).

`is_incremental()` returns true only if:

- The target relation already exists.
- `--full-refresh` is not set.
- The model is materialized as incremental.

Important: the filter inside `is_incremental()` is your responsibility (except for microbatch). dbt does not automatically know what "new" means.

9.3 The canonical pattern

```
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge',
    on_schema_change='append_new_columns'
) }}

select *
from {{ ref('stg_orders') }}

{% if is_incremental() %}
where order_date >= (
    select coalesce(max(order_date), '1900-01-01') from {{ this }}
)
{% endif %}
```

Add a lookback for late-arriving data:

```
{% if is_incremental() %}
where order_date >= (
    select dateadd(day, -3, coalesce(max(order_date), '1900-01-01')) from {{ this }}
)
{% endif %}
```

Then ensure the strategy replaces rows for those dates, not just appends.

9.4 Incremental strategies on Databricks

Strategy	DML	Requires	Use
`append`	`INSERT INTO`	Nothing	Append-only events, no updates
`merge` (default)	`MERGE INTO`	Delta/Hudi; `unique_key` recommended	Upserts by key
`delete+insert`	Delete matching keys, insert	Delta, `unique_key`, DBR 12.2+	Full-record replacement
`replace_where`	`INSERT ... REPLACE WHERE`	Delta, `incremental_predicates`	Replace partitions/ranges
`insert_overwrite`	Replace partitions/clusters	`partition_by` or clustering	Recompute whole partitions
`microbatch`	`replace_where` per batch	Delta, `event_time`, `begin`, `batch_size`	Large time-series

append

```
{{ config(materialized='incremental', incremental_strategy='append') }}

select *
from {{ ref('stg_events') }}
{% if is_incremental() %}
where event_ts > (select coalesce(max(event_ts), '1900-01-01') from {{ this }})
{% endif %}
```

Risk: if the filter overlaps, you get duplicates. Use only for truly append-only data or deduplicate downstream.

merge

```
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge'
) }}
```

dbt generates a `MERGE` with `WHEN MATCHED THEN UPDATE` and `WHEN NOT MATCHED THEN INSERT`. If `unique_key` is omitted, it behaves like append.

Databricks merge options (1.9+):

```
{{ config(
    unique_key='id',
    target_alias='t',
    source_alias='s',
    matched_condition='t.updated_at < s.updated_at',
    not_matched_condition='s.id is not null',
    not_matched_by_source_condition='t.updated_at < current_timestamp()',
    not_matched_by_source_action='delete',
    merge_with_schema_evolution=true
) }}
```

Use these for soft deletes, conditional updates, and schema evolution. Review generated SQL in `target/run/`.

delete+insert

```
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='delete+insert',
    incremental_predicates='order_date >= current_date - interval 30 days'
) }}
```

Deletes rows matching keys in the new batch, then inserts. Simpler than merge when you replace whole records.

replace_where

```
{{ config(
    materialized='incremental',
    incremental_strategy='replace_where',
    incremental_predicates='order_date >= current_date - interval 7 days'
) }}
```

Only rows matching the predicate are replaced. Caution: `replace_where` inserts by column order on Databricks; keep column order stable or use `on_schema_change`.

insert_overwrite

```
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by='order_date'
) }}
```

Replaces entire partitions returned by the query. If no `partition_by`, it replaces the whole table. Use `liquid_clustered_by` for clustered tables (dbt-databricks uses `replace on` keys).

9.5 on_schema_change

Value	Behavior
`ignore` (default)	Ignore new/changed columns; may fail on removed columns
`fail`	Error if schema changes
`append_new_columns`	Add new columns to the target
`sync_all_columns`	Add/remove/reorder to match the model (can be expensive)

Choose deliberately:

- Additive sources: `append_new_columns`.
- Stable contracts: `fail` (catch changes early).
- Development: `sync_all_columns`, but test on a clone.

9.6 Late-arriving data

Late data breaks naive max-date filters. Patterns:

1. **Lookback window**: reprocess the last N days on every run. Simple and robust if N covers lateness.
2. **High-water mark table**: store the max event time processed, plus a buffer.
3. **Merge by key**: even if you select a wide window, `merge` upserts, so duplicates are avoided (but cost increases).
4. **Microbatch with `lookback`**: dbt reprocesses N batches automatically.

Choose based on source SLAs and cost. Document the lateness assumption.

9.7 Idempotency and reruns

An incremental model must be safe to run twice. Tests:

- Run twice without new data: row counts should not change.
- Run with an overlapping window: no duplicate keys.
- Run `--full-refresh`: results should match the incremental table.

If a run fails midway, dbt's transaction semantics depend on the adapter. Databricks Delta supports atomic operations; still, design models so partial batches can be rerun.

9.8 Backfills

```
dbt run -s fct_orders --full-refresh
```

For microbatch, use event-time bounds:

```
dbt run -s fct_orders --event-time-start "2024-01-01" --event-time-end "2024-02-01"
```

For large backfills:

- Run in date slices to control cost.
- Use a separate warehouse/compute if possible.
- Notify downstream consumers.
- Compare results with `audit_helper`.

9.9 Microbatch deep dive

Microbatch is designed for large time-series data. Instead of writing `is_incremental()` logic, you declare:

```
{{ config(
    materialized='incremental',
    incremental_strategy='microbatch',
    event_time='order_date',
    begin='2020-01-01',
    batch_size='day',
    lookback=2,
    full_refresh=false
) }}

select *
from {{ ref('stg_orders') }}
```

dbt splits the work into one query per day (or hour/month/year), filters upstream models that declare `event_time`, and replaces each batch atomically. On Databricks it uses `replace_where`.

Required configs: `event_time`, `begin`, `batch_size`.

Optional: `lookback` (default 1), `concurrent_batches`.

To let dbt auto-filter an upstream model, give it an `event_time`:

```
models:
  - name: stg_orders
    config:
      event_time: order_date
```

To opt out for a specific ref: `{{ ref('upstream').render() }}`.

Backfill and retry:

```
dbt run -s fct_orders --event-time-start "2024-01-01" --event-time-end "2024-02-01"
dbt retry
```

Microbatch benefits:

- No manual `is_incremental()` filter.
- Idempotent batches.
- Parallel batch execution when safe.
- Partial retry of failed batches.

Limitations:

- Requires a reliable event time.

- Upstream models need `event_time` for filtering; otherwise full scans.
- Not ideal for non-time-series or heavily updated data.

9.10 Incremental vs snapshot vs materialized view vs streaming table

Need	Best choice
Upsert changed records by key	Incremental `merge`
Preserve full history of changes	Snapshot
Append-only events	Incremental `append` or streaming table
Warehouse-managed refresh of an aggregation	Materialized view
Continuous ingestion from files/streams	Streaming table
Recompute partitions	`insert_overwrite` or microbatch
Small table, simple logic	Table (full rebuild)

9.11 Performance and cost of incremental models

- Filter early; avoid scanning the full source.
- Partition/cluster on the filter column.
- Use `merge` only when needed; append is cheaper.
- Beware of small files; run `OPTIMIZE` or use liquid clustering.
- Test with `--full-refresh` periodically to catch drift.
- Monitor merge cardinality; exploding joins create duplicate keys and slow merges.
- Use `dbt build` so tests run before downstream models consume bad data.

9.12 Exercises

Exercise 9.1 - Merge. Convert `fct_orders` to incremental with `unique_key='order_id'` and `merge`. Run twice. Insert a changed order in the source and verify the target updates.

Exercise 9.2 - Append. Create `fct_events` with `append` and an event-time filter. Insert an event with an older timestamp and observe that it is not picked up. Add a lookback and rerun.

Exercise 9.3 - Delete+insert. Create a model with `delete+insert` on `order_id`. Compare the generated SQL with `merge` in `target/run/`.

Exercise 9.4 - Microbatch. Create a microbatch model over `stg_orders` with `event_time='order_date'`, `begin='1990-01-01'`, `batch_size='year'`. Run it and inspect the number of batches in the logs.

Exercise 9.5 - Late data. Simulate a late order (order_date 3 days before max) and prove your model handles it. What lookback do you need?

Exercise 9.6 - Idempotency. Run your incremental model twice without source changes. Compare row counts and key counts. Then `--full-refresh` and compare.

Answers. 9.1: The merge updates the matched row. 9.2: Without lookback, the older event is outside the filter; add `-3 days`. 9.3: `delete+insert` runs a `DELETE + INSERT` (or `REPLACE ON` on newer DBR); `merge` runs `MERGE INTO`. 9.4: One batch per year from `begin` to today. 9.5: A lookback >= the lateness window (e.g., 7 days). 9.6: Counts and distinct keys should be stable; full refresh should match.

9.13 Interview check

- How does `is_incremental()` work, and when is it false?
- What is the difference between `merge`, `delete+insert`, and `replace_where`?
- How do you handle late-arriving data?
- Why can `append` create duplicates?
- What does `on_schema_change` do?
- When would you choose microbatch over a manual filter?
- How do you backfill an incremental model safely?
- How do you test idempotency?

PART VI - DATABRICKS

10. The Databricks Adapter in Depth

10.1 What the adapter does

`dbt-databricks` translates dbt's generic operations into Databricks SQL and APIs:

- Creates schemas, views, tables, incremental tables, materialized views, and streaming tables.
- Runs `MERGE`, `DELETE`, `INSERT`, `REPLACE WHERE`, and `INSERT OVERWRITE`.
- Reads metadata from Unity Catalog and `INFORMATION_SCHEMA`.
- Submits Python models as notebooks or jobs.
- Applies Unity Catalog tags, masks, row filters, and grants.
- Manages compute selection per model.

The adapter is a plugin. dbt Core provides the framework; the adapter provides the Databricks dialect.

10.2 Connection and environment strategy

A production-grade `profiles.yml` usually has three targets:

```yaml
dbt_dbx:
  target: dev
  outputs:
    dev:
      type: databricks
      catalog: analytics_lab
      schema: dev_jane
      host: dbc-xxxx.cloud.databricks.com
      http_path: /sql/1.0/warehouses/xxxxxxxxxxxxxxxx
      auth_type: oauth
      threads: 4

    ci:
      type: databricks
      catalog: analytics_lab
      schema: ci_pr_123
      host: dbc-xxxx.cloud.databricks.com
      http_path: /sql/1.0/warehouses/xxxxxxxxxxxxxxxx
      auth_type: oauth
      threads: 8

    prod:
      type: databricks
      catalog: analytics_lab
      schema: gold
      host: dbc-xxxx.cloud.databricks.com
      http_path: /sql/1.0/warehouses/xxxxxxxxxxxxxxxx
      auth_type: oauth
      threads: 8
```

Key decisions:

Decision	Recommendation
Catalog	One per environment/domain; isolate dev from prod
Schema	Per developer in dev, fixed in prod, per PR in CI
Compute	Serverless SQL warehouse for most models; job clusters for Python/large jobs
Auth	OAuth for humans; service principal for CI/CD
Threads	4-8 for a small warehouse; tune to concurrency
Token storage	Environment variable or secret manager; never Git

Separate catalogs vs separate schemas

- **Separate catalogs** (`analytics_dev`, `analytics_prod`) give strong isolation and simpler grants.
- **Separate schemas** in one catalog are cheaper to manage but easier to mix up.

For learning, one catalog with `dev_<name>` schemas is fine. For production, separate catalogs or at least separate schemas with least-privilege grants.

10.3 Unity Catalog and three-level naming

dbt-databricks uses the profile's `catalog` and `schema` to build `catalog.schema.identifier`.

```yaml
# profiles.yml
catalog: analytics_lab
schema: silver
```

```
-- model target
analytics_lab.silver.stg_orders
```

Model-level overrides:

```
{{ config(
    database='analytics_lab',   -- catalog on Databricks
    schema='gold',
    alias='orders'
) }}
```

Prefer `schema` and `alias`; use `database` only when you need to cross catalogs. Some teams use `catalogs.yml` for multi-catalog/mesh setups (newer feature); keep it simple unless you need cross-platform or multi-catalog builds.

Custom schema macro for Unity Catalog

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}
{%- endmacro %}
```

This produces `analytics_lab.silver`, `analytics_lab.gold`, exactly matching the medallion layers.

10.4 Databricks-specific table configs

Config	Purpose	Example
`table_format`	Delta or Iceberg compatibility	`iceberg`
`file_format`	Underlying file format	`delta`, `parquet`
`location_root`	External storage location	`/mnt/data/orders`
`partition_by`	Physical partitions	`['order_date']`
`liquid_clustered_by`	Liquid clustering columns	`['customer_id']`
`auto_liquid_cluster`	Let Databricks choose clustering	`true`
`tblproperties`	Delta table properties	`{'delta.autoOptimize.optimizeWrite': 'true'}`
`databricks_tags`	Unity Catalog tags	`{'domain': 'finance'}`
`compression`	File compression	`zstd`
`skip_optimize`	Skip post-build `OPTIMIZE`	`true`

Example:

```
{{ config(
    materialized='table',
    file_format='delta',
    partition_by=['order_date'],
    liquid_clustered_by=['customer_id'],
    tblproperties={
        'delta.autoOptimize.optimizeWrite': 'true',
        'delta.autoOptimize.autoCompact': 'true'
    },
    databricks_tags={'domain': 'finance', 'pii': 'false'}
) }}
```

Do not combine `partition_by` and `liquid_clustered_by`; Databricks does not allow both. Do not combine `liquid_clustered_by` and `auto_liquid_cluster` on the same model.

Partitioning vs liquid clustering

	Partitioning	Liquid clustering
Best for	Low-cardinality columns used in filters (date)	High-cardinality columns, changing access patterns
Physical layout	Directory per partition	Clustered files, no fixed partition dirs
Maintenance	`OPTIMIZE/ZORDER` or compaction	`OPTIMIZE`; Databricks can adapt
Risk	Too many small partitions	Requires recent DBR for best results

For new tables, liquid clustering is often the better default on Databricks. Use partitioning for very large, predictable date ranges and when downstream tools rely on partitions.

Iceberg

dbt-databricks supports `table_format: iceberg` (Delta UniForm or managed Iceberg, depending on version/config). Requirements and behavior vary by adapter version; check the current docs before adopting. For most Unity Catalog Delta tables, the default is fine.

10.5 Materialized views

```
{{ config(
    materialized='materialized_view',
    partition_by='order_date',
    schedule={
        'cron': '0 0 * * * ? *',
        'time_zone_value': 'Etc/UTC'
    },
    tblproperties={'delta.autoOptimize.optimizeWrite': 'true'}
) }}

select
    order_date,
    customer_id,
    sum(order_total) as revenue
from {{ ref('fct_orders') }}
group by 1, 2
```

Requirements and notes:

- Unity Catalog and a serverless SQL warehouse (per Databricks docs).
- `dbt run` is effectively a deployment step; Databricks refreshes the MV.
- Query changes require a drop/recreate for MVs (except schedule changes).
- Supported configs include `partition_by`, `liquid_clustered_by` (1.11+), `tblproperties`, `databricks_tags`, `schedule`, `row_filter` (1.12+).
- `on_configuration_change` controls how dbt handles config changes.
- MVs are not a universal replacement for incremental models; they have SQL limitations and refresh costs.

Use MVs for:

- Aggregations that back dashboards.
- Queries where Databricks can incrementally maintain results.
- Serving layers where freshness can be seconds/minutes rather than immediate.

10.6 Streaming tables

```
{{ config(
    materialized='streaming_table',
    partition_by='ingest_date',
    schedule={'every': '10 MINUTES'}
) }}

select *
from stream read_files(
    '/Volumes/analytics_lab/bronze/landing/orders/',
    format => 'json',
    header => true
)
```

Notes:

- `streaming_table` is provided by dbt-databricks.
- Requires Unity Catalog and serverless SQL warehouse/Lakeflow.
- Use `STREAM` and `read_files`/Auto Loader patterns for incremental ingestion.

- Changing the query does not reprocess already-ingested rows; use `--full-refresh` to reprocess available data.
- Supports schedules (`cron`, `every`, `on_update` with `at_most_every` in 1.12+), partitioning, liquid clustering (1.11+), tags, and row filters (1.12+).
- Streaming tables are ideal for making ingestion part of the dbt DAG, so lineage and tests cover the full path.

10.7 Python models

dbt-databricks supports Python models using PySpark:

```
import pyspark.sql.functions as F

def model(dbt, session):
    dbt.config(
        materialized="table",
        submission_method="all_purpose_cluster",
        create_notebook=True
    )

    df = dbt.ref("stg_orders")

    result = (
        df.groupBy("customer_id")
          .agg(
              F.count("order_id").alias("order_count"),
              F.sum("order_total").alias("lifetime_value")
          )
    )

    return result
```

Supported materializations: `table` and `incremental`. Python models cannot be views or ephemeral.

Submission methods:

Method	Behavior	Use
`all_purpose_cluster`	Runs on the cluster/warehouse from `http_path`; can use Command API or notebook	Development, fast iteration
`job_cluster`	Creates a short-lived job cluster per model	Production, cost control
`serverless_cluster`	Runs on serverless compute	Low-ops production
`workflow_job`	Creates/updates a reusable Databricks workflow	Complex orchestration; persistent artifacts

Config example:

```python
def model(dbt, session):
    dbt.config(
        materialized="incremental",
        submission_method="job_cluster",
        job_cluster_config={
            "spark_version": "15.4.x-scala2.12",
            "node_type_id": "i3.xlarge",
            "num_workers": 2
        },
        packages=["pandas==2.2.2"]
    )
    ...
```

Incremental Python models:

```python
def model(dbt, session):
    dbt.config(materialized="incremental", unique_key="customer_id")
    df = dbt.ref("stg_orders")
    if dbt.is_incremental:
        max_date = session.sql(
            f"select max(order_date) from {dbt.this}"
        ).collect()[0][0]
        df = df.filter(df.order_date >= max_date)
    return df
```

Limitations:

- Requires workspace/API access for notebook upload.
- Job clusters add startup time and cost.
- Debugging is harder than SQL; keep Python models few and well-tested.
- Unit tests and contracts are limited for Python models.
- Databricks Runtime compatibility matters; pin versions in `job_cluster_config`.

10.8 Selecting compute per model

Add named computes to the profile:

```yaml
dev:
  type: databricks
  catalog: analytics_lab
  schema: dev_jane
  host: dbc-xxxx.cloud.databricks.com
  http_path: /sql/1.0/warehouses/default
  auth_type: oauth
  threads: 4
  compute:
    small:
      http_path: /sql/1.0/warehouses/small
    large:
      http_path: /sql/1.0/warehouses/large
```

Then:

```yaml
models:
  dbt_dbx:
    marts:
      +databricks_compute: large
```

Or per model:

```
{{ config(materialized='table', databricks_compute='large') }}
```

Use cases:

- Send heavy marts to a larger warehouse.
- Keep light staging on a small warehouse.
- Isolate Python model SQL from interactive queries.

For Python models, `databricks_compute` controls the SQL side; use `cluster_id/http_path` or `job_cluster_config` for the Python execution.

10.9 Grants and Unity Catalog privileges

dbt can apply grants at build time:

```
{{ config(
    materialized='table',
    grants={
        'select': ['analyst_role', 'bi_service_principal'],
        'modify': ['data_engineering_role']
    }
) }}
```

Or in YAML:

```
models:
  - name: fct_orders
    config:
      grants:
        select: ["analyst_role"]
```

Notes:

- Grants are applied as `GRANT ... ON TABLE/VIEW` statements.
- On Unity Catalog, roles/groups must exist and the dbt principal must have grant authority.
- Prefer group-based grants over individual users.
- Use a post-hook for schema-level grants or complex policies.
- Review grants in PRs; a missing grant can break dashboards silently.

10.10 Tags, masks, and row filters

Table and column tags

```yaml
models:
  - name: dim_customers
    config:
      databricks_tags:
        domain: customer
        pii: "true"
    columns:
      - name: email
        databricks_tags:
          data_classification: pii
        column_mask:
          function: analytics_lab.security.mask_email
          using_columns: "customer_id"
```

Row filters

```yaml
models:
  - name: fct_orders
    config:
      row_filter:
        function: analytics_lab.security.region_filter
        columns: [region]
```

Supported on table, incremental, materialized view, and streaming table; not on regular views. The function must exist and the principal must have permission to apply it.

These features make dbt a governance tool, not just a transformation tool. Use them for PII, regional restrictions, and tenant isolation.

10.11 Delta performance features

Feature	What it does	How to use
`OPTIMIZE`	Compacts small files	dbt-databricks runs it after clustering builds; can be skipped
`ZORDER`	Co-locates related data (legacy)	`zorder` config / manual SQL
Liquid clustering	Adaptive clustering	`liquid_clustered_by` or `auto_liquid_cluster`
Predictive optimization	Databricks-managed maintenance	Enable in Databricks; consider `skip_optimize`
Auto compaction	Compacts small files on write	`delta.autoOptimize.autoCompact`
Optimized writes	Coalesces files on write	`delta.autoOptimize.optimizeWrite`
Photon	Vectorized query engine	Enable on warehouse/cluster
Caching	Databricks disk cache	Use for repeated BI queries

Do not blindly add `ZORDER` and liquid clustering; they compete for layout control. Pick one strategy per table and measure.

10.12 Databricks SQL warehouse sizing

Size	Use
2X-Small / X-Small	Learning, small dev workloads
Small / Medium	Typical dbt runs, small marts
Large	Heavy transformations, many concurrent users
2X-Large+	Very large joins/aggregations, backfills

Rules:

- Scale up for heavy transformations, out for concurrency.
- Serverless scales automatically but has startup latency.
- Set `threads` relative to warehouse capacity; too many threads cause queuing.
- Use query tags (dbt adds comments/metadata) to attribute cost.
- Monitor `system.query.history` and warehouse utilization.

10.13 Databricks-specific troubleshooting

Error	Cause	Fix
`PERMISSION_DENIED` on `CREATE TABLE`	Missing privilege on schema	Grant `CREATE TABLE`/`CREATE VIEW`
`CATALOG_NOT_FOUND`	Wrong catalog or no `USE CATALOG`	Check profile and grants
`DELTA_TABLE_NOT_FOUND`	Incremental target missing and `is_incremental()` logic assumes it exists	First run should create the table; check `--full-refresh`
`ConcurrentModificationException`	Two writes to the same Delta table	Avoid parallel writes to one table; serialize jobs
`MERGE` cardinality error	Duplicate keys in source	Deduplicate source or fix grain
`UNSUPPORTED_FEATURE` for MV	Unsupported SQL/config	Simplify query; check MV limitations
Python model timeout	Job cluster startup / long run	Increase timeout; use `job_cluster` efficiently
`read_files` path not found	Wrong volume/path or permission	Check Unity Catalog volume grants
Slow tests	Full scans	Add `where`, partition filters, or sample
Schema drift errors	Source added/removed columns	`on_schema_change`, contract, or explicit columns

10.14 Exercises

Exercise 10.1 - Configs. Create a table model with `partition_by='order_date'`, `liquid_clustered_by` removed, and `tblproperties` for auto-compaction. Inspect `SHOW CREATE TABLE`.

Exercise 10.2 - MV. Create a materialized view `mv_daily_revenue` from `rpt_revenue_daily`. Run it, then query it. Change the schedule and rerun. What changes in Databricks?

Exercise 10.3 - Streaming table. If your workspace has a volume, create a streaming table over JSON files using `read_files` and `STREAM`. If not, write the model and explain the requirements.

Exercise 10.4 - Python. Create a Python model that computes customer lifetime value from `stg_orders`. Use `all_purpose_cluster` in dev and `job_cluster` in prod. Compare run times.

Exercise 10.5 - Grants. Add a `grants` config to `fct_orders` for a group you own. Verify with `SHOW GRANTS ON TABLE`.

Exercise 10.6 - Compute. Add two named computes to your profile and route `marts` to the larger one. Verify in the logs which compute was used.

Answers. 10.1: `SHOW CREATE TABLE` shows `PARTITIONED BY`, `TBLPROPERTIES`, and no clustering. 10.2: Schedule changes are applied without a full drop/recreate; query changes may require one. 10.3: Requires UC + serverless SQL warehouse; use `STREAM read_files(...)`. 10.4: Job clusters take longer to start but cost less for long runs. 10.5: The principal needs grant authority. 10.6: Look for "using compute resource " in the logs.

10.15 Interview check

- What is the difference between a SQL warehouse and an all-purpose cluster in dbt?
- How does dbt-databricks map `catalog`, `schema`, and `alias` to Unity Catalog?
- When would you use a materialized view instead of an incremental model?
- What are the requirements for streaming tables?
- How do you run Python models cost-effectively?
- How do you apply row filters and column masks through dbt?
- What causes a Delta concurrent modification error, and how do you avoid it?
- How do you choose partition columns versus liquid clustering?

PART VII - ADVANCED ENGINEERING

11. Configuration, Selection, State, and Deployment Patterns

11.1 Variables and environment configuration

Variables let you parameterize logic without editing SQL:

```yaml
# dbt_project.yml
vars:
  start_date: "2024-01-01"
  payment_methods: ["card", "bank_transfer"]
```

```sql
where order_date >= '{{ var("start_date") }}'
```

Override at runtime:

```bash
dbt run --vars '{"start_date": "2024-06-01"}'
```

Precedence:

1. CLI `--vars`.
2. `vars.yml` (dbt 1.12+) or `dbt_project.yml` `vars:`.
3. Macro default `var('name', default)`.

dbt 1.12 introduced `vars.yml` at the project root. Variables defined there are available during project parsing, which is useful for project-level config.

Environment variables

```yaml
# profiles.yml
token: "{{ env_var('DATABRICKS_TOKEN') }}"
```

```sql
select '{{ env_var("DBT_ENV", "dev") }}' as environment
```

Rules:

- Never put secrets in `dbt_project.yml` or model files.
- Use `env_var` with a default only for non-secrets.
- In CI, inject secrets from the CI secret store.
- In dbt platform, use environment variables and extended attributes.

Target-based logic

```
{% if target.name == 'prod' %}
    where order_date >= dateadd(day, -365, current_date)
{% else %}
    where order_date >= dateadd(day, -30, current_date)
{% endif %}
```

Use sparingly. Hidden environment-specific logic makes production behavior hard to reproduce. Prefer variables with explicit defaults.

11.2 Node selection

Selection determines what runs. Master these operators:

Selector	Meaning
`stg_orders`	Exact model
`staging`	Folder/name path
`tag:finance`	Tag
`config.materialized:incremental`	Config
`source:bronze.orders_raw`	Source
`exposure:revenue_dashboard`	Exposure
`test_type:unit`	Unit tests
`test_type:data`	Data tests
`resource_type:model`	Resource type
`package:dbt_utils`	Package
`state:modified`	Changed vs state
`result:error`	Failed in previous run
`selector:my_selector`	YAML selector

Graph operators:

```
dbt run -s +fct_orders          # ancestors + model
dbt run -s fct_orders+          # model + descendants
dbt run -s +fct_orders+         # both
dbt run -s @fct_orders          # model, ancestors, descendants
dbt run -s 2+fct_orders         # two levels up
```

Set operations:

```
dbt run -s "tag:finance,tag:marts"       # union
dbt run -s "tag:finance,tag:daily"       # intersection (comma within quotes)
dbt run -s "tag:finance --exclude tag:deprecated"
```

Use `dbt list` to preview selection before running.

YAML selectors

`selectors.yml`:

```yaml
selectors:
  - name: finance_daily
    definition:
      union:
        - method: tag
          value: finance
        - method: tag
          value: daily
    description: "Finance models that run daily"
```

Use:

```
dbt run --selector finance_daily
```

Selectors are version-controlled and easier to review than long CLI strings. dbt 1.12 adds composable selectors that can be referenced within `--select`/`--exclude`.

Indirect selection

By default, `dbt test -s fct_orders` selects tests directly on `fct_orders`. Tests on upstream models are not selected. Use:

```
dbt test -s fct_orders --indirect-selection=buildable
dbt build -s fct_orders --indirect-selection=eager
```

Options: `eager`, `buildable`, `cautious`, `empty`. Choose based on how much confidence you need versus cost.

11.3 State, deferral, and slim CI

State comparison uses a previous `manifest.json`:

```
dbt build -s state:modified+ --state ./prod-manifest
```

Typical slim CI:

1. Download the production manifest artifact.
2. Run only modified models and their descendants.
3. Defer unselected upstream refs to production relations.

```
dbt build `
  --select state:modified+ `
  --defer `
  --state ./prod-manifest
```

`--defer` makes `ref()` for unselected nodes point to the state environment (production), so you do not need to rebuild upstream tables in CI.

`dbt clone` copies relations (metadata-level clone) into a target schema, useful for creating a development environment cheaply:

```
dbt clone --state ./prod-manifest --target dev
```

dbt 1.12 also bundles the dbt State plugin (opt-in via `--manage-state`), which can skip/clone/reuse nodes based on logic and data freshness. dbt State is a paid platform capability; see Chapter 17.

State caveats

- The state manifest must come from a comparable environment (same dbt version, ideally same project).
- `state:modified` detects file changes, not data changes.
- Macros and packages can affect many nodes; state understands config/macro changes, but test it.
- Use `state:modified.body` or `state:modified.configs` for narrower comparisons when needed.

11.4 Commands you must know

Command	Purpose
`dbt init`	Create a project/profile
`dbt debug`	Test connection and config
`dbt deps`	Install packages
`dbt parse`	Parse project, write manifest
`dbt compile`	Render SQL without running
`dbt run`	Execute models
`dbt test`	Execute tests
`dbt build`	Run models, tests, seeds, snapshots in DAG order
`dbt seed`	Load CSV seeds
`dbt snapshot`	Run snapshots
`dbt source freshness`	Check source freshness
`dbt docs generate`	Build docs/catalog
`dbt docs serve`	Serve docs locally
`dbt ls` / `dbt list`	List selected nodes
`dbt show`	Preview model results
`dbt run-operation`	Run a macro
`dbt clone`	Clone relations from state
`dbt retry`	Retry failed nodes from last run
`dbt clean`	Remove `target/` and `dbt_packages/`
`dbt --version`	Versions

Useful flags:

```
--select / -s
--exclude
--full-refresh
--vars
--target
--profiles-dir
--project-dir
--threads
--fail-fast
--warn-error
--no-partial-parse
--defer --state
--event-time-start / --event-time-end
--empty            # build schema with no data (where supported)
--sample           # sample mode (platform)
```

11.5 Model versions and deprecation

Model versions let you evolve a model without breaking consumers:

```yaml
models:
  - name: dim_customers
    latest_version: 2
    versions:
      - v: 1
        deprecation_date: "2026-06-01"
      - v: 2
```

Consumers pin:

```
select * from {{ ref('dim_customers', version=1) }}
```

`ref('dim_customers')` resolves to the latest version. dbt can create a `dim_customers` pointer view when configured. Use versions for breaking changes; do not use them to avoid naming things well.

11.6 Groups, access, and dbt Mesh

Groups define ownership and access boundaries:

```yaml
groups:
  - name: finance
    owner:
      name: Finance Analytics
      email: finance-analytics@example.com
```

```yaml
models:
  - name: fct_orders
    group: finance
    access: protected   # private, protected, public
```

Access levels:

Access	Who can ref it
`private`	Same group only
`protected`	Same project (default)
`public`	Other projects (mesh)

Cross-project refs use `dependencies.yml` in the downstream project:

```yaml
projects:
  - name: finance_project
```

```sql
select * from {{ ref('finance_project', 'fct_orders') }}
```

dbt Mesh requires a platform account or compatible discovery setup. It lets teams own domains while sharing governed interfaces.

11.7 Hooks

Hooks run SQL before/after models or at run boundaries:

```sql
{{ config(
    pre_hook="delete from {{ this }} where order_date = current_date",
    post_hook="optimize {{ this }} zorder by (customer_id)"
) }}
```

Project-level hooks:

```yaml
on-run-start:
  - "{{ log('Starting run', info=True) }}"
on-run-end:
  - "grant select on all tables in schema {{ target.schema }} to role analyst"
```

Hook cautions:

- Hooks run for every selected model; they can be expensive.
- Use `{% if execute %}` guards in macros.
- Hooks are not a substitute for a proper orchestration step.
- Databricks `OPTIMIZE` in a post-hook can be costly on large tables; prefer scheduled maintenance or `skip_optimize`.

11.8 User-defined functions (UDFs)

dbt 1.11+ supports functions as first-class resources. Structure:

```
functions/
├── mask_email.sql
└── functions.yml
```

`functions/mask_email.sql`:

```sql
create or replace function {{ this }} (email string)
returns string
return concat('***@', split_part(email, '@', 2));
```

`functions/functions.yml`:

```yaml
version: 2

functions:
  - name: mask_email
    config:
      schema: security
    arguments:
      - name: email
        data_type: string
    returns:
      data_type: string
```

dbt 1.12 adds JavaScript UDFs (Snowflake/BigQuery) and Python UDFs on Databricks (Unity Catalog required). Check adapter support before relying on them. UDFs are useful for shared masking/tokenization logic, but they add deployment surface; document and test them.

11.9 dbt State and advanced reuse

dbt State (formerly state-aware orchestration) compares model logic and upstream data freshness and can skip, clone, or reuse nodes. Key config:

```yaml
models:
  - name: fct_orders
    config:
      state:
        lag_tolerance: "2 hours"
```

Use `dbt state explain -s fct_orders` (v2) or `dbt-state explain` (v1) to see decisions. Requirements:

- dbt platform account for authentication.
- Opt-in via `--manage-state`, `DBT_ENGINE_MANAGE_STATE=true`, or `manage_state: true`.
- Works with v1, v2, and platform; not a replacement for good incremental design.

Use cases: development without rebuilding upstream, CI cost reduction, scheduled jobs that only rebuild changed data.

11.10 Exercises

Exercise 11.1 - Variables. Add `start_date` and `end_date` variables. Use them in `fct_orders` to filter dates. Run with different CLI values and confirm the compiled SQL changes.

Exercise 11.2 - Selection. Run `dbt list` for: all models tagged finance, all ancestors of `fct_orders`, all descendants of `stg_orders`, and all models excluding staging. Save the commands in a README.

Exercise 11.3 - State. Create a production-like manifest by running `dbt parse` on a clean branch, then modify one model and run `dbt build -s state:modified+ --defer --state <dir>`. Which nodes run?

Exercise 11.4 - Selector. Create `selectors.yml` with a `daily_finance` selector and run it.

Exercise 11.5 - Hooks. Add a post-hook to `rpt_revenue_daily` that logs a row count. Verify it runs.

Exercise 11.6 - Versions. Create version 1 and version 2 of `dim_customers`, set `latest_version: 2`, and update one downstream model to pin version 1. Compile both.

Answers. 11.1: Use `{{ var('start_date') }}`; `--vars` overrides. 11.2: `dbt list -s tag:finance`; `dbt list -s +fct_orders`; `dbt list -s stg_orders+`; `dbt list -s resource_type:model --exclude staging`. 11.3: Modified model plus descendants; deferred upstream refs point to state. 11.4: Use `--selector daily_finance`. 11.5: Look for the log line in stdout. 11.6: `ref('dim_customers', version=1)` compiles to the v1 relation.

11.11 Interview check

- What is the difference between `--select` and `--exclude`?
- How does `state:modified` work, and what are its limitations?
- What does `--defer` do?
- How do you design a slim CI pipeline?
- When would you use model versions?
- What is the difference between `private`, `protected`, and `public` access?
- How do hooks interact with failures?
- What is dbt State, and how is it different from `state:modified`?

12. Orchestration and CI/CD

12.1 Local, CI, and production

A mature setup has three environments:

Environment	Purpose	Data	Trigger
Development	Iterate on models	Small/dev schemas	Developer
CI	Validate a pull request	Isolated schema, production manifest	PR
Production	Serve consumers	Production schemas	Schedule/event

The same code should run in all three; only configuration, credentials, and data differ.

12.2 Git workflow

A simple, effective workflow:

1. Branch from `main`: `feature/add-refunds`.
2. Make changes; run `dbt build -s state:modified+` locally.
3. Open a PR; CI runs `dbt build` in an isolated schema.
4. Review SQL, tests, docs, and lineage impact.
5. Merge; production deploys on a schedule or on merge.
6. Tag releases if you need reproducibility.

Rules:

- Never commit `target/`, `dbt_packages/`, `logs/`, `.env`, or `profiles.yml`.
- Pin dbt and package versions.
- Require tests for new models.
- Review changes to incremental logic and contracts carefully.

- Use `dbt list -s state:modified+` in the PR description.

12.3 GitHub Actions: a complete slim CI example

`.github/workflows/dbt-ci.yml`:

```yaml
name: dbt CI

on:
  pull_request:
    branches: [main]

jobs:
  dbt-build:
    runs-on: ubuntu-latest
    env:
      DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
      DATABRICKS_HTTP_PATH: ${{ secrets.DATABRICKS_HTTP_PATH }}
      DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}
      DBT_PROFILES_DIR: .
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Write profile
        run: |
          cat > profiles.yml <<EOF
          dbt_dbx:
            target: ci
            outputs:
              ci:
                type: databricks
                catalog: analytics_lab
                schema: ci_pr_${{ github.event.number }}
                host: ${{ secrets.DATABRICKS_HOST }}
                http_path: ${{ secrets.DATABRICKS_HTTP_PATH }}
                token: ${{ secrets.DATABRICKS_TOKEN }}
                threads: 8
          EOF

      - name: Download production manifest
        run: |
          mkdir -p state
          # Replace with your artifact source (dbt platform, S3, GitHub release)
          # curl -o state/manifest.json "${{ secrets.PROD_MANIFEST_URL }}"

      - name: dbt deps
        run: dbt deps

      - name: dbt build (modified +)
        run: |
          dbt build --select state:modified+ --defer --state state --fail-fast

      - name: Upload artifacts
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: dbt-artifacts
          path: target/
```

Notes:

- Use GitHub secrets or an OIDC integration; never hard-code tokens.
- A service principal is better than a personal token.

- `state:modified+` requires a production manifest; without it, build everything (fine for small projects).
- Add a step to clean up the CI schema after the PR (a scheduled cleanup or a TTL policy).
- If you use dbt platform, its CI job can handle deferral and artifacts for you.

12.4 Production deployment

Options:

Orchestrator	How it runs dbt	Best for
dbt platform jobs	Native dbt scheduling, environments, CI, catalog	Teams wanting managed dbt
Databricks Workflows	`dbt` task (task type) or shell task	Teams already on Databricks
Airflow	BashOperator, PythonOperator, Cosmos	Complex cross-system DAGs
Dagster	dbt assets integration	Asset-oriented orchestration
Prefect	Shell/Python tasks	Python-centric orchestration
Cron + VM	`dbt build` in a script	Small teams, prototypes (not ideal)

Production checklist:

- Separate credentials and catalog/schema.
- Run `dbt build`, not just `dbt run`.
- Alert on failure (Slack, email, PagerDuty).
- Capture artifacts (`manifest.json`, `run_results.json`).
- Set retries for transient warehouse errors.
- Prevent overlapping runs on the same tables.
- Run source freshness before or alongside the build.
- Publish docs/catalog after deployment.
- Back up snapshots and critical tables.

12.5 Databricks Workflows

Databricks has a native dbt task type. A typical workflow:

1. Sync the Git repo to a Databricks Git folder.
2. Create a job with a `dbt` task pointing to the project directory.
3. Select commands: `dbt deps`, `dbt build`.
4. Choose a SQL warehouse or job cluster.
5. Add notifications and retries.
6. Schedule or trigger on file arrival.

Benefits: no external orchestrator, Unity Catalog integration, job compute. Limitations: less flexible than Airflow for cross-system dependencies.

12.6 Airflow example

Using `BashOperator`:

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

with DAG(
    "dbt_daily",
    start_date=datetime(2024, 1, 1),
    schedule="0 6 * * *",
    catchup=False,
) as dag:
    dbt_build = BashOperator(
        task_id="dbt_build",
        bash_command="cd /opt/dbt/dbt_dbx && dbt build --target prod",
        env={
            "DATABRICKS_TOKEN": "{{ conn.databricks.password }}",
        },
    )
```

Better: use Astronomer Cosmos to render each dbt model as an Airflow task with lineage and selective runs.

12.7 Scheduling patterns

- **Daily batch**: source freshness -> staging -> marts -> tests -> docs. Most common.
- **Hourly incremental**: only recent partitions; avoid full marts if not needed.
- **Event-driven**: trigger on new files/streams; streaming tables or Auto Loader.
- **Backfill**: isolated runs with event-time bounds; do not collide with scheduled runs.
- **Blue/green**: build into a new schema, validate, swap views/aliases; complex but safe for critical marts.

12.8 Monitoring and alerting

Monitor:

- Run status and duration (`run_results.json`).
- Test failures and warnings.
- Source freshness.
- Row counts/volume anomalies.
- Warehouse cost and queuing.
- Downstream BI freshness.

Alert on:

- Failed production build.
- Freshness SLA breach.
- Test failure on a critical model.
- Run duration above threshold.
- Cost anomaly.

Use dbt artifacts, the dbt platform UI, Elementary, or Databricks system tables. Alert fatigue is real: route only actionable failures to humans.

12.9 Exercises

Exercise 12.1 - Local CI simulation. Create a branch, modify a model, and run `dbt build -s state:modified+ --defer --state <prod_manifest>` against a CI schema.

Exercise 12.2 - GitHub Actions. Add the workflow above to a GitHub repo (use a test Databricks workspace/catalog). Open a PR and confirm the job runs.

Exercise 12.3 - Databricks Workflow. Create a Databricks job with a dbt task that runs `dbt build`. Trigger it manually and inspect the run.

Exercise 12.4 - Alerting. Configure a failure notification (email/Slack) for your job. Force a test failure and confirm the alert.

Exercise 12.5 - Cost review. Compare warehouse usage for a full build versus a `state:modified+` build. Record the difference.

Answers. 12.1: Only modified nodes and descendants run; upstream refs are deferred. 12.2: Ensure the service principal has warehouse/catalog permissions. 12.3: The dbt task needs the project files in a Git folder or workspace path. 12.4: Use the orchestrator's notification settings. 12.5: Expect a large reduction when few models change.

12.10 Interview check

- What is slim CI, and what does it require?
- How do you deploy dbt to production?
- How do you handle secrets in CI?
- What do you alert on?
- How do you prevent two production runs from colliding?
- What is the difference between `dbt run` and `dbt build` in production?
- How do you roll back a bad deployment?

13. Performance and Cost Optimization

13.1 Where time and money go

A dbt run's cost is dominated by:

1. **Warehouse compute** for model SQL and tests.
2. **Data scanned** (full table scans, missing partition filters).
3. **Shuffles/joins** in Spark.
4. **Small files** and table maintenance.
5. **Idle warehouses** and oversized compute.
6. **Redundant rebuilds** (running everything every time).
7. **Tests** that scan large tables.

Optimize in that order. A prettier DAG does not reduce cost if every model scans a billion rows.

13.2 Modeling for performance

- **Filter early**: push date filters into staging or source CTEs.
- **Select only needed columns**: wide tables cost I/O and memory.
- **Avoid views on views on views**: each layer re-executes.
- **Pre-aggregate** where the grain allows.
- **Use incremental** for large append-only facts.
- **Partition/cluster** on common filter columns.
- **Avoid exploding joins**: check cardinality before joining.
- **Use `QUALIFY`/window functions** instead of self-joins where possible.
- **Materialize reusable intermediates** as views, not ephemeral monsters.

13.3 Databricks tuning

Lever	Effect
Warehouse size	More compute per query; scale up for heavy transforms
Serverless	Automatic scaling; startup latency
Photon	Faster SQL/Spark; enable where supported
Liquid clustering	Better data skipping for selective filters
Partitioning	Prunes data for date filters
`OPTIMIZE`	Compacts files; can be scheduled
Predictive optimization	Managed maintenance; avoid duplicate manual optimize
`ANALYZE TABLE`	Statistics for cost-based optimization
Cache	Speeds repeated BI queries
File size	Aim for ~100-256 MB files (guideline, not law)

13.4 dbt-level tuning

- `threads`: match warehouse concurrency; more is not always faster.
- `--select` / `state:modified+`: build only what changed.
- `dbt build` interleaves tests; if tests are expensive, run them selectively.
- Disable unnecessary models in dev (`enabled: false` or `--exclude`).
- Use `--empty` for schema-only CI (where supported).
- Avoid `--full-refresh` on huge models unless necessary.
- Limit `store_failures` to failing tests and clean up tables.
- Use `dbt retry` instead of rerunning everything.
- Cache parsing with partial parse (v1) or the Rust parser (v2).

13.5 Test cost

Tests are queries too. A `unique` test on a billion-row table scans it. Strategies:

- Add `where` filters for recent partitions.
- Test at the staging layer where data is smaller.
- Use `limit` for exploratory tests, not production correctness.

- Use unit tests for logic instead of scanning production data.
- Run heavy tests less frequently (weekly) and cheap tests every run.
- Use dbt's `warn_if`/`error_if` thresholds to avoid noise.
- In v2, `--batch-tests` can combine compatible tests into fewer queries.

13.6 Measuring

Artifacts to inspect:

- `run_results.json`: per-node execution time, status, adapter response.
- `manifest.json`: graph and configs.
- `catalog.json`: table stats.
- Databricks query history: duration, bytes read, warehouse.
- `system.query.history`: cost attribution by user/tag.
- `system.billing.usage`: DBU consumption.

A simple performance review:

1. Find the 10 slowest nodes.
2. For each, check bytes scanned, shuffle, and join cardinality.
3. Decide: filter earlier, change materialization, partition/cluster, or split the model.
4. Measure again; keep a before/after table.

13.7 Cost guardrails

- Set warehouse auto-stop (5-10 minutes) for dev.
- Use serverless for sporadic workloads; provisioned for steady high concurrency.
- Separate dev/CI/prod warehouses; restrict prod access.
- Tag queries/models by team/domain for chargeback.
- Alert on long-running queries.
- Review full refreshes and backfills before running.
- Clean up CI schemas automatically.
- Avoid `SELECT *` in tests and models.
- Use dbt State or state selection to avoid redundant builds.

13.8 Exercises

Exercise 13.1 - Slowest nodes. Run `dbt build` and parse `run_results.json` to list the five slowest nodes. Identify one optimization for each.

Exercise 13.2 - Partition pruning. Create a table partitioned by `order_date`. Run a query filtered on `order_date` and compare bytes scanned with an unfiltered query (Databricks query history).

Exercise 13.3 - Threads. Run the same build with `--threads 2` and `--threads 8`. Compare duration and warehouse queuing.

Exercise 13.4 - State selection. Measure a full build versus `state:modified+`. Record cost/time.

Exercise 13.5 - Test cost. Add a `where` filter to a large test and compare runtime.

Answers. 13.1: Look for full scans, exploding joins, and unnecessary tables. 13.2: Partition pruning should reduce bytes read dramatically. 13.3: More threads may help or hurt depending on warehouse size and DAG shape. 13.4: Slim builds usually win when few models change. 13.5: Filtered tests scan less data.

13.9 Interview check

- How do you find the most expensive model in a dbt run?
- When does increasing `threads` hurt?
- How does partitioning reduce cost?
- What is the difference between `OPTIMIZE` and `ZORDER`?
- How do you keep tests cheap?
- How would you reduce a $10k/month dbt bill?
- What metrics would you put on a dashboard for dbt operations?

14. Governance, Security, and dbt Mesh

14.1 Governance goals

- **Correctness**: tests, contracts, reviews.
- **Discoverability**: docs, lineage, ownership.
- **Access control**: least privilege, PII protection.
- **Auditability**: who changed what, when.
- **Accountability**: owners and SLAs.
- **Interoperability**: stable interfaces across teams.

dbt supports all of these, but none are automatic. Governance is a set of habits enforced by tooling.

14.2 Ownership and groups

Use `groups` and `meta` to record ownership:

```yaml
groups:
  - name: finance
    owner:
      name: Finance Analytics
      email: finance-analytics@example.com

models:
  - name: fct_orders
    group: finance
    meta:
      owner: finance-analytics
      sla: "daily by 07:00 UTC"
      tier: "gold"
```

Expose ownership in docs and catalog. Review it quarterly; stale owners are worse than none.

14.3 Access control

Databricks/Unity Catalog

- Grant `USE CATALOG`/`USE SCHEMA` broadly only where appropriate.
- Grant `SELECT` on gold marts to BI groups.
- Do not grant `MODIFY` on production tables to analysts.
- Use service principals for dbt jobs, not personal accounts.
- Use row filters/column masks for sensitive data.
- Audit grants with `SHOW GRANTS` and system tables.

dbt access levels

```yaml
models:
  - name: fct_orders
    group: finance
    access: public
```

- `private`: same group only.
- `protected`: same project (default).
- `public`: cross-project/mesh.

Public models are interfaces. Treat them like APIs: contracts, tests, docs, versioning.

14.4 Contracts and interfaces

Use contracts for public marts and cross-team dependencies. A contract:

- Locks column names and types.
- Makes breaking changes visible in CI.
- Enables safe downstream refs in mesh.

Not every model needs a contract. Over-contracting staging slows iteration.

14.5 PII and sensitive data

Patterns:

1. **Tag** columns/tables (`pii: true`, `data_classification`).
2. **Mask** columns with Unity Catalog mask functions.
3. **Filter** rows with row filters (region, tenant).
4. **Minimize** data: do not select PII into marts unless needed.
5. **Hash/tokenize** identifiers where analytics only needs joins.
6. **Audit** access and queries.
7. **Document** data handling in the model description.

Example mask function:

```
CREATE OR REPLACE FUNCTION analytics_lab.security.mask_email(email STRING)
RETURNS STRING
RETURN concat('***@', split_part(email, '@', 2));
```

Apply via dbt config as shown in Chapter 10.

14.6 dbt Mesh

dbt Mesh lets domain teams own separate dbt projects and share public models:

- Each project has its own repo, schedule, and owners.
- Cross-project refs use `dependencies.yml`.
- Public models are governed interfaces.
- The platform provides discovery and lineage across projects.

When to use mesh:

- Multiple teams with distinct domains and release cadences.
- A monorepo is becoming a bottleneck.
- You need clear ownership boundaries.

When not to:

- Small team, single domain.
- You cannot invest in contracts, docs, and coordination.
- The overhead exceeds the coordination cost.

14.7 Project health checks

Use `dbt_project_evaluator` and custom checks:

- Every model has a description and owner.
- Every source has freshness (where applicable).
- Every fact/dimension has PK tests.
- No duplicate sources or models.
- Naming conventions are followed.
- No models with zero downstream consumers.
- No direct source references in marts.
- Public models have contracts.

Treat failures as a prioritized backlog.

14.8 Security of the dbt project itself

- Secrets never in Git; use environment variables/secret managers.
- Restrict who can merge to `main`.
- Pin packages and review upgrades.
- Run `dbt deps` in CI from a lock file.
- Scan for accidental PII in seeds and tests.

- Limit `store_failures` tables; they can contain sensitive rows.
- Protect production credentials; use a service principal with least privilege.
- Log and monitor access to artifacts.

14.9 Exercises

Exercise 14.1 - Ownership. Add groups and owners to three models. Generate docs and confirm ownership appears (where supported).

Exercise 14.2 - Contract. Make `fct_orders` public and add a contract. Break a column type intentionally and confirm CI fails.

Exercise 14.3 - PII. Tag an email column, create a mask function, and apply it. Query the table as a non-privileged user and confirm the mask.

Exercise 14.4 - Row filter. Create a region filter function and apply it to a table. Verify with a test user.

Exercise 14.5 - Evaluator. Install `dbt_project_evaluator` and run it. List the top five project health issues.

Answers. 14.1: Ownership appears in docs/catalog metadata. 14.2: The contract failure lists the mismatched type. 14.3: The mask function must be owned/granted correctly. 14.4: Row filters require supported materializations and UC. 14.5: Typical issues: missing descriptions, missing tests, duplicate sources, naming violations.

14.10 Interview check

- How do you implement least privilege for a dbt project?
- What is the difference between a contract and a test?
- How do you protect PII in a gold table?
- When would you use dbt Mesh?
- How do you make a model a stable interface for other teams?
- What would you monitor for governance violations?
- How do you handle a breaking change to a public model?

PART VIII - INTELLIGENCE

15. The dbt Semantic Layer and Metrics

15.1 Why a semantic layer

Without a semantic layer, every dashboard, notebook, and analyst defines "revenue" slightly differently. The result is the classic meeting where three numbers claim to be revenue.

A semantic layer defines metrics **once**, in code, and serves them consistently to BI tools, APIs, and AI agents. dbt's implementation is built on **MetricFlow**, which translates metric requests into SQL against your dbt models.

The stack:

```
dbt models (facts/dimensions)
        |
Semantic models (entities, dimensions, measures/simple metrics)
        |
Metric definitions (simple, ratio, derived, cumulative, conversion)
        |
MetricFlow query planner
        |
BI tools / APIs / AI agents / exports
```

15.2 Semantic model concepts

Concept	Meaning	Example
Semantic model	A dbt model annotated with semantic metadata	`fct_orders`
Entity	A join key with a type	`order` (primary), `customer` (foreign)
Dimension	A way to group/filter	`order_date`, `market_segment`
Measure / simple metric	An aggregation over a column	`sum(order_total)`
Metric	A business calculation	revenue, AOV, conversion rate
Time spine	A table of dates used for time-based calculations	one row per day

Entity types:

- **Primary**: one row per entity in this model, unique and non-null.
- **Unique**: unique but may be a subset, may contain nulls.
- **Foreign**: many rows can reference the same entity.
- **Natural**: a real-world key (for example, a national ID).

Dimensions:

- **Categorical**: group-by attributes (country, segment).

- **Time**: dates/timestamps with a granularity (day, month, year).

15.3 The v1.12 semantic model syntax

dbt 1.12 introduced a new YAML specification that nests semantic metadata under the model. A complete example:

```yaml
version: 2

models:
  - name: fct_orders
    description: "One row per order."
    semantic_model:
      enabled: true
      name: orders

    agg_time_dimension: order_date

    columns:
      - name: order_id
        entity:
          type: primary
          name: order

      - name: customer_id
        entity:
          type: foreign
          name: customer

      - name: order_date
        granularity: day
        dimension:
          type: time
          name: order_date

      - name: order_status
        dimension:
          type: categorical
          name: order_status

      - name: payment_method_name
        dimension:
          type: categorical
          name: payment_method

    metrics:
      - name: order_revenue
        description: "Sum of order totals."
        type: simple
        label: "Order Revenue"
        agg: sum
        expr: order_total

      - name: order_count
        description: "Number of orders."
        type: simple
        label: "Order Count"
        agg: count
        expr: order_id

      - name: average_order_value
        description: "Revenue divided by order count."
        type: simple
        label: "Average Order Value"
        agg: average
        expr: order_total
```

A customer dimension model:

```yaml
models:
  - name: dim_customers
    semantic_model:
      enabled: true
    columns:
      - name: customer_id
        entity:
          type: primary
          name: customer
      - name: market_segment
        dimension:
          type: categorical
          name: market_segment
      - name: first_order_date
        granularity: day
        dimension:
          type: time
          name: first_order_date
```

Key rules:

- Each dbt model can define one semantic model.
- Entity names are the join keys; use the same entity name across models to join.
- Dimensions are scoped to the primary entity of their semantic model.
- `agg_time_dimension` is the default time dimension for simple metrics.
- Avoid double underscores in semantic model names.

> **Version note.** Older projects (pre-1.12) use top-level `semantic_models:` with `measures:` and `dimensions:`. The concepts are the same; the YAML shape differs. If you are on 1.11 or earlier, use the older syntax or upgrade. dbt 1.12 also supports Apache Ossie documents as an alternative semantic spec.

15.4 Metric types

Type	Definition	Example
Simple	Aggregation over one column	`sum(order_total)`
Ratio	Numerator / denominator	revenue / orders
Derived	Expression over other metrics	revenue - refunds
Cumulative	Accumulates over time	cumulative revenue
Conversion	Funnel/conversion	visit -> purchase

Simple metric

```yaml
metrics:
  - name: order_revenue
    type: simple
    agg: sum
    expr: order_total
```

Ratio metric (top-level)

```yaml
metrics:
  - name: average_order_value
    type: ratio
    numerator: order_revenue
    denominator: order_count
    label: "Average Order Value"
```

Derived metric

```yaml
metrics:
  - name: net_revenue
    type: derived
    expr: order_revenue - refund_amount
    label: "Net Revenue"
```

Cumulative metric

```yaml
metrics:
  - name: cumulative_revenue
    type: cumulative
    input_metric: order_revenue
    window: "7 days"   # optional; omit for unbounded
```

Cumulative and conversion metrics need a time spine. Configure it in `dbt_project.yml`:

```yaml
vars:
  metricflow_time_spine_model: ref('metricflow_time_spine')
```

The time spine model:

```sql
{{ config(materialized='table') }}

select
    date_day as date_day
from {{ ref('date_spine') }}
```

15.5 Filters and dimensions

Metrics can be filtered:

```yaml
metrics:
  - name: fulfilled_revenue
    type: simple
    agg: sum
    expr: order_total
    filter: "{{ Dimension('order__order_status') }} = 'F'"
```

Dimensions are referenced as `entity__dimension`, for example `order__order_status` or `customer__market_segment`.

15.6 Querying metrics

In the dbt platform/local CLI with Semantic Layer access:

```
dbt sl list metrics
dbt sl query --metrics order_revenue,order_count --group-by order__order_status
dbt sl query --metrics order_revenue --group-by order__order_date --order-by order__order_date
```

In self-hosted setups, MetricFlow commands may use `mf query` or the dbt CLI depending on installation. The important concept: you request metrics and dimensions; MetricFlow generates the SQL, including the joins.

Example generated query (conceptual):

```
select
    order_status,
    sum(order_total) as order_revenue,
    count(order_id) as order_count
from analytics_lab.gold.fct_orders
group by order_status
```

The value is not the SQL; it is the guarantee that every consumer gets the same definition.

15.7 Exports

Exports materialize metric results for tools that cannot query the Semantic Layer directly:

```
exports:
  - name: daily_revenue_export
    config:
      export_as: table
      schema: gold
    metrics:
      - name: order_revenue
      - name: order_count
    dimensions:
      - order__order_date
```

Run exports with the platform's export command or schedule. Exports are useful for BI tools without a native Semantic Layer connector.

15.8 Testing metrics

- Add data tests to the underlying models (keys, relationships, ranges).
- Unit-test metric logic where supported.
- Reconcile metric totals to the source of truth.
- Check dimension join coverage (orphan foreign keys).
- Test time spine completeness.
- Monitor query latency and failures.

A metric is only as trustworthy as its models. The Semantic Layer does not replace data quality.

15.9 Databricks and BI integration

The Semantic Layer can serve:

- Tableau, Power BI, Excel, Google Sheets, Hex, Mode, and custom APIs.
- AI agents through the dbt MCP server.

- Exports to Delta tables for tools without connectors.

On Databricks, the Semantic Layer queries the gold models through a SQL warehouse. Ensure the service principal has `SELECT` on the semantic models and warehouse access, and that metrics use models with stable grains and tested keys.

15.10 Semantic Layer design practices

1. Model facts and dimensions first; metrics come after.
2. Define entities consistently across models (`customer`, `order`, `product`).
3. Use one primary entity per semantic model.
4. Keep metric names business-facing (`revenue`, not `sum_order_total`).
5. Document definitions, including exclusions and time zones.
6. Prefer simple metrics as building blocks; compose ratio/derived metrics.
7. Test joins and grains; fan-out silently inflates metrics.
8. Version breaking metric changes like API changes.
9. Limit the number of metrics; too many is as bad as none.
10. Reconcile with finance/BI before publishing.

15.11 Exercises

Exercise 15.1 - Semantic model. Add a semantic model to `fct_orders` with a primary entity `order`, a foreign entity `customer`, time dimension `order_date`, and simple metrics `order_revenue` and `order_count`.

Exercise 15.2 - Dimension model. Add a semantic model to `dim_customers` with a primary entity `customer` and a categorical dimension `market_segment`.

Exercise 15.3 - Ratio. Create `average_order_value` as a ratio of `order_revenue` to `order_count`.

Exercise 15.4 - Filter. Create `fulfilled_revenue` filtered to `order_status = 'F'`.

Exercise 15.5 - Query. Use `dbt sl` (or MetricFlow) to query revenue by market segment for the last 90 days.

Exercise 15.6 - Time spine. Create a date spine and configure `metricflow_time_spine_model`. Build a cumulative revenue metric.

Answers. 15.1: Follow the YAML in 15.3. 15.3: Use `type: ratio`, `numerator`, `denominator`. 15.4: Use a `filter` with `Dimension('order__order_status')`. 15.5: Group by `customer__market_segment` and filter on `order__order_date`. 15.6: The spine must have one row per day and a `date_day` column.

15.12 Interview check

- What problem does a semantic layer solve?
- What is the difference between a measure and a metric?
- What are entities, and why are they necessary for joins?
- What is a time spine used for?
- How do you test a metric?
- How do you handle a metric definition change?
- How does the Semantic Layer interact with BI tools?

16. AI in dbt: Wizard, Copilot, MCP, and Agent Skills

16.1 The AI landscape in dbt

As of 2026, dbt Labs organizes AI into a few distinct capabilities:

Capability	What it is	Where it runs
dbt Wizard	Agentic AI for the full development lifecycle: investigate, build, validate, ship	dbt platform Studio IDE, CLI, Desktop (beta)
dbt Copilot	Inline AI quick actions: generate SQL, docs, tests, semantic models; chat in Insights/Canvas	dbt platform
dbt MCP server	Standardized tools that expose dbt metadata, Semantic Layer, CLI, and APIs to AI agents	Local (self-hosted) or remote
Agent skills	Reusable instructions (`SKILL.md`) that teach a coding agent your project's conventions	Local project/package
dbt VS Code extension	LSP-powered editor features (completion, lineage, compiled SQL, errors)	VS Code / Cursor
Databricks AI	Assistant, Genie, AI/BI, AI functions, model serving	Databricks

The important shift: AI is not just autocomplete. It is grounded in **project context** - lineage, tests, contracts, metrics, and warehouse metadata. An agent that knows your DAG can answer "what breaks if I change this model?" far better than a generic chatbot.

16.2 dbt Wizard

Wizard is the recommended agent for governed data development. It can:

- Investigate lineage and downstream impact.
- Generate or refactor SQL from natural language.
- Generate documentation, tests, metrics, and semantic models.
- Compile/run against the warehouse to validate changes.
- Show diffs before persisting changes.
- Work across multiple tasks/branches (Desktop).

Install Wizard locally (Windows)

```
irm https://public.cdn.getdbt.com/dbt-wizard/install/install-wizard.ps1 | iex
```

Then, in your project:

```
wizard login
wizard "summarize this project and list models with no tests"
```

Useful commands:

Command	Purpose
`wizard "prompt"`	Start an interactive session
`wizard exec "prompt"`	Run one prompt non-interactively
`wizard review --uncommitted`	Review uncommitted changes
`wizard review --base main`	Review a branch diff
`wizard resume --last`	Resume a session
`wizard apply TASK_ID`	Apply the latest diff
`wizard login` / `logout`	Authenticate
`wizard mcp add dbt`	Connect MCP tools
`wizard update`	Update

Prerequisites:

- A built `target/` directory (`dbt parse`, `dbt compile`, or `dbt build`).
- Access to a supported AI provider (dbt-managed or BYOK).
- A dbt account for billing/authentication (Wizard usage is metered).

Wizard is warehouse-agnostic and works with dbt v1 and v2.

Good Wizard prompts

Be specific about the outcome and constraints:

- "Add a `dim_products` model from `stg_products` with a surrogate key, tests for unique/non-null, and a description of the grain. Follow the existing marts conventions."
- "Refactor `int_orders_enriched` to remove the payment method join and create `int_orders_payment_methods`. Update downstream refs and tests."
- "Why is `fct_orders` failing the `relationships` test to `dim_customers`? Inspect the test failures and propose a fix. Do not change the test until you explain the root cause."
- "Generate a semantic model for `fct_orders` with the `order` primary entity and `customer` foreign entity, plus simple metrics for revenue and order count."
- "Review this branch for breaking changes to public models and missing tests."

Guardrails

- Review every diff; never auto-apply to production.
- Ask for a plan before a multi-file refactor.
- Require the agent to run `dbt build -s ...` and show results.
- Keep changes small; large agent diffs are hard to review.
- Use Git branches and PRs; AI does not bypass code review.
- Do not paste secrets or sensitive data into prompts.

16.3 dbt Copilot

Copilot provides inline assistance in the dbt platform:

- Generate SQL for a model from a prompt.
- Generate documentation and tests for selected columns.
- Generate semantic models/metrics.
- Chat with data in Insights (powered by the Semantic Layer).
- Build visual models in Canvas (Enterprise+).

Copilot is configured separately from Wizard. Supported providers include dbt-managed OpenAI, BYOK OpenAI, and BYOK Azure OpenAI. Usage is metered in actions. AI features are enabled by default; admins can turn them off.

Copilot vs Wizard:

	Copilot	Wizard
Scope	Inline quick actions, chat	Full-lifecycle agent
Best for	Generate a test/doc/metric quickly	Multi-step investigation, refactor, validation
Availability	Platform plans (Starter+)	Platform + CLI + Desktop

If your organization has both, use Copilot for small inline tasks and Wizard for multi-file work.

16.4 The dbt MCP server

MCP (Model Context Protocol) is an open standard for connecting AI applications to tools and data. The dbt MCP server exposes:

- dbt CLI commands (self-hosted).
- Semantic Layer queries.
- Metadata discovery (models, lineage, tests, exposures).
- Administrative APIs.
- Codegen tools.
- dbt v2 tools.
- Product docs search.

Two flavors:

Flavor	Runs	Use
Self-hosted	On your machine via `uvx dbt-mcp`	Development, local CLI, full toolset
Remote	dbt-hosted HTTP endpoint	Consumption, no install, read-only use cases

Local setup with VS Code

1. Install `uv` (which provides `uvx`): follow the uv docs for Windows.
2. Create `.vscode/mcp.json` in your project:

```
{
  "servers": {
    "dbt": {
      "command": "uvx",
      "args": ["dbt-mcp"],
      "env": {
        "DBT_PROJECT_DIR": "${workspaceFolder}",
        "DBT_PROFILES_DIR": "${workspaceFolder}"
      }
    }
  }
}
```

3. Open the MCP panel in VS Code and start the server.
4. Ask your AI assistant (GitHub Copilot Chat, Claude, etc.) to use dbt tools.

Depending on your setup, you may also set `DBT_HOST`, `DBT_TOKEN`, `DBT_PROD_ENV_ID`, `DBT_ACCOUNT_ID`, and other variables. Refer to the dbt MCP environment variable reference for the full list.

What MCP enables

- "Which models depend on `stg_orders`?"
- "Run `dbt build -s state:modified+` and summarize failures."
- "Query revenue by market segment using the Semantic Layer."
- "Find models with no tests and no description."
- "Explain why this job failed."

MCP is a read/write tool layer; treat it with the same permissions and review discipline as a human with CLI access.

16.5 Agent skills

Agent skills are reusable instructions stored in `SKILL.md` files. They teach a coding agent your conventions:

```
skills/
└── dbt-modeling/
    └── SKILL.md
```

Example `SKILL.md`:

```
# dbt modeling conventions

- Staging models are prefixed `stg_` and materialized as views.
- Marts are prefixed `fct_` or `dim_` and materialized as tables.
- Every primary key has `unique` and `not_null` tests.
- Use `decimal(18,2)` for money, `date` for dates.
- Never use `select *` in marts.
- Add a description stating the grain of every mart.
```

Enable skills:

```
# dbt_project.yml
flags:
  ai_provider: claude   # wizard, claude, openai, codex, cursor, gemini
```

Include skills in your project's `skills/` directory or install a package that ships skills. `dbt deps` installs them into the agent's directory (for example `.claude/skills`); `dbt clean` removes them.

Skills are a lightweight way to make AI output consistent with your team's standards, without repeating instructions in every prompt.

16.6 AI in the VS Code extension

The dbt VS Code extension provides:

- Autocomplete for `ref`, sources, macros, and doc blocks.
- Go-to-definition and find references.
- Jinja syntax error detection.
- Compiled SQL preview and CTE preview.
- Lineage (visual and textual).
- Run/build/test commands.
- Column-level lineage and type checking with `static_analysis: strict` (v2).

This is not generative AI by itself, but it is the context layer that makes AI useful: the agent can see the project structure, compiled SQL, and lineage. The extension and Wizard are designed to work together.

16.7 Databricks AI capabilities

Databricks offers complementary AI features:

Feature	Use in a dbt workflow
Databricks Assistant	Generate/explain SQL in the editor; useful for exploring sources before modeling
Genie	Natural-language Q&A over governed data; can sit on gold tables
AI/BI dashboards	Consume marts and metrics
Unity Catalog AI functions	Call LLMs from SQL for classification/extraction
Model Serving	Deploy ML models; dbt can prepare features
Vector search	Build RAG pipelines over documents

A practical combination: dbt builds trusted gold tables and semantic models; Genie/BI serves them; the Databricks Assistant helps explore; dbt Wizard/MCP helps build and maintain the project.

16.8 AI-assisted development workflow

A disciplined loop:

1. **Understand**: ask the agent to summarize the model, its grain, upstream/downstream, and tests.
2. **Plan**: ask for a step-by-step plan and files to change. Review it.
3. **Implement**: let the agent edit in a branch, or apply changes yourself.
4. **Validate**: run `dbt build -s <selection>` and read the results. Do not trust "it should work."
5. **Review**: inspect the diff, tests, docs, and lineage impact.
6. **Ship**: PR, CI, human review.

Prompts that work well:

- Provide context: model names, business rules, constraints, target dialect.
- Ask for tests and documentation in the same request.
- Ask the agent to show the compiled SQL and explain assumptions.
- Constrain scope: "only touch these three files."
- Require validation: "run `dbt build -s ...` and paste the summary."

Prompts that fail:

- "Build me a data warehouse" (too broad).
- "Fix the tests" without showing failures.
- "Make it faster" without a baseline.
- "Use best practices" without defining them.

16.9 Security, privacy, and governance for AI

- AI providers may receive prompts, metadata, and query results you approve. Read the provider terms.
- Never send secrets, tokens, or PII in prompts.
- Use the least-privileged account for MCP/Wizard.
- Review generated SQL for correctness and cost (a Cartesian join is still a Cartesian join).
- Keep human review for production changes.
- Log AI-generated changes in PRs; disclose when material.
- Restrict MCP tools that can execute commands if your policy requires it.
- Prefer self-hosted MCP for development; use remote MCP for consumption.

16.10 Cost and access

- Wizard is metered by tokens/credits; Copilot by actions; MCP `text_to_sql` consumes Copilot actions.
- BYOK shifts billing to your provider but still requires configuration.
- Set spend limits and monitor usage.
- AI features are enabled by default on the platform; admins can disable them.
- CLI Wizard requires a dbt account and a built project.

16.11 Hands-on AI labs

Lab 16.1 - Project Q&A. Build the project, then ask Wizard or an MCP-connected assistant: "List the five models with the most downstream dependencies and explain the grain of each." Verify the answer against `dbt ls` and the YAML.

Lab 16.2 - Generate a model + tests + docs. Prompt: "Create `dim_products` from `stg_products` with a surrogate key, unique/not-null tests, a description of the grain, and a market segment dimension. Follow the existing marts conventions." Review the diff, run `dbt build -s dim_products`, fix anything the agent missed.

Lab 16.3 - Impact analysis. Ask: "What breaks if I rename `order_total` to `order_amount` in `fct_orders`?" Compare the answer to `dbt ls -s fct_orders+` and the docs lineage.

Lab 16.4 - Incident triage. Break a test intentionally. Ask the agent to diagnose using the test failure, the model SQL, and upstream data. Require a root cause before a fix.

Lab 16.5 - Semantic model. Ask the agent to generate a semantic model for `fct_orders`. Check entity names, grain, metric definitions, and tests. Query a metric.

Lab 16.6 - MCP. Configure the local MCP server in VS Code and ask: "Run `dbt build -s state:modified+` and summarize failures." Confirm it used the project's environment.

16.12 Exercises

Exercise 16.1. Write three prompts for common tasks: adding a test, refactoring a model, and explaining a failure. For each, list the context you must provide.

Exercise 16.2. Configure an agent skill with five project conventions. Test whether the agent follows them.

Exercise 16.3. Compare a Wizard-generated model with a hand-written one. What did the agent get right or wrong? What would you change in the prompt?

Exercise 16.4. Identify three risks of giving an AI agent CLI access to your dbt project. Propose mitigations.

Answers. 16.1: Context includes model names, grain, business rules, dialect, constraints, and validation commands. 16.2: Use `SKILL.md` and `ai_provider`; test with a small task. 16.3: Likely issues: missing tests, generic descriptions, wrong materialization. 16.4: Destructive commands, secret exposure, unreviewed production changes; mitigate with least privilege, sandbox schemas, and human review.

16.13 Interview check

- What is the difference between dbt Wizard and dbt Copilot?
- What is MCP, and why is it useful for data teams?
- How do you keep AI-generated dbt code safe?
- What context does an AI agent need to be useful in dbt?
- How would you evaluate whether AI improved your team's productivity?
- What are the privacy risks of sending dbt metadata to an AI provider?

17. dbt v2 / Fusion: What Changes and How to Migrate

17.1 What dbt v2 is

dbt v2 is the Rust-based engine (the productionized Fusion engine) that powers both local dbt and the dbt platform. It is installed with a new PyPI name:

```
pip install dbt
```

There are two distributions:

Distribution	Package	When
dbt v2	`dbt`	Recommended, full experience
dbt OSS	`dbt-oss`	Apache 2.0 runtime requirement

dbt v1.x remains supported. Both produce compatible manifests (v12), so they can run side by side during migration.

17.2 Why it matters

- **Faster parsing/compilation** via Rust, especially on large projects.
- **Stricter validation**: unknown configs, YAML keys, CLI flags, and missing macros fail at parse time.
- **Static analysis**: SQL is parsed into a logical plan; column-level lineage and type checks.
- **Built-in LSP**: editor features without a separate language server.
- **New capabilities**: checks, batch tests, docs v2, model freshness, `dbt lint`, dbt State integration, local unit tests (experimental), agent skills.

17.3 Installing and switching

For a new environment:

```
python -m venv .venv-dbt2
.\.venv-dbt2\Scripts\Activate.ps1
pip install dbt
dbt --version
```

Adapters: dbt v2 supports BigQuery, Databricks, Redshift, Snowflake, ClickHouse (private beta), Spark (beta), and DuckDB (CLI). For Databricks, use the v2-compatible `dbt-databricks` adapter (the docs state the dbt v2 Databricks adapter is the supported connection method for v2). Check the current compatibility matrix before migrating.

To test the v2 parser while staying on dbt 1.12:

```
dbt parse --use-v2-parser
```

This is opt-in and low-risk; it catches compatibility issues without changing runtime behavior.

17.4 What breaks: strict validation

In v1, misspelled configs and unexpected keys were often ignored. In v2 they error. Common fixes:

- Remove deprecated configs and flags.
- Fix `dbt_project.yml` keys that do not match the project name.
- Move top-level YAML anchors under `anchors:`.
- Remove self-referential YAML anchors.
- Replace `--models/-m` with `--select/-s`.
- Define missing variables and macros (v2 fails at parse, not compile).
- Resolve all deprecation warnings before upgrading.

Run `dbt-autofix` (or the dbt migration agent skill) to automate many fixes. Review its changes; do not blind-merge.

17.5 Static analysis

dbt v2 adds a static analysis phase:

```yaml
# dbt_project.yml
models:
  dbt_dbx:
    +static_analysis: strict
```

Modes:

Mode	Behavior
`baseline` (default)	Parses SQL, catches most errors as warnings
`strict`	Resolves column types and references; errors before execution; enables column-level lineage

With strict mode:

- Column-level lineage in docs.
- Column go-to-definition and find references in the LSP.
- Nonexistent-column and type detection.
- `SELECT *` expansion.
- Warehouse-specific syntax checks.

Strict mode requires the project to be resolvable; fix parse errors first. It can be applied per model or project-wide.

17.6 New commands and features

Feature	Command/config	Notes
Checks	`dbt check`	Project standards enforced at parse/build
Batch tests	`dbt test --batch-tests`	Groups compatible tests into fewer queries
Docs v2	`dbt docs generate`, `dbt docs serve`	Parquet artifacts, column lineage, DuckDB-WASM
Model freshness	`freshness` config, `dbt freshness`	Warn/error when model data is stale
Lint	`dbt lint [file] --fix`	SQLFluff-compatible
dbt State	`--manage-state`, `dbt state explain`	Reuse/skip/clone based on logic + freshness
Local unit tests	`compute: local` + env flag	Experimental DuckDB execution
Agent skills	`skills/`, `ai_provider`	`dbt deps` installs skills
Info schema	`--generate-info-schema`	Parquet metadata for agents/tools
`dbt login`	`dbt login`	Platform authentication

Checks

`checks/` contains SQL rules that enforce project standards:

```sql
-- checks/models_have_descriptions.sql
select *
from {{ ref('...') }}
where description is null
```

Exact check syntax depends on the current v2 release; the concept is a parse-time policy layer. Checks run automatically with `dbt build`; use `--skip-checks` to bypass.

17.7 Behavior changes to know

- `dbt build` runs all unit tests first (v1 ran them in lineage order).
- `dbt compile` continues compiling unrelated nodes after an error; downstream nodes are skipped.
- `get_relation()` prints the fully qualified name at parse time, not `None`.
- Project macros are available in YAML in v2 (not in v1).
- `config.get()` does not return `meta` values; use `config.meta_get()`/`meta_require()`.
- `dbt clean` will not delete files outside the project or in resource paths.
- Seeds with extra commas do not create extra columns.
- Threads: v2 manages Databricks/Snowflake parallelism automatically; `threads` is a cap.
- Duplicate docs blocks and package version conflicts error.
- Some deprecated flags are no-ops or errors.

17.8 Migration plan

1. **Inventory**: list custom macros, packages, materializations, and deprecated configs.
2. **Upgrade to 1.12** first if you are older; fix deprecations.
3. **Test the parser**: `dbt parse --use-v2-parser`.
4. **Create a v2 environment**: separate venv, install `dbt` and the compatible adapter.
5. **Run `dbt-autofix`** in a branch; review the diff.
6. **Run `dbt parse`, `dbt compile`, `dbt build`** on a dev schema.
7. **Enable `static_analysis: baseline`**, then `strict` where feasible.
8. **Validate packages** for v2 compatibility (`require-dbt-version` includes 2.0, or test them).
9. **Run CI in parallel** (v1 and v2) until confident.
10. **Switch production** when tests, performance, and artifacts match.

Do not migrate and refactor business logic in the same PR. Separate the engine upgrade from semantic changes.

17.9 When to stay on v1

Stay on v1.x if:

- A critical package or adapter is not v2-compatible.
- Your orchestration depends on a removed flag or artifact behavior.
- You cannot test the migration safely.
- You need a feature only available in v1 (rare going forward).

dbt v1 remains supported, but new capabilities will land in v2 first. Plan the migration rather than avoiding it.

17.10 Exercises

Exercise 17.1 - Parser test. Run `dbt parse --use-v2-parser` on your project. Fix every error and warning.

Exercise 17.2 - Static analysis. Enable `static_analysis: strict` on one mart. Fix the errors. What does column-level lineage show?

Exercise 17.3 - Batch tests. Run `dbt test --batch-tests` and compare the number of queries/runtime with a normal run.

Exercise 17.4 - Lint. Run `dbt lint` on a model. Fix three violations with `--fix` and review the diff.

Exercise 17.5 - Migration plan. Write a one-page plan for migrating your project to v2, including risks, owners, and rollback.

Answers. 17.1: Typical errors: unknown config keys, missing vars, invalid YAML anchors. 17.2: Strict mode requires explicit columns and resolvable references; docs show column lineage. 17.3: Expect fewer queries and lower test runtime on compatible tests. 17.4: Review auto-fixes; formatting changes can obscure logic changes. 17.5: Include a parallel-run period and a rollback to v1.

17.11 Interview check

- What is dbt v2, and how does it differ from dbt Core 1.x?
- What is static analysis, and why does it matter?
- How do you migrate a project safely?
- What breaks in strict validation?
- When would you use `--use-v2-parser`?
- How does dbt v2 change CI/CD?

PART IX - CAPSTONE

18. Capstone: Build the ClearPay Lakehouse End to End

18.1 Scenario

You are the analytics engineer for **ClearPay**, a payments company. The commerce platform lands raw orders, customers, and line items in a Databricks bronze schema. Finance needs daily revenue, refunds, and customer lifetime value. Operations needs late-order and refund monitoring. The data team wants a governed, tested, documented dbt project with CI/CD and a small semantic layer.

Your job: take the raw data from zero to a production-ready dbt project, then defend your design decisions.

18.2 What you will build

```
analytics_lab/
├── bronze/                 # raw tables (sources)
├── silver/                 # staging, intermediate, snapshots
└── gold/                   # facts, dimensions, reports, metrics
```

Models (minimum):

Layer	Model	Grain	Materialization
Staging	`stg_orders`	one row per order	view
Staging	`stg_customers`	one row per customer	view
Staging	`stg_lineitems`	one row per order line	view
Staging	`stg_refunds`	one row per refund	view
Intermediate	`int_orders_enriched`	one row per order	view
Intermediate	`int_order_line_totals`	one row per order	view
Intermediate	`int_customer_order_history`	one row per customer	view
Mart	`fct_orders`	one row per order	incremental
Mart	`fct_refunds`	one row per refund	incremental
Mart	`dim_customers`	one row per customer	table
Mart	`dim_payment_methods`	one row per method	table
Report	`rpt_revenue_daily`	one row per day	table or MV
Report	`rpt_customer_lifetime_value`	one row per customer	table

Plus:

- Source freshness on bronze tables.
- Generic tests on all keys and relationships.
- Singular tests for business rules (no negative revenue, refunds <= order total, no future dates).

- A unit test for the order-line aggregation.
- A contract on `fct_orders` and `dim_customers`.
- A snapshot of customers (SCD2).
- A materialized view or streaming table (choose one and justify).
- Semantic model + 3 metrics.
- Documentation with descriptions, exposures, and lineage.
- CI pipeline using `state:modified+`.
- An AI-assisted change, reviewed and validated.

18.3 Dataset setup

If `samples.tpch` is available:

```
CREATE OR REPLACE TABLE analytics_lab.bronze.orders_raw AS
SELECT *, current_timestamp() AS _ingested_at
FROM samples.tpch.orders;

CREATE OR REPLACE TABLE analytics_lab.bronze.customer_raw AS
SELECT *, current_timestamp() AS _ingested_at
FROM samples.tpch.customer;

CREATE OR REPLACE TABLE analytics_lab.bronze.lineitem_raw AS
SELECT *, current_timestamp() AS _ingested_at
FROM samples.tpch.lineitem;
```

Create the seeds:

`seeds/payment_methods.csv`:

```
payment_method_code,payment_method_name,is_refundable
1,Credit Card,true
2,Bank Transfer,true
3,Wallet,false
4,Cash,false
```

`seeds/refunds.csv`:

```
refund_id,order_id,refund_date,refund_amount,refund_reason
9001,1,2024-01-15,10.00,damaged
9002,2,2024-01-20,5.50,late_delivery
9003,3,2024-02-01,20.00,customer_request
```

`seeds/fx_rates.csv` (optional):

```
currency,rate_to_usd,rate_date
USD,1.0,2024-01-01
EUR,1.08,2024-01-01
GBP,1.27,2024-01-01
```

If `samples.tpch` is unavailable, use any small CSV and adapt column names. The architecture matters more than the exact data.

18.4 Milestones

Milestone 0 - Project and repository

- Fix the project name/config mismatch (`dbt_dbx` vs `databricks_dbt`).
- Add `.gitignore` for `.venv`, `target/`, `dbt_packages/`, `logs/`, `profiles.yml`, `.env`.
- Add `requirements.txt` with pinned `dbt-databricks`.
- Commit a README with setup steps.
- Create `dev_<yourname>` schema for development.

Done when: `dbt debug` passes and a trivial model builds into your dev schema.

Milestone 1 - Sources and staging

- Declare sources with descriptions, `loaded_at_field`, and freshness.
- Build the four staging models with explicit casts and deduplication where needed.
- Add PK tests (`unique`, `not_null`) and accepted values for statuses.
- Run `dbt build -s staging`.

Done when: all staging tests pass, and source freshness returns PASS/WARN with an explanation.

Milestone 2 - Intermediate and marts

- Build the intermediates and marts in the table above.
- Add `relationships` tests from facts to dimensions.
- Add a singular test for negative revenue and future dates.
- Run `dbt build`.

Done when: the DAG builds, tests pass, and `rpt_revenue_daily` reconciles to `fct_orders` for three sample days.

Milestone 3 - Contracts and unit tests

- Add contracts to `fct_orders` and `dim_customers` with explicit `data_types`.
- Add a unit test for `int_order_line_totals` (two lines for one order).
- Break a column type intentionally and confirm the contract fails; then fix it.

Done when: contract and unit tests run in CI, and you can explain what each catches.

Milestone 4 - Snapshot

- Add an `updated_at` column to `stg_customers` (or use the check strategy).
- Create `customers_snapshot` with `unique_key='customer_id'`.
- Update a customer's market segment in bronze, rerun the snapshot, and show the history.
- Add a test for one current row per customer.

Done when: the snapshot contains old and new versions with correct `dbt_valid_from`/`dbt_valid_to`.

Milestone 5 - Incremental fact with late data

- Convert `fct_orders` to incremental with `merge` on `order_id`.
- Add a 3-day lookback.
- Simulate a late order and prove it is picked up.

- Run twice without changes and prove idempotency.
- Run `--full-refresh` and compare counts.

Done when: no duplicate keys, late data handled, and the full refresh matches the incremental result.

Milestone 6 - Databricks optimization

Choose **one**:

- A materialized view `mv_revenue_daily` over `rpt_revenue_daily` with a schedule.
- A streaming table over JSON events in a Unity Catalog volume.
- Liquid clustering on `fct_orders` by `customer_id`.

Add `tblproperties` for auto-compaction. Measure query performance before/after.

Done when: you can explain the trade-off and show a before/after metric.

Milestone 7 - Semantic layer

- Add a semantic model to `fct_orders` with `order` primary entity, `customer` foreign entity, time dimension `order_date`, and simple metrics `order_revenue`, `order_count`.
- Add a semantic model to `dim_customers` with `customer` primary entity and `market_segment` dimension.
- Add `average_order_value` as a ratio metric.
- Query revenue by market segment and by month.
- Document the metric definitions.

Done when: two consumers (for example, a CLI query and a BI tool) return the same revenue for the same period.

Milestone 8 - CI/CD

- Add a GitHub Actions workflow using a service principal.
- Use `dbt build -s state:modified+ --defer --state <prod-manifest>`.
- Add a production job (dbt platform, Databricks Workflows, or Airflow).
- Add failure notifications.
- Add a cleanup step for CI schemas.

Done when: a PR runs a slim CI build and a merge triggers production.

Milestone 9 - Observability and docs

- Add exposures for the finance dashboard and the operations alert.
- Generate docs and verify lineage from source to exposure.
- Add Elementary or a simple artifact dashboard for test results.
- Add a runbook: what to do when a test fails.

Done when: a new team member can navigate from a dashboard to the source table using docs.

Milestone 10 - Governance

- Add groups/owners and tags to all marts.
- Add `databricks_tags` and a column mask for a PII column.
- Grant `SELECT` on gold to an analyst group.

- Make `fct_orders` and `dim_customers` public with contracts.
- Run `dbt_project_evaluator` and fix the top five issues.

Done when: access is least-privilege and ownership is visible in docs.

Milestone 11 - AI and v2 readiness

- Use Wizard, Copilot, or an MCP-connected agent to add one model with tests and docs. Review the diff and validate with `dbt build`.
- Configure an agent skill with your conventions.
- Run `dbt parse --use-v2-parser`; fix any issues.
- Write a short evaluation: what the AI did well, what it missed, and how you verified it.

Done when: the AI-assisted change is merged through the normal PR process with human review.

Milestone 12 - Present and defend

Prepare a 15-minute walkthrough:

1. Architecture and layering.
2. Grain and key decisions for each mart.
3. Testing strategy and what would wake you up at night.
4. Incremental/late-data handling.
5. Cost/performance choices.
6. Governance and access.
7. What you would do next with more time.

Done when: you can answer the interview questions in Part X about your own project.

18.5 Acceptance rubric

Area	Needs work (1)	Good (3)	Excellent (5)
Modeling	Duplicated logic, unclear grain	Layered, named consistently, documented	Clear contracts, reusable intermediates, justified materializations
Testing	Only PK tests	PK/FK, accepted values, business rules	Unit tests, contracts, freshness, failure ownership, cheap tests
Incremental	Full rebuild only	Merge with key	Late data, idempotency, backfill plan, monitoring
Databricks	Default table/view	Partition/cluster or MV	Measured performance/cost, correct UC governance
CI/CD	Manual runs	PR build + prod job	Slim CI, deferral, alerts, rollback
Docs	Model names only	Descriptions + lineage	Exposures, ownership, runbook, metric definitions
Security	Broad grants	Group grants	Masks/row filters, least privilege, audit
AI	None	Used for a small task	Validated, governed, with a skill and evaluation

Target: at least “Good” in every area, “Excellent” in three.

18.6 Validation queries

```sql
-- Row counts by layer
select 'stg_orders' as model, count(*) from analytics_lab.silver.stg_orders
union all
select 'fct_orders', count(*) from analytics_lab.gold.fct_orders
union all
select 'dim_customers', count(*) from analytics_lab.gold.dim_customers;

-- Duplicate keys
select order_id, count(*) c
from analytics_lab.gold.fct_orders
group by 1 having c > 1;

-- Orphan customers
select count(*)
from analytics_lab.gold.fct_orders f
left join analytics_lab.gold.dim_customers d using (customer_id)
where d.customer_id is null;

-- Revenue reconciliation
select
    (select sum(order_total) from analytics_lab.gold.fct_orders) as fact_revenue,
    (select sum(revenue) from analytics_lab.gold.rpt_revenue_daily) as report_revenue;
```

18.7 Stretch goals

- Add a second source system (for example, payments or refunds from a JSON volume) and a streaming table.
- Implement SCD2 for products.
- Add a dbt Mesh project for finance with a public `fct_orders`.
- Build a metrics API or BI dashboard on the Semantic Layer.
- Add a data contract with a schema registry.
- Implement cost anomaly alerts from `system.billing.usage`.
- Write a migration plan to dbt v2 and run CI in parallel.

18.8 Capstone questions

1. Why did you choose view/table/incremental/MV for each model?
2. What is the grain of each mart, and how is it enforced?
3. Which test would catch a 10% revenue drop? Which would not?
4. How does your pipeline handle a late-arriving order?
5. What happens if the source schema adds a column?
6. How do you roll back a bad deployment?
7. Where is PII, and who can see it?
8. What is your cost per run, and how would you reduce it by 30%?
9. Which parts of the project would break if you moved to dbt v2 today?
10. What did the AI get wrong, and how did you catch it?

PART X - MASTERY

19. Interview Questions and Answers (100+)

Use these for self-testing. Cover the answer, then explain it out loud as if you were in an interview. Answers are deliberately concise; add examples from your capstone.

19.1 Fundamentals

#	Question	Answer
1	What is dbt?	A transformation framework that compiles Jinja+SQL, builds a DAG, executes models/tests in dependency order, and produces docs/artifacts inside the warehouse.
2	What does dbt not do?	Extract/load data, store data, replace a general-purpose scheduler or BI tool.
3	ETL vs ELT?	ETL transforms before loading; ELT loads raw then transforms in the warehouse. dbt is an ELT transformation tool.
4	What is a model?	A SQL `SELECT` file that dbt persists as a view/table/incremental/etc.
5	What does `ref()` do?	Resolves the correct relation for the target and adds a DAG dependency.
6	`ref()` vs `source()`?	`ref()` points to a dbt model; `source()` points to declared raw tables.
7	What is a DAG?	A directed acyclic graph of nodes and dependencies; dbt uses it for ordering and parallelism.
8	What is a materialization?	The strategy for persisting a model (view, table, incremental, ephemeral, MV, streaming table).
9	What is a profile?	Connection/environment configuration (`profiles.yml`): adapter, host, catalog, schema, credentials, threads.
10	What is the target?	The active output in a profile (`dev`, `ci`, `prod`).
11	What is an adapter?	A plugin that translates dbt operations into a warehouse's SQL/API (e.g., dbt-databricks).
12	What is a macro?	A reusable Jinja function that returns SQL or a value.
13	What is a package?	A distributable collection of macros/models/tests installed via `packages.yml`.

#	Question	Answer
14	What is an artifact?	Generated metadata: `manifest.json`, `run_results.json`, `catalog.json`, `sources.json`.
15	Why version-control dbt?	Reviewable logic, reproducibility, CI, rollback, collaboration.

19.2 Modeling

#	Question	Answer
16	What is a staging model?	A 1:1-ish model over a source that renames, casts, and cleans; no business logic.
17	What is an intermediate model?	Reusable business logic/joins between staging and marts; often not exposed to BI.
18	What is a mart?	A consumption-ready fact, dimension, or report model.
19	What is grain?	The meaning of one row (e.g., one row per order). It must be declared and tested.
20	Fact vs dimension?	Facts are events/measurements (orders, payments); dimensions are descriptive context (customer, product).
21	Surrogate key vs natural key?	Surrogate is generated (hash/sequence); natural is from the business data.
22	Why avoid `select *` in marts?	Hides schema changes, breaks contracts, scans unused columns, makes lineage unclear.
23	How do you deduplicate?	`row_number() over (partition by key order by updated_at desc)` + filter = 1, or `qualify`.
24	How do you handle fan-out?	Check cardinality before joining; aggregate to the target grain; test uniqueness after joins.
25	What is a slowly changing dimension?	A dimension that changes over time; Type 2 preserves history with valid-from/valid-to.
26	Star schema vs wide table?	Star is normalized for reuse/governance; wide tables are denormalized for a specific consumer.
27	Where does business logic live?	In intermediate/marts or macros, never duplicated across models.
28	How do you name models?	`stg_`, `int_`, `fct_`, `dim_`, `rpt_`; plural entities; `_id`, `_at`, `_date`, `is_` conventions.
29	How do you choose a materialization?	Based on size, freshness, query frequency, recompute cost, and warehouse features.
30	What is an ephemeral model?	A model inlined as a CTE; no relation, not directly queryable/testable.

19.3 Jinja, macros, packages

#	Question	Answer
31	Why use Jinja?	Reuse logic, parameterize environments, generate SQL dynamically.
32	`{{ }}` vs `{% %}`?	Output vs statement/control flow.
33	What is `is_incremental()`?	True only when the target exists, is incremental, and this is not a full refresh.
34	What is `this`?	The current model's relation object.
35	What does `run_query` do?	Executes SQL and returns a result table; guard with `execute`.
36	Why is `execute` important?	During parsing, dbt does not run queries; introspection should be skipped.
37	What is `adapter.dispatch`?	Selects an adapter-specific macro implementation for cross-platform projects.
38	Macro vs model for logic?	Macros for reusable SQL fragments; models for persisted, testable transformations.
39	How do you test a macro?	Use it in a model with known inputs, unit-test the model, or `dbt run-operation` for side effects.
40	What is `dbt_utils.generate_surrogate_key`?	A null-safe hash over a list of columns.
41	Risk of packages?	Supply-chain risk, version drift, incompatible macros, hidden introspection cost.
42	How do you pin packages?	Exact version or narrow range; commit `package-lock.yml`; test upgrades.

19.4 Testing and contracts

#	Question	Answer
43	How does a dbt test work?	A SQL query returns failing rows; zero rows = pass.
44	Generic vs singular test?	Generic is reusable via YAML (`unique`, `relationships`); singular is a one-off SQL file in `tests/`.
45	What are the four built-in tests?	`unique`, `not_null`, `accepted_values`, `relationships`.
46	How do you make a test warn only?	`severity: warn` in the test config.
47	What does `store_failures` do?	Writes failing rows to a table for debugging.
48	Unit test vs data test?	Unit tests use fixed inputs to test logic; data tests validate production data.
49	What does a contract enforce?	Column names/types (and constraints) at build time; it fails if the model doesn't match.

#	Question	Answer
50	Where are contracts supported?	Table, view, incremental, materialized view; not ephemeral.
51	How do you test an incremental model?	Run twice, check idempotency/duplicate keys, test late data, and compare to a full refresh.
52	How do you keep tests cheap?	Filter recent partitions, test smaller staging models, use unit tests, batch tests, and thresholds.
53	What is source freshness?	A timestamp-based check that raw data is recent; not a substitute for data tests.
54	What is a flaky test?	A test whose result depends on time/order/volatile data; fix by fixing dates or isolating data.
55	What test catches duplicate joins?	`unique` on the expected grain + `relationships` on foreign keys.

19.5 Sources, seeds, snapshots, docs

#	Question	Answer
56	Why declare sources?	Lineage, freshness, tests, environment portability, documentation.
57	When use a seed?	Small, static reference data under version control. Never large or frequently changing data.
58	How do you load a seed?	`dbt seed`; `--full-refresh` to reload.
59	What is a snapshot?	An SCD2 history table tracking row changes over time.
60	Timestamp vs check strategy?	Timestamp uses an `updated_at` column; check compares selected columns (more expensive, works without a timestamp).
61	What happens on a hard delete?	Depends on `hard_deletes/invalidate_hard_deletes`; it can be ignored or marked invalid.
62	How do you query current snapshot rows?	`where dbt_valid_to is null` (or the configured current sentinel).
63	Should you snapshot a mart?	No; snapshot a staging model closest to the source.
64	What is an exposure?	A documented downstream consumer (dashboard, app, ML model) with dependencies.
65	What is a doc block?	Reusable Markdown referenced with `{{ doc('name') }}`.
66	What is in `catalog.json`?	Warehouse metadata: columns, types, stats, and table info from `dbt docs generate`.
67	How do you measure impact?	`dbt ls -s +exposure:name` and lineage in docs.

19.6 Incremental and microbatch

#	Question	Answer
68	How does incremental work?	First run creates a table; later runs apply a strategy (merge/append/replace) to new/changed rows.
69	When is `is_incremental()` false?	First run, full refresh, or non-incremental materialization.
70	What does `unique_key` do?	Identifies rows for merge/update/delete strategies.
71	Merge without `unique_key`?	It inserts all new rows (append-like).
72	`merge` vs `delete+insert`?	Merge updates matched rows; delete+insert deletes matching keys and inserts whole records.
73	What does `replace_where` require?	Delta and `incremental_predicates`; replaces only matching rows.
74	What does `insert_overwrite` do?	Replaces entire partitions/clusters returned by the query.
75	What is `on_schema_change`?	Controls how schema changes are handled (`ignore`, `fail`, `append_new_columns`, `sync_all_columns`).
76	How do you handle late data?	Lookback window, high-water mark with buffer, merge by key, or microbatch `lookback`.
77	How do you backfill?	`--full-refresh` for a full rebuild; event-time bounds for microbatch; run in slices for cost.
78	What is microbatch?	An incremental strategy that processes independent time batches using `event_time`, `begin`, and `batch_size`.
79	Databricks microbatch implementation?	Uses `replace_where`; batches can run concurrently and be retried.
80	How do you test idempotency?	Run twice with no changes; counts/keys should not change; compare to full refresh.
81	Why can incremental drift?	Logic changes without a full refresh, late data outside the window, schema changes, or failed partial runs.
82	Incremental vs MV?	Incremental is dbt-managed DML; MV is warehouse-managed refresh. Choose based on SQL support, freshness, and control.

19.7 Databricks

#	Question	Answer
83	Three-level namespace?	`catalog.schema.table` in Unity Catalog.
84	SQL warehouse vs cluster?	Warehouse is SQL compute (default `http_path`); clusters are general Spark compute (some Python model modes).
85	Databricks default table format?	Delta Lake.

#	Question	Answer
86	Partition vs liquid clustering?	Partitioning uses directories for low-cardinality filters; liquid clustering adapts file layout for broader access patterns.
87	Can you combine partition and liquid clustering?	No.
88	MV requirements?	Unity Catalog and a serverless SQL warehouse; supported SQL/configs.
89	Streaming table requirements?	Unity Catalog + serverless SQL warehouse/Lakeflow; use `STREAM/read_files`.
90	Changing a streaming table query?	Applies to future rows only; `--full-refresh` to reprocess available data.
91	Python model materializations?	`table` and `incremental` only.
92	Python submission methods?	`all_purpose_cluster`, `job_cluster`, `serverless_cluster`, `workflow_job`.
93	How to select compute per model?	`databricks_compute` config + named `compute` entries in the profile.
94	How to apply UC grants?	`grants` config in YAML/model, or post-hooks; group-based, least privilege.
95	Row filters/column masks?	UC functions applied via `row_filter/column_mask` configs (supported materializations only).
96	Why `ConcurrentModificationException`?	Concurrent writes to the same Delta table; serialize or isolate writes.
97	How to optimize Delta tables?	`OPTIMIZE`, liquid clustering, compaction, predictive optimization, right file sizes.
98	What is `skip_optimize`?	Per-model opt-out of dbt's post-build `OPTIMIZE`; global `DATABRICKS_SKIP_OPTIMIZE` takes precedence.

19.8 Production, CI/CD, performance

#	Question	Answer
99	What is slim CI?	Building/testing only modified nodes and descendants using state + deferral.
100	What does `--defer` do?	Points unselected upstream refs at the state environment instead of rebuilding them.
101	What is `state:modified`?	A selector comparing the current project to a previous manifest; file/config changes, not data changes.
102	How do you deploy dbt?	Orchestrator (dbt platform, Databricks Workflows, Airflow) runs `dbt build` with prod credentials; artifacts/alerting.
103	What do you alert on?	Failed builds, freshness breaches, critical test failures, duration/cost anomalies.
104	How do you avoid overlapping runs?	Scheduler concurrency limits, job locks, serialized writes, separate targets.

#	Question	Answer
105	How do you find slow models?	`run_results.json`, query history, bytes scanned, shuffles, joins; sort by duration.
106	Does more `threads` always help?	No; it can overwhelm the warehouse and increase queuing. Tune to capacity.
107	How do you cut costs?	Incremental/slim builds, partition pruning, right-size compute, avoid full scans, optimize tests, cluster tables.
108	How do you roll back?	Revert the code, redeploy the previous commit, or swap schemas/views; snapshots preserve history.
109	What is dbt State?	A platform capability that skips/clones/reuses nodes based on logic and data freshness; not the same as `state:modified`.
110	How do you monitor data quality over time?	Elementary/dbt artifacts, test history, row-count anomalies, freshness dashboards.

19.9 Semantic Layer, AI, and v2

#	Question	Answer
111	What is the Semantic Layer?	A metrics layer that defines metrics/dimensions once and serves consistent SQL to BI/APIs/AI.
112	What is an entity?	A join key with a type (primary/foreign/unique/natural).
113	What is a time spine?	A date table used for cumulative/conversion metrics and time-based calculations.
114	Simple vs ratio metric?	Simple aggregates a column; ratio divides numerator by denominator metrics.
115	How do you test metrics?	Test underlying keys/grains/joins, unit-test logic, reconcile totals, validate time spine.
116	Wizard vs Copilot?	Wizard is an agent for multi-step, validated development; Copilot is inline quick actions/chat.
117	What is MCP?	Model Context Protocol: a standard for exposing dbt metadata/tools to AI agents.
118	What is an agent skill?	A `SKILL.md` file with reusable instructions for an AI coding agent.
119	What is dbt v2?	The Rust-based engine (Fusion) installed via `pip install dbt`; faster, stricter, static analysis, LSP.
120	What is static analysis?	Parsing/validating SQL semantics without executing; strict mode enables column lineage/type checks.
121	What is `--use-v2-parser`?	A dbt 1.12 opt-in flag to test v2 parser compatibility.
122	What breaks in v2?	Deprecated configs/flags, unknown YAML keys, missing vars/macros, duplicate docs, invalid anchors.

#	Question	Answer
123	When stay on v1?	Incompatible packages/adapters or unvalidated migration; v1 remains supported.

19.10 Scenario and debugging questions

#	Question	Answer
124	A dashboard shows double revenue. How do you debug?	Check grain/fan-out, duplicate keys, join cardinality, filters, and whether the dashboard sums an already-aggregated table.
125	A test passes in dev but fails in prod. Why?	Different data volume, late data, permissions, schema drift, environment-specific logic, or tests filtered in dev.
126	A model takes 40 minutes. What do you check?	Bytes scanned, joins/shuffles, materialization, partitioning, warehouse size, exploded rows, and unnecessary columns.
127	A source added a column and the pipeline broke. Why?	`select *`/contract mismatch/`on_schema_change: fail`; make columns explicit and choose schema policy.
128	A snapshot has duplicate current rows. Why?	Non-unique `unique_key`, duplicate source rows, or concurrent snapshot runs; deduplicate and test.
129	Incremental row counts dropped after a rerun. Why?	The filter selected a narrower window; `merge` deleted/replaced rows, or a full refresh rebuilt from partial data.
130	CI is slow and expensive. How do you fix it?	Slim CI, state/deferral, smaller warehouse, selective tests, `--empty` for schema checks, caching deps.
131	An AI agent proposes a 20-file refactor. What do you do?	Ask for a plan and scope, split into small changes, review diffs, run targeted builds/tests, and never auto-merge.
132	Two developers built into the same schema. What went wrong?	Shared target schema; use per-developer schemas or isolated catalogs and `generate_schema_name`.
133	A production job succeeds but data is stale. Why?	Source freshness not checked, incremental filter missed late data, upstream job failed silently, or the MV/stream didn't refresh.
134	You must change a public model's column type. How?	Version the model or add a new column, contract it, communicate, deprecate the old version, and migrate consumers.

19.11 Ten coding tasks

Task 1 - Deduplicate latest record.

```sql
select *
from (
    select *,
           row_number() over (
               partition by order_id
               order by updated_at desc
           ) as rn
    from {{ ref('stg_orders') }}
)
where rn = 1
```

Task 2 - Incremental merge with a 7-day lookback.

```sql
{{ config(materialized='incremental', unique_key='order_id', incremental_strategy='merge') }}

select *
from {{ ref('stg_orders') }}

{% if is_incremental() %}
where order_date >= (
    select dateadd(day, -7, coalesce(max(order_date), '1900-01-01')) from {{ this }}
)
{% endif %}
```

Task 3 - Snapshot with check strategy.

```sql
{% snapshot customers_snapshot %}
{{ config(
    target_schema='silver',
    unique_key='customer_id',
    strategy='check',
    check_cols=['customer_name', 'market_segment']
) }}
select customer_id, customer_name, market_segment
from {{ ref('stg_customers') }}
{% endsnapshot %}
```

Task 4 - Generic test for a positive amount.

```sql
{% test positive_amount(model, column_name) %}
select * from {{ model }} where {{ column_name }} < 0
{% endtest %}
```

Task 5 - Surrogate key macro.

```sql
{% macro surrogate_key(fields) %}
    md5(concat_ws('|',
        {%- for f in fields -%}
            coalesce(cast({{ f }} as string), '')
            {%- if not loop.last %}, {% endif -%}
        {%- endfor -%}
    ))
{% endmacro %}
```

Task 6 - Fill missing dates.

```sql
with spine as (
    {{ dbt_utils.date_spine(
        datepart="day",
        start_date="to_date('2024-01-01')",
        end_date="to_date('2025-01-01')"
    ) }}
),

revenue as (
    select order_date, sum(order_total) as revenue
    from {{ ref('fct_orders') }}
    group by 1
)

select
    s.date_day,
    coalesce(r.revenue, 0) as revenue
from spine s
left join revenue r on s.date_day = r.order_date
```

Task 7 - Pivot payment methods.

```sql
select
    order_date,
    {% for method in ['Credit Card', 'Bank Transfer', 'Wallet', 'Cash'] %}
    sum(case when payment_method_name = '{{ method }}' then order_total else 0 end)
        as {{ method | lower | replace(' ', '_') }}_revenue
        {%- if not loop.last %},{% endif %}
    {% endfor %}
from {{ ref('fct_orders') }}
group by 1
```

Task 8 - Selection commands.

```
dbt build -s +fct_orders                  # ancestors + fact
dbt test -s tag:finance                   # finance tests
dbt build -s state:modified+ --defer --state ./state
dbt list -s +exposure:revenue_dashboard
```

Task 9 - CI job (minimal).

```yaml
name: dbt-ci
on: [pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: pip install -r requirements.txt
      - run: dbt deps
      - run: dbt build --select state:modified+ --defer --state state
        env:
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}
```

Task 10 - Debug fan-out.

```sql
-- Find keys with more than one row after a join
select order_id, count(*) as c
from {{ ref('fct_orders') }}
group by 1
having c > 1;

-- Inspect the join cardinality before aggregating
select count(*) as rows, count(distinct o.order_id) as orders
from orders o
join lineitems l on o.order_id = l.order_id;
```

20. Cheat Sheets, Glossary, Resources, and a 30-Day Plan

20.1 Command cheat sheet

Command	What it does
`dbt init`	Create a project/profile
`dbt debug`	Validate connection/config
`dbt deps`	Install packages
`dbt parse`	Parse and write manifest
`dbt compile`	Render SQL only
`dbt run`	Build models
`dbt test`	Run tests
`dbt build`	Seeds, models, snapshots, tests in DAG order
`dbt seed`	Load CSVs
`dbt snapshot`	Run snapshots
`dbt source freshness`	Check source timestamps
`dbt docs generate`	Build docs/catalog artifacts
`dbt docs serve`	Serve docs locally
`dbt show -s model`	Preview results
`dbt ls -s ...`	List selected nodes
`dbt run-operation macro`	Run a macro
`dbt clone`	Clone relations from state
`dbt retry`	Retry failed nodes
`dbt clean`	Remove generated files
`dbt --version`	Versions

20.2 Configuration precedence

1. In-file `{{ config(...) }}` (highest).
2. Properties YAML (`models: - name: ... config:`).
3. `dbt_project.yml` folder/project config.
4. Adapter defaults (lowest).

CLI flags override some configs at runtime (for example `--full-refresh`).

20.3 Materialization cheat sheet

Materialization	Persists?	Rebuild behavior	Use
view	Yes (view)	Recreates	Light transforms/staging
table	Yes (table)	Full rebuild	Marts, dashboards
incremental	Yes (table)	DML on new/changed rows	Large facts/events
ephemeral	No	Inlined CTE	Tiny helpers
materialized_view	Yes (MV)	Warehouse refresh	Aggregations
streaming_table	Yes (ST)	Incremental ingestion	Streams/files

20.4 Incremental cheat sheet

Strategy	Key config	Databricks
append	none	all formats
merge	`unique_key`	Delta/Hudi
delete+insert	`unique_key`, DBR 12.2+	Delta
replace_where	`incremental_predicates`	Delta, DBR 12+
insert_overwrite	`partition_by`/clustering	Delta
microbatch	`event_time`, `begin`, `batch_size`	Delta, `replace_where`

20.5 Databricks config cheat sheet

Config	Example
`materialized`	`table`, `view`, `incremental`, `materialized_view`, `streaming_table`
`file_format`	`delta`
`table_format`	`iceberg`
`partition_by`	`['order_date']`
`liquid_clustered_by`	`['customer_id']`
`auto_liquid_cluster`	`true`
`tblproperties`	`{'delta.autoOptimize.optimizeWrite': 'true'}`
`databricks_tags`	`{'domain': 'finance'}`
`column_mask`	`{function: 'cat.sch.mask_email'}`
`row_filter`	`{function: 'cat.sch.region_filter', columns: [region]}`
`databricks_compute`	`large`
`grants`	`{select: ['analyst_role']}`
`schedule`	`{cron: '0 0 * * * ? *'}` or `{every: '2 HOURS'}`

20.6 Jinja context cheat sheet

Name	Use
`ref('model')`	Model relation + dependency
`source('src', 'table')`	Source relation

Name	Use
`config(...)`	Set config
`var('name', default)`	Variable
`env_var('NAME')`	Environment variable
`target.name/schema/catalog`	Environment
`this`	Current relation
`is_incremental()`	Incremental guard
`model.name`	Model metadata
`run_started_at`	Run timestamp
`adapter.get_relation(...)`	Metadata lookup
`run_query(sql)`	Execute SQL
`exceptions.raise_compiler_error`	Fail compilation
`log(msg, info=True)`	Log message

20.7 Node selection cheat sheet

Selector	Meaning
`model`	Exact node
`folder`	Path/name
`tag:x`	Tag
`config.materialized:table`	Config
`source:src.table`	Source
`exposure:x`	Exposure
`test_type:unit`	Unit tests
`state:modified`	Changed vs state
`result:error`	Previously failed
`+model`, `model+`, `+model+`, `@model`	Graph operators
`--exclude`	Subtract
`--selector name`	YAML selector

20.8 Testing cheat sheet

Test	Catches
`unique`	Duplicate PKs/grain issues
`not_null`	Missing keys/required values
`accepted_values`	Unexpected statuses/categories
`relationships`	Orphan FKs/fan-out
Singular SQL	Business rules
Unit test	Logic regression with fixed inputs
Contract	Column name/type drift
Freshness	Stale ingestion

20.9 Troubleshooting cheat sheet

Symptom	First checks
Connection/auth	`dbt debug`, env var, token expiry, host/http_path
Catalog/schema not found	`SHOW CATALOGS`, grants, profile
Model in wrong schema	`generate_schema_name`, `+schema`, target
Config ignored	Project name mismatch, missing `+`, precedence
Duplicate rows	Grain, joins, incremental key, source duplicates
Test fails only in prod	Data volume, late data, filters, permissions
Slow run	Full scans, joins, threads, warehouse size, tests
MV/ST errors	UC + serverless requirement, unsupported SQL/config
Python model fails	Notebook permissions, cluster config, packages, timeout
CI fails	Secrets, service principal grants, state manifest path

20.10 Glossary

Term	Definition
Adapter	Plugin translating dbt operations to a warehouse.
Analytics engineering	The discipline of building tested, documented transformation layers.
Artifact	Generated metadata files in `target/`.
Catalog (dbt)	The docs/catalog artifact describing warehouse metadata.
Catalog (UC)	Top-level Unity Catalog container.
Contract	Enforced column names/types for a model.
CTE	Common table expression (`with ... as`).
DAG	Directed acyclic graph of dependencies.
Deferral	Using state relations for unselected upstream refs.
Dimension	Descriptive attribute/table for grouping.
Ephemeral	Model inlined as a CTE, not persisted.
Exposure	Documented downstream consumer.
Fact	Event/measurement table.
Freshness	Timestamp-based recency check.
Generic test	Reusable YAML test (`unique`, etc.).
Grain	Meaning of one row.
Group	Ownership/access boundary.
Incremental	Materialization processing new/changed data.
Jinja	Templating language used by dbt.
Macro	Reusable Jinja function.
Manifest	`manifest.json`, the full parsed project graph.
Mart	Consumption-ready model.
Materialization	Persistence strategy.
Metric	Business calculation defined in the Semantic Layer.
MetricFlow	dbt's metric query engine.

Term	Definition
Microbatch	Time-batch incremental strategy.
Model	SQL file that dbt runs.
Model version	A versioned interface for a model.
Node	Any dbt resource (model, test, seed, snapshot, etc.).
Package	Reusable dbt code installed via `packages.yml`.
Parse	Reading/rendering the project into a manifest.
Partition	Physical data subdivision.
Primary key	Unique identifier for a row.
Profile	Connection/environment config.
Project	The dbt codebase and `dbt_project.yml`.
Ref	Reference to another model.
Run results	`run_results.json` execution metadata.
Schema	Namespace for tables (and YAML test file).
Seed	CSV loaded by dbt.
Semantic layer	Metrics/dimensions layer for consistent consumption.
Semantic model	Model annotated with entities/dimensions/metrics.
Singular test	One-off SQL test in `tests/`.
Snapshot	SCD2 history table.
Source	Declared raw table.
Staging	Cleaning/renaming layer.
State	Previous manifest used for comparison.
Static analysis	SQL validation without execution (v2).
Streaming table	Databricks incremental ingestion table.
Surrogate key	Generated key.
Target	Active profile output.
Time spine	Date table for metric calculations.
Unit test	Fixed-input test of model logic.
Var	Project/CLI variable.
View	Virtual table.
YAML selector	Named, version-controlled selection.
dbt State	Platform capability for skip/clone/reuse.
dbt Wizard	Agentic AI for development.
dbt Copilot	Inline AI assistance.
MCP	Model Context Protocol for AI tools.

20.11 Resources

- dbt docs: https://docs.getdbt.com
- dbt Learn courses: https://learn.getdbt.com
- dbt package hub: https://hub.getdbt.com
- dbt Community Slack: https://www.getdbt.com/community/join
- dbt certification: https://www.getdbt.com/dbt-certification

- dbt MCP server: https://github.com/dbt-labs/dbt-mcp
- Databricks dbt docs: https://docs.databricks.com (search "dbt")
- Databricks Unity Catalog: https://docs.databricks.com/data-governance/unity-catalog
- Delta Lake: https://docs.delta.io
- MetricFlow: https://docs.getdbt.com/docs/build/about-metricflow
- dbt GitHub: https://github.com/dbt-labs/dbt-core
- dbt-databricks: https://github.com/databricks/dbt-databricks
- SQL style guide: https://www.sqlstyle.guide
- Use The Index, Luke (SQL performance): https://use-the-index-luke.com

20.12 Certification and skill validation

The dbt Analytics Engineering Certification tests:

- dbt fundamentals and project structure.
- Modeling and materializations.
- Jinja/macros/packages.
- Testing, documentation, and sources.
- Incremental models and snapshots.
- Deployment and environments.

To prepare:

1. Finish the capstone.
2. Do the interview questions without notes.
3. Practice explaining trade-offs, not just syntax.
4. Build a small project from an unfamiliar dataset.
5. Review the official docs for anything you use daily.

20.13 30-day plan

Day	Focus	Task
1	Foundations	Read Chapters 1-2; run `dbt debug`; build `stg_orders`
2	Models/DAG	Chapter 3; build 3 staging models; inspect lineage
3	Config/schemas	Fix project name; override schema macro; per-dev schema
4	Layering	Chapter 4; build intermediates and 2 marts
5	Tests	Add PK/FK tests; run `dbt build`; fix failures
6	Docs	Descriptions, doc blocks, exposures; `dbt docs serve`
7	Review	Rebuild from scratch; write a README
8	Jinja	Chapter 5; write 3 macros; use one in a model
9	Packages	Chapter 6; install dbt_utils; use surrogate key/star

Day	Focus	Task
10	Testing	Chapter 7; singular test, custom generic test
11	Unit tests	Add a unit test; break/fix logic
12	Contracts	Contract `fct_orders`; cause and fix a mismatch
13	Sources/seeds	Chapter 8; freshness, seed lookup
14	Snapshots	Build SCD2; update a row; query history
15	Incremental	Chapter 9; merge, idempotency, backfill
16	Microbatch	Build a microbatch model; inspect batches
17	Databricks	Chapter 10; partition/cluster, tags; inspect DDL
18	MV/ST	Build an MV or streaming table; test refresh
19	Python	Build a simple Python model; compare compute
20	Selection/state	Chapter 11; YAML selector, `state:modified+`
21	CI	Chapter 12; GitHub Actions slim CI
22	Production	Deploy a scheduled job; add alerts
23	Performance	Find 5 slow nodes; optimize one; measure
24	Governance	Groups, grants, mask; run evaluator
25	Semantic	Chapter 15; semantic model + 3 metrics
26	AI	Chapter 16; Wizard/MCP lab; agent skill
27	v2	Chapter 17; `--use-v2-parser`; static analysis
28	Capstone	Complete milestones 0-6
29	Capstone	Complete milestones 7-12
30	Interview	Answer 100+ questions; present your project

20.14 Final checklist

- ☐ I can explain dbt's compilation/execution model.
- ☐ I can design staging/intermediate/marts with clear grains.
- ☐ I can choose and justify a materialization.
- ☐ I can write and debug Jinja macros.
- ☐ I can test keys, relationships, and business rules.
- ☐ I can use contracts and unit tests.
- ☐ I can build sources, seeds, snapshots, and docs.
- ☐ I can design an incremental model with late-data handling.
- ☐ I can use Databricks MVs, streaming tables, and Python models.
- ☐ I can select nodes and build a slim CI pipeline.
- ☐ I can optimize performance and cost.
- ☐ I can implement governance and least privilege.

- [ ] I can define and query Semantic Layer metrics.
- [ ] I can use dbt AI features safely and critically.
- [ ] I can migrate to dbt v2 with a tested plan.
- [ ] I can explain every choice in my capstone.

You are not done when you finish reading. You are done when you can build it, break it, explain it, and teach it. Go build.